Contents

1 Specifica **6**

1.1 Introduzione Descrizione Generale 6
1.2 Modellazione per casi d'uso 6
1.3 Glossario dei termini di dominio 7
1.4 Requisiti funzionali 8
1.4.1 RF01 - Servizi offerti agli utenti 8
1.4.1.1 RF01.1 - Visualizzazione della situazione corrente 8
1.4.1.1.1 RF01.1.1 - Mappa del traffico 9
1.4.1.1.2 RF01.1.2 - Elenco dei convogli . . . 9
1.4.1.1.3 RF01.1.3 - Elenco delle stazioni . . . 9
1.4.1.1.4 RF01.1.4 - Aggiornamento continuo 10
1.4.1.1.5 RF01.1.5 - Indicatori di sintesi . . . 10
1.4.1.2 RF01.2 - Visualizzazione dei guasti 10
1.4.1.2.1 RF01.2.1 - Elenco degli allarmi aperti 10
1.4.1.2.2 RF01.2.2 - Distinzione fra guasto segnalato e guasto dedotto 11
1.4.1.3 RF01.3 - Amministrazione dei dati della rete 11
1.4.1.3.1 RF01.3.1 - Stazioni 11
1.4.1.3.2 RF01.3.2 - Convogli 12
1.4.1.3.3 RF01.3.3 - Tratte elementari 12
1.4.1.3.4 RF01.3.4 - Itinerari e assegnazione dei convogli 12
1.4.1.3.5 RF01.3.5 - Integrita' dei dati alle eliminazioni 13
1.4.1.4 RF01.4 - Comandi operativi 13
1.4.1.4.1 RF01.4.1 - Invio della squadra di manutenzione 13
1.4.1.4.2 RF01.4.2 - Chiusura dell'allarme e ripresa della circolazione 14
1.4.1.4.3 RF01.4.3 - Soppressione di un convoglio fermo in stazione 15
1.4.1.5 RF01.5 - Consultazione dello storico 15
1.4.1.6 RF01.6 - Accesso al sistema 15
1.4.2 RF02 - Correttezza dello stato rappresentato 16
1.4.2.1 RF02.1 - Gestione dei guasti 16
1.4.2.1.1 RF02.1.1 - Guasto della stazione . . 16

1.4.2.1.2 RF02.1.2 - Guasto del convoglio . . 18
1.4.2.1.3 RF02.1.3 - Ciclo di vita del guasto . 19
1.4.2.1.4 RF02.1.4 - Chiusura automatica quando la causa rientra 19
1.4.2.2 RF02.2 - Gestione dei convogli 20
1.4.2.2.1 RF02.2.1 - Ritardi 20
1.4.2.2.2 RF02.2.2 - Informazioni di stato del convoglio 20
1.4.2.2.3 RF02.2.3 - Andata e ritorno sullo stesso itinerario 21
1.4.2.2.4 RF02.2.4 - Passaggi asimmetrici in stazione 22
1.4.2.3 RF02.3 - Gestione delle stazioni 22
1.4.2.3.1 RF02.3.1 - Informazioni rese dalla stazione 22
1.4.2.3.2 RF02.3.2 - Segnale di vita verso la centrale 23
1.4.2.3.3 RF02.3.3 - Segnale di vita dei sensori verso la stazione 23
1.4.2.3.4 RF02.3.4 - Stato che la stazione dichiara di se' 23
1.4.2.4 RF02.4 - Identita' del nodo di campo 23
1.4.2.4.1 RF02.4.1 - L'identita' dichiarata viene verificata 23
1.4.2.4.2 RF02.4.2 - Esito negativo: il nodo non entra in servizio 24
1.4.2.4.3 RF02.4.3 - Portata della dipendenza 24
1.4.2.5 RF02.5 - Modifica dell'itinerario a sistema acceso 24
1.4.2.5.1 RF02.5.1 - Convoglio escluso dall'itinerario 24
1.4.2.5.2 RF02.5.2 - Convoglio ancora assegnato 25
1.4.2.6 RF02.6 - Anomalie dedotte dalla centrale . . 25
1.4.2.6.1 RF02.6.1 - Stazione silenziosa oltre la soglia 26
1.4.2.6.2 RF02.6.2 - Convoglio fermo fuori stazione oltre la soglia 26
1.4.2.6.3 RF02.6.3 - Fotografia periodica dello stato 26
1.4.2.7 RF02.7 - Memoria storica 27

1.4.3 RF03 - Contratti di interazione fra le parti 27
1.4.3.1 RF03.1 - Autonomia dei nodi 27
1.4.3.2 RF03.2 - Disaccoppiamento fra campo e centrale 28
1.4.3.3 RF03.3 - Due fonti di verita' distinte per lo stesso passaggio 28
1.4.3.4 RF03.4 - I sensori usano solo l'interfaccia pubblica del nodo 29
1.4.3.5 RF03.5 - L'itinerario e' fornito dalla centrale 29
1.4.4 RF04 - Accesso e protezione 30
1.4.4.1 RF04.1 - Autenticazione 30
1.4.4.2 RF04.2 - Autorizzazione per ruolo 30
1.4.4.3 RF04.3 - Protezione dei canali 31
1.5 Requisiti non funzionali osservabili 31
1.6 Scenari di verifica 32
1.6.1 SV01 - Stazione non percorribile e successivo ripristino 32
1.6.2 SV02 - Interruzione del collegamento fra stazione e centrale 33
1.6.3 SV03 - Guasto del convoglio, in stazione e in tratta . . 33
1.6.4 SV04 - Convoglio fermo fra due stazioni 34
1.6.5 SV05 - Modifica dell'itinerario a convogli in marcia . . 34
1.6.6 SV06 - Corsa completa con inversione al capolinea . . 35
1.6.7 Copertura dei requisiti da parte degli scenari 35

2 Progettazione **36**
2.1 Diagramma delle componenti 36
2.2 Diagramma dei microservizzi 37
2.3 Stack tecnologico 37
2.4 Scelte progettuali e giustifiacazioni 39
2.4.1 Metodi di interfacciamento tra microservizzi 39
2.4.2 Sul metodo di repository 40
2.4.3 Sul sistema di validazione 40
2.5 Parametri temporali 41
2.6 Modello 43
2.6.1 Il database della Centrale 43
2.6.2 I db locali dei nodi di campo 45
2.7 Topic 46
2.8 Vocabolario degli stati osservabili 49
2.8.1 Stato del convoglio 49
2.8.2 Stato operativo della stazione 49

2.8.3 Stato composto mostrato dall'interfaccia 50
2.9 Diagrammi degli stati 51
2.9.1 Del Treno 51
2.9.1.1 mappa delle macchine 52
2.9.1.2 T1 ciclo di vita del processo treno 53
2.9.1.3 T2 validazione dell'id sul database centrale . 54
2.9.1.4 T3 caricamento dell'itinerario 55
2.9.1.5 T4 tick e gestore degli eventi 56
2.9.1.6 T5 fase del viaggio 57
2.9.1.7 T6 condizione del convoglio 58
2.9.1.8 T7 direzione di marcia e capolinea 59
2.9.1.9 T8 telemetria verso la centrale 60
2.9.1.10 T9 alert in ingresso dalla centrale 61
2.9.1.11 T10 sensori di bordo e api rest 62
2.9.1.12 T11 stato delle stazioni noto al treno 62
2.9.2 Della Stazione 63
2.9.2.1 mappa delle macchine 64
2.9.2.2 S1 ciclo di vita del processo stazione 64
2.9.2.3 S2 validazione dell'id sul database centrale . 65
2.9.2.4 S3 heartbeat verso la centrale 66
2.9.2.5 S4 connettivita' verso la centrale e store and forward 66
2.9.2.6 S5 ciclo di vita di un evento della stazione . . 67
2.9.2.7 S6 api rest dei sensori di stazione 68
2.9.2.8 S7 sorveglianza dei sensori di binario 69
2.9.2.9 S8 ingestione dei passaggi dei treni 69
2.9.2.10 S9 alert in ingresso dalla centrale 70
2.9.2.11 S10 stato operativo della stazione 71
2.9.2.12 S11 presenza di un treno sui binari 72
2.9.3 Del Server Centrale 72
2.9.3.1 mappa delle macchine 73
2.9.3.2 M1 ciclo di vita del processo della centrale . 74
2.9.3.3 M2 validazione dell'id dei nodi di campo . . . 74
2.9.3.4 M3 ingestione della telemetria dei treni . . . 75
2.9.3.5 M4 ingestione degli heartbeat delle stazioni . 76
2.9.3.6 M5 ingestione dei transiti 77
2.9.3.7 M6 ingestione degli allarmi dal campo 78
2.9.3.8 M7 ciclo di vita del guasto 79
2.9.3.9 M8 watchdog della centrale 80

2.9.3.10 M9 ciclo di vita del token e autorizzazione delle api rest . . . 81
2.9.3.11 M10 gateway rest, crud e comandi operativi . 82
2.9.3.12 M11 canale websocket realtime . . . 83
2.9.3.13 M12 stato runtime della stazione . . . 84
2.9.3.14 M13 stato del treno . . . 85
2.9.4 Riepilogo requisiti - macchine a stati . . . 86
2.10 Diagrammi di sequenza . . . 89
2.10.1 mappa dei diagrammi . . . 90
2.10.2 SQ01 avvio del nodo treno e validazione dell'id . . . 90
2.10.3 SQ02 avvio del nodo stazione e validazione dell'id . . . 91
2.10.4 SQ03 caricamento dell'itinerario del convoglio . . . 92
2.10.5 SQ04 telemetria del convoglio fino alla mappa . . . 93
2.10.6 SQ05 heartbeat della stazione . . . 94
2.10.7 SQ06 passaggio in stazione, dal binario alla centrale . 95
2.10.8 SQ07 keepalive dei sensori di binario . . . 96
2.10.9 SQ08 convoglio trattenuto da una stazione guasta . . . 97
2.10.10 SQ09 guasto rilevato dai sensori di bordo . . . 99
2.10.11 SQ10 fail-stop della stazione e chiusura automatica . . 100
2.10.12 SQ11 convoglio fermo fuori stazione . . . 102
2.10.13 SQ12 store and forward durante l'interruzione del collegamento . . . 103
2.10.14 SQ13 accesso OAuth2, token e permessi per ruolo . . . 104
2.10.15 SQ14 comandi di ripristino dell'operatore . . . 106
2.10.16 SQ15 soppressione di un convoglio . . . 108
2.10.17 SQ16 modifica dell'itinerario a sistema acceso . . . 109
2.10.18 SQ17 amministrazione dell'anagrafica della rete . . . 110
2.10.19 SQ18 dashboard in tempo reale e snapshot periodico . 113
2.10.20 riepilogo requisiti - diagrammi di sequenza . . . 113
2.11 Diagrammi delle classi . . . 116
2.11.1 Il componente Treno (C1) . . . 117
2.11.2 Il componente Stazione (C2) . . . 118
2.11.3 Il componente Centrale: mappa delle tre viste . . . 119
2.11.4 Il componente Centrale: verso l'operatore (C3) . . . 120
2.11.5 Il componente Centrale: verso il campo, la porta MQTT (C4a) . . . 121
2.11.6 Il componente Centrale: il dominio, gestore degli eventi e automi (C4b) . . . 122
2.11.7 Il componente Centrale: la persistenza (C5) . . . 123
2.11.8 Il componente Web App (C6) . . . 124

2.11.9 riepilogo requisiti - diagrammi delle classi 125
2.11.10 riepilogo classi - macchine a stati 126
2.12 Interfaccia utente 133
2.12.1 Mappa della navigazione 133

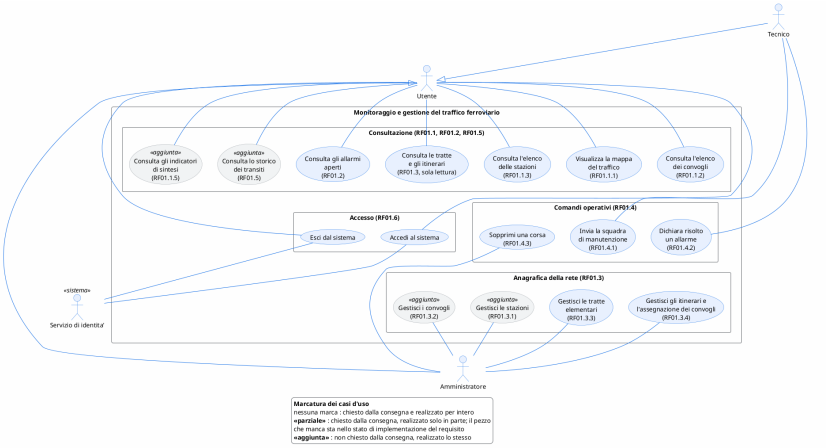
3 Validazione **134**
3.1 Come è stato verificato 134
3.2 Ambiente di prova 135
3.3 Test automatici 135
3.3.1 Le prove dei casi limite 141
3.4 Esecuzione degli scenari 142
3.5 Verifica dei requisiti non funzionali 142
3.6 Limiti della verifica 143

1 Specifica

1.1 Introduzione Descrizione Generale

Il sistema è al fine della gestione e del monitoraggio di una rete ferroviaria, in particolare delle stazioni e dei treni che la compongono. Tale servizio è Fornito attraverso una app web che fornisce il neccessarrio

1.2 Modellazione per casi d'uso



1.3 Glossario dei termini di dominio

Stazione nodo della rete in cui i convogli entrano, sostano ed escono. Ha un nome, una posizione geografica e uno stato operativo.

Tratta elementare collegamento diretto fra due stazioni adiacenti, senza fermate intermedie. E' l'unita' con cui si compone un percorso.

Itinerario sequenza ordinata di stazioni che un convoglio deve percorrere, con il tempo di percorrenza previsto fra una tappa e la successiva. Nelle schermate e' chiamato anche **tratta**.

Estremo dell'itinerario la prima e l'ultima stazione della sequenza. E' il punto in cui il convoglio inverte la marcia, a meno che quell'estremo sia anche il capolinea.

Capolinea l'estremo su cui il servizio si chiude, deciso dall'amministratore quando compone l'itinerario. Il convoglio che ci arriva termina la corsa e resta fermo li'. E' una proprieta' dell'itinerario e non della stazione: la stessa stazione puo' chiudere un itinerario ed essere una fermata qualunque di un altro. Un itinerario senza capolinea si percorre senza fine.

Direzione verso di percorrenza dell'itinerario, **andata** o **ritorno**.

Corsa percorrenza dell'itinerario da un estremo all'altro in una direzione.

Convoglio il treno in servizio, identificato da un numero. Ha una posizione, uno stato, un ritardo accumulato e un numero di passeggeri a bordo.

Transito passaggio di un convoglio per una stazione. E' di due tipi, **entrata** e **uscita**, ed e' sempre riferito a un istante.

Telemetria dichiarazione periodica con cui il convoglio comunica come sta e dove si trova.

Battito segnale periodico con cui un nodo dichiara di essere vivo. Il suo valore informativo sta nell'assenza: e' il silenzio a dire qualcosa.

Guasto condizione anomala aperta su una sorgente (una stazione, un convoglio, una tratta), con un tipo, una severita', un istante di apertura e uno stato di lavorazione.

Allarme il guasto cosi' come viene presentato all'operatore.

Ripristino dichiarazione che la condizione anomala e' rientrata e che cio' che era bloccato puo' riprendere.

Ritardo differenza fra l'avanzamento previsto della corsa e quello effettivo, espressa in minuti e riferita al singolo convoglio.

Sosta permanenza del convoglio in stazione fra l'entrata e l'uscita.

Servizio di identita' la parte del sistema che custodisce gli utenti, ne verifica le credenziali e rilascia gli attestati di accesso. E' l'unico posto in cui una password viene digitata e l'unico che la conosce.

Attestato di accesso cio' che l'utente ottiene identificandosi e che deve esibire a ogni richiesta successiva. Dice chi e' e che ruolo ha, ha una durata limitata e chi lo riceve puo' controllarne da solo l'autenticita', senza interpellare il servizio di identita'.

Rinnovo sostituzione di un attestato in scadenza con uno nuovo, fatta senza chiedere di nuovo le credenziali all'utente. Serve perche' una sessione di lavoro dura piu' di un attestato.

1.4 Requisiti funzionali

La numerazione è gerarchica per raggruppamenti e presenta anche le dipendenze

1.4.1 RF01 - Servizi offerti agli utenti

Enunciato: il sistema espone a tecnico e amministratore, attraverso un'unica applicazione web, la situazione corrente della rete, la sua storia e i comandi previsti per il loro ruolo.

Nota di lettura: RF01 non stabilisce nessuna verita' sul dominio, la **espone**. Se lo stato descritto da RF02 e' sbagliato, RF01 mostra correttamente un dato sbagliato: per questo ogni ramo di RF01 dipende dal corrispondente ramo di RF02.

Stato di implementazione: parziale — la visualizzazione, l'amministrazione dei dati e l'accesso ci sono per intero, e i tre comandi operativi di RF01.4 fanno quello che devono. Mancano la ripartizione dei comandi per ruolo (RF04.2) e la schermata dello storico (RF01.5).

1.4.1.1 RF01.1 - Visualizzazione della situazione corrente

Stato di implementazione: completo, tutte e cinque le voci.

1.4.1.1.1 RF01.1.1 - Mappa del traffico

- enunciato: la posizione dei convogli e' rappresentata su una mappa geografica, insieme al percorso che stanno seguendo e al loro stato.
- dipende da: RF02.2.2.
- stato di implementazione: completo. Il convoglio viene disegnato sulla posizione interpolata fra le due stazioni della tratta che sta percorrendo, con sopra il percorso e il colore dello stato; il pannello di dettaglio ne mostra lo stato composto (vedi il vocabolario degli stati osservabili). Se la centrale non garantisce la posizione (RF02.2.2.1) il convoglio **non viene disegnato** ma elencato a parte, con il nome e il motivo: sulla mappa una posizione inventata non si distinguerebbe da una vera.

1.4.1.1.2 RF01.1.2 - Elenco dei convogli

- enunciato: per ogni convoglio sono visibili il numero, la stazione o la tratta su cui si trova, lo stato, il ritardo accumulato e i passeggeri a bordo.
- dipende da: RF02.2.
- stato di implementazione: completo. Lo stato e' quello composto, un valore per ogni automa del convoglio (condizione, fase del viaggio, verso), e non il riassunto che ne tiene la centrale. Dove la posizione non e' garantita l'elenco scrive un punto di domanda con il motivo: il trattino vuol dire "non si applica", il punto di domanda "c'e', ma non lo sappiamo".

1.4.1.1.3 RF01.1.3 - Elenco delle stazioni

- enunciato: per ogni stazione sono visibili lo stato operativo, i convogli fermi sui binari in quel momento, quelli attesi in ingresso e quelli attesi in uscita.
- dipende da: RF02.3.1.
- stato di implementazione: completo. Lo stato operativo lo tiene la centrale, i convogli sui binari li dichiara il nodo di stazione: la schermata mette insieme le due cose. Nel tabellone degli attesi il convoglio a posizione incerta resta in elenco, con un avviso accanto all'orario: la stima e' calcolata sull'ultima posizione certa.

1.4.1.1.4 RF01.1.4 - Aggiornamento continuo

- enunciato: le schermate si aggiornano da sole mentre l'operatore le guarda, senza che lui debba ricaricare la pagina o chiedere l'aggiornamento.
- esito atteso: un evento rilevante (un transito, un cambio di stato, un guasto, l'esito di un comando) e' visibile sulla schermata entro pochi secondi dal momento in cui accade; inoltre, ogni `P_ria`, la schermata riceve una fotografia completa dello stato corrente.
- **distinzione che conta**: l'aggiornamento e' **spinto dal sistema quando l'evento accade**, non ottenuto da interrogazioni a orologio da parte della schermata. La fotografia periodica non e' il meccanismo di aggiornamento: serve a riallineare chi si e' collegato dopo o ha perso qualcosa.
- stato di implementazione: completo, distinzione compresa: l'evento viene spinto sulla websocket quando accade, e la fotografia periodica viaggia sullo stesso canale.

1.4.1.1.5 RF01.1.5 - Indicatori di sintesi

- enunciato: una schermata di riepilogo riassume in pochi numeri i convogli in circolazione, le stazioni operative, i guasti aperti e il ritardo medio.
- rispetto alla consegna: e' un'aggiunta, la consegna non la chiede.
- stato di implementazione: completo. I numeri li calcola la centrale e viaggiano dentro la stessa fotografia periodica di RF01.1.4, quindi si aggiornano da soli come il resto.

1.4.1.2 RF01.2 - Visualizzazione dei guasti

Stato di implementazione: completo, entrambe le voci.

1.4.1.2.1 RF01.2.1 - Elenco degli allarmi aperti

- enunciato: sono visibili i guasti attivi con la sorgente (quale stazione, quale convoglio, quale tratta), il tipo, la severita' e l'istante di apertura.
- dipende da: RF02.1.
- stato di implementazione: completo.

1.4.1.2.2 RF01.2.2 - Distinzione fra guasto segnalato e guasto dedotto

- enunciato: l'elenco distingue il guasto **segnalato dal campo** (qualcuno lo ha dichiarato) da quello **dedotto dalla centrale** (un nodo ha smesso di dare notizie).
- **perche' e' un requisito e non un dettaglio**: sono due condizioni con cause diverse e con reazioni diverse. Un guasto segnalato indica un problema del nodo; un guasto dedotto puo' indicare anche soltanto un problema di collegamento, e chi decide se mandare la squadra deve poterli distinguere senza indovinare.
- dipende da: RF02.6.
- stato di implementazione: completo. L'API e gli eventi in tempo reale portano l'origine dell'allarme, e la schermata la mostra esplicitamente come "segnalato dal campo" oppure "dedotto dalla centrale".

1.4.1.3 RF01.3 - Amministrazione dei dati della rete

Enunciato del ramo: la creazione, la modifica e l'eliminazione dell'anagrafica della rete sono riservate all'amministratore; il tecnico vede tutto ma non modifica nulla.

Rispetto alla consegna: la consegna chiede le tratte (RF01.3.3) e la modifica delle stazioni attraversate (RF01.3.4). Stazioni (RF01.3.1) e convogli (RF01.3.2) sono un'aggiunta: la consegna da' per scontata una rete gia' popolata, e serviva un posto in cui crearla.

Stato di implementazione: completo, tutte e cinque le voci.

1.4.1.3.1 RF01.3.1 - Stazioni

- enunciato: l'amministratore crea, modifica ed elimina le stazioni della rete.
- stato di implementazione: completo. Dal form si scelgono il codice (l'id della stazione, fisso in modifica), il nome e le coordinate; le coordinate lasciate vuote valgono 0, e la mappa piazza la stazione vicino a quelle collegate. Stato e binari non si scrivono: una stazione nuova nasce OFFLINE con un binario, poi lo stato lo decidono il battito del nodo e i comandi del tecnico. La PUT della Centrale accetterebbe ancora tutti e due, ma la web app non li manda.

1.4.1.3.2 RF01.3.2 - Convogli

- enunciato: l'amministratore crea, modifica ed elimina i convogli.
- stato di implementazione: completo. Dal form si scelgono il nome del convoglio, solo in creazione perche' e' la chiave primaria, e l'itinerario. Stato, ritardo e passeggeri no: lo stato lo tengono gli automi della Centrale con la telemetria, il ritardo lo misura la Centrale ai passaggi e i passeggeri li dichiara il convoglio. Un convoglio nuovo nasce "fermo"; la PUT della Centrale accetterebbe ancora lo stato, ma la web app non lo manda.

1.4.1.3.3 RF01.3.3 - Tratte elementari

- enunciato: l'amministratore crea, modifica ed elimina i collegamenti diretti fra due stazioni adiacenti.
- stato di implementazione: completo. Il form rifiuta un tempo di percorrenza sotto il minuto.

1.4.1.3.4 RF01.3.4 - Itinerari e assegnazione dei convogli

- enunciato: l'amministratore compone un itinerario come sequenza ordinata di stazioni con i tempi di percorrenza previsti, e vi assegna sopra i convogli che devono percorrerlo.
- **scelta di quanto dura il servizio**: sull'itinerario si puo' marcare un estremo come capolinea, e allora la corsa si chiude li' invece di ripartire all'incontrario. Quale dei due estremi non e' indifferente: l'ultima stazione chiude dopo la sola andata, la prima dopo andata e ritorno. Una fermata intermedia non e' ammessa, perche' spezzerebbe l'itinerario in due senza dire che ne e' del pezzo restante.
- dipende da: RF02.5, perche' la modifica deve restare corretta anche con convogli gia' in marcia.
- stato di implementazione: completo; quello che succede ai convogli gia' in marcia e' il ramo RF02.5. Il capolinea si marca e si toglie con la bandierina accanto alle due stazioni estreme, e toglierlo e' l'unico modo per rimettere in servizio un convoglio che ha gia' chiuso la corsa.

1.4.1.3.5 RF01.3.5 - Integrita' dei dati alle eliminazioni

- enunciato: l'eliminazione di un'entita' non lascia riferimenti pendenti, e non e' permessa finche' qualcosa la sta ancora usando.
- esito atteso: o l'eliminazione viene rifiutata con una spiegazione comprensibile, oppure viene eseguita insieme a tutto cio' che senza quell'entita' non avrebbe piu' significato.
- stato di implementazione: completo. Ogni eliminazione o viene rifiutata con il motivo (la stazione usata da una tratta, la tratta usata da un itinerario o gia' nei transiti storici), oppure porta via i record che senza quell'entita' non significano piu' niente. Una parte della garanzia la tengono i vincoli del database.

1.4.1.4 RF01.4 - Comandi operativi

Chi puo' dare quale comando: la squadra la manda il **tecnico**, la corsa la sopprime l'**amministratore** (comando facoltativo nella consegna). La chiusura dell'allarme la consegna non la assegna: sta sul tecnico perche' e' il seguito dell'invio degli operatori.

Stato di implementazione: parziale, solo per i permessi. I tre comandi fanno per intero quello che devono, ma il filtro dei permessi li concede a entrambi i ruoli: e' l'unico punto in cui il sistema e' piu' permissivo della consegna (RF04.2).

1.4.1.4.1 RF01.4.1 - Invio della squadra di manutenzione

- enunciato: il tecnico manda la squadra su una stazione guasta; la stazione resta inutilizzabile per tutta la durata dell'intervento.
- soggetto: il tecnico, che nella consegna e' il ruolo a cui questo compito e' assegnato.
- precondizione: sulla stazione esiste almeno un guasto aperto. Non si mandano operatori su una stazione sana: il comando viene rifiutato con 409.
- esito atteso: la stazione risulta in MANUTENZIONE a chi la guarda, e ci resta finche' il tecnico non chiude il guasto; il campo viene informato e i guasti aperti *restano aperti*, perche' l'invio degli operatori apre l'intervento e non lo conclude.

- nota di progetto: MANUTENZIONE e' uno stato della Centrale e vale "guasta piu' squadra inviata": e' il "preso in carico" del guasto visto dal lato della stazione, quindi il guasto non ha bisogno di un terzo stato. Sul nodo lo stato resta GUASTA, ed e' quello che continua a trattenere i treni in ingresso. Ai convogli lo dice la Centrale con l'alert `STATO_STAZIONE`: per il twin di bordo GUASTA e MANUTENZIONE sono la stessa cosa, quei binari non si attraversano. Durante l'intervento nessun evento del campo riscrive lo stato: il battito (che arriva ancora come GUASTA) aggiorna solo l'ultimo contatto, e un alert di guasto nuovo non riporta la stazione a GUASTA. La regola sta in un punto solo, la scrittura dello stato in cache (M12): gli eventi del campo non escono dalla manutenzione, i comandi dell'operatore si'.

- stato di implementazione: completo. Il comando porta la stazione in MANUTENZIONE, scrive la riga di storico e avvisa il campo; il ritorno in servizio e' legato alla fine dell'intervento (RF01.4.2). L'intervento sopravvive a un riavvio della centrale: all'avvio la riga piu' recente di `Storico_Stato_Stazioni`, letta insieme ai guasti ancora aperti, dice quali stazioni erano in intervento.

1.4.1.4.2 RF01.4.2 - Chiusura dell'allarme e ripresa della circolazione

- enunciato: il tecnico dichiara concluso l'intervento; il guasto passa a chiuso e cio' che era trattenuto per causa sua riprende.

- soggetto: il tecnico, per la ragione scritta sopra al ramo: chiudere l'allarme e' il seguito dell'invio degli operatori.

- esito atteso: il guasto risulta risolto, la stazione torna ONLINE se non le restano altri guasti aperti, la transizione viene storicizzata e i convogli fermi in stazione ripartono.

- dipende da: RF01.4.1, di cui chiude l'intervallo.

- stato di implementazione: completo. La chiusura manda al campo il RESOLVED, che riporta il nodo a ONLINE, e lo `STATO_STAZIONE` che dichiara la stazione agibile: e' quest'ultimo a liberare i convogli. La riga di storico chiude il passaggio per MANUTENZIONE.

- limite dichiarato: non viene registrato *chi* ha chiuso il guasto (`Guasto.operatore` resta vuoto); la consegna non lo chiede (RF02.1.3).

1.4.1.4.3 RF01.4.3 - Soppressione di un convoglio fermo in stazione

- enunciato: l'amministratore annulla una corsa; il convoglio passa in SOPPRESSO e non viaggia piu'.
- soggetto: l'amministratore, che nella consegna e' il ruolo a cui questo comando e' assegnato.
- precondizione: il convoglio e' fermo in una stazione.
- rispetto alla consegna: la consegna lo scrive fra parentesi, come comando facoltativo; e' stato realizzato lo stesso.
- stato di implementazione: completo. La precondizione e' una guardia del comando: un convoglio che non risulta fermo in una stazione non si sopprime, e l'operatore riceve un 400 con il motivo.

1.4.1.5 RF01.5 - Consultazione dello storico

- enunciato: sono consultabili i transiti passati, ciascuno con il convoglio, la stazione, la tratta, il verso (entrata o uscita) e l'istante.
- rispetto alla consegna: e' un'aggiunta. La consegna chiede alla centrale di **memorizzare** lo storico dei transiti, e quello e' RF02.7; mostrarlo in una schermata non lo chiede nessuno.
- dipende da: RF02.7.
- stato di implementazione: parziale. La Centrale serve lo storico (`GET /api/transiti`) e la web app lo carica nello store, ma nell'interfaccia non c'e' nessuna schermata che lo mostri.

1.4.1.6 RF01.6 - Accesso al sistema

- enunciato: l'uso dell'applicazione richiede di identificarsi; l'identita' accompagna tutte le operazioni successive e termina con l'uscita esplicita.
- l'identificazione avviene **fuori dall'applicazione**, sulla schermata del servizio di identita': l'applicazione non vede mai le credenziali, riceve solo l'attestato di accesso.
- rimando: il dettaglio dei permessi e' in RF04.
- stato di implementazione: completo.

1.4.2 RF02 - Correttezza dello stato rappresentato

Enunciato: lo stato che il sistema mostra e conserva corrisponde a quello che sta realmente accadendo sulla rete, anche in presenza di guasti, di interruzioni del collegamento e di modifiche fatte mentre il sistema e' in funzione.

Stato di implementazione: parziale. Sono completi il ramo dei convogli (RF02.2), quello delle stazioni (RF02.3), l'identita' dei nodi di campo (RF02.4), la modifica dell'itinerario a sistema acceso (RF02.5) e le anomalie che la centrale deduce da sola (RF02.6). Delle conseguenze del guasto del convoglio sull'infrastruttura (RF02.1.2.2) e' realizzata quella sulla stazione e non quella sulla tratta; sono parziali il ciclo di vita del guasto, che non registra la presa in carico (RF02.1.3), e la memoria storica, che di cinque cose ne scrive tre (RF02.7).

1.4.2.1 RF02.1 - Gestione dei guasti

Stato di implementazione: parziale — il guasto della stazione e la chiusura automatica ci sono; del guasto del convoglio manca la conseguenza sulla tratta (RF02.1.2.2.2), e al ciclo di vita del guasto manca la presa in carico (RF02.1.3).

1.4.2.1.1 RF02.1.1 - Guasto della stazione

Stato di implementazione: parziale, per il solo RF02.1.1.2.3.

1. RF02.1.1.1 - Interruzione del collegamento con la centrale

   Stato di implementazione: completo.

   (a) RF02.1.1.1.1 - Nessun dato perduto durante l'interruzione

   - enunciato: mentre il collegamento con la centrale non e' disponibile, la stazione conserva localmente tutto cio' che produce e lo consegna quando il collegamento torna.
   - precondizione: la stazione continua a funzionare e i suoi sensori continuano a rilevare.
   - esito atteso: al ritorno del collegamento la centrale riceve tutti gli eventi prodotti durante l'interruzione, nessuno escluso.
   - stato di implementazione: completo. Il nodo accoda cio' che non riesce a consegnare e svuota la coda quando il collegamento torna. La coda ha una copia sul SQLite locale (tabella `bufferChache`), quindi regge anche a un riavvio del processo.

(b) RF02.1.1.1.2 - Ordine di consegna preservato

- enunciato: la consegna differita rispetta l'ordine in cui gli eventi sono stati generati; se una singola consegna fallisce, l'evento non consegnato precede comunque quelli successivi.
- **perche' e' un requisito a se'**: senza questo vincolo l'entrata e l'uscita di uno stesso convoglio possono arrivare invertite, e lo storico racconterebbe un convoglio che esce da una stazione prima di entrarci.
- dipende da: RF02.7.
- stato di implementazione: completo. L'evento che non riesce a partire rientra in **testa** alla coda, ed e' esattamente questo che impedisce a un'uscita di precedere la propria entrata.

2. RF02.1.1.2 - Stazione non percorribile

   Stato di implementazione: completo.

   Che cosa vuol dire "non percorribile": la stazione e' **guasta** (lo dice il campo) oppure in **manutenzione** (la squadra mandata dalla centrale, RF01.4.1). Per il convoglio sono la stessa cosa; a distinguerle serve solo all'operatore.

   (a) RF02.1.1.2.1 - Il convoglio diretto alla stazione non percorribile si ferma

   - enunciato: se la prossima stazione dell'itinerario non e' percorribile, il convoglio non vi entra e resta fermo dove si trova.
   - esito atteso: il convoglio risulta FERMO, la sua posizione non avanza e il ritardo cresce.
   - dipende da: RF02.2.2.1.
   - stato di implementazione: completo, per tutte e due le condizioni: il guasto dichiarato dal campo e la manutenzione dichiarata dalla centrale con `STATO_STAZIONE`.

   (b) RF02.1.1.2.2 - Riparte solo al ripristino

   - enunciato: il convoglio trattenuto riprende la corsa quando arriva il ripristino di quella stazione, non prima e non per scadenza di un tempo.
   - stato di implementazione: completo, senza nessun tempo massimo. Il ripristino che conta e' quello dello **stato della stazione**, non la chiusura del singolo allarme: un `RESOLVED` che arriva mentre la squadra e' ancora sul posto lascia il convoglio dov'e'.

(c) RF02.1.1.2.3 - Il blocco vale anche per chi e' gia' fermo li'

- enunciato: un convoglio gia' in sosta nella stazione che si guasta non riparte finche' il guasto e' aperto.
- **perche' e' un requisito a se'**: senza di esso il guasto bloccherebbe solo chi arriva dopo, e la stazione non risulterebbe davvero non percorribile.
- stato di implementazione: completo. Il convoglio tiene in memoria lo stato di ogni stazione che lo riguarda e lo ricontrolla prima di ogni partenza. Un convoglio acceso *dopo* l'annuncio lo trova nell'itinerario scaricato all'avvio (campo `stato` di ogni tappa), perche' gli alert su un topic li sente solo chi era in ascolto.

1.4.2.1.2 RF02.1.2 - Guasto del convoglio

Stato di implementazione: parziale — il guasto viene registrato e chiude la stazione in cui il convoglio e' fermo; la tratta invece resta percorribile.

1. RF02.1.2.1 - Il guasto viene registrato in centrale

   - enunciato: quando un convoglio si guasta, la centrale apre e conserva il guasto corrispondente.
   - stato di implementazione: completo.

2. RF02.1.2.2 - Conseguenze del guasto

   Stato di implementazione: parziale — realizzato per il convoglio guasto in stazione, non per quello guasto in tratta.

   (a) RF02.1.2.2.1 - Convoglio guasto **in stazione**: la stazione diventa non percorribile

   - enunciato: se il convoglio si guasta mentre e' fermo in una stazione, quella stazione risulta guasta.
   - dipende da: RF02.2.2.1, perche' la conseguenza si sceglie in base a dove si trova il convoglio.
   - stato di implementazione: completo. Quando arriva il guasto critico di un convoglio che risulta in una stazione, la centrale porta anche quella stazione a GUASTA, apre su di lei un guasto dedotto (tipo `convoglio_guasto_in_stazione`) e lo pubblica sul campo: i convogli diretti li' si fermano e il nodo stazione si allinea. Il guasto dedotto si chiude insieme a quello

del convoglio, quando l'operatore lo risolve, e la stazione torna ONLINE se non le resta nessun altro guasto aperto.

(b) RF02.1.2.2.2 - Convoglio guasto **in tratta**: la tratta diventa non percorribile

- enunciato: se il convoglio si guasta fra due stazioni, e' la tratta elementare che sta percorrendo a risultare non percorribile; nessuna stazione cambia stato.
- dipende da: RF02.2.2.1.
- stato di implementazione: non realizzato. L'allarme si apre sul convoglio, ma la tratta non ha uno stato di percorribilita': resta libera e nessun altro convoglio viene fermato. Vale anche per l'allarme dedotto (RF02.6.2).

1.4.2.1.3 RF02.1.3 - Ciclo di vita del guasto

- enunciato: un guasto ha uno stato di lavorazione (aperto, preso in carico, chiuso), un assegnatario e una traccia di come e' evoluto; sulla stessa sorgente e per lo stesso motivo non esiste piu' di un guasto aperto per volta.
- **perche' l'unicita' e' un requisito**: le anomalie dedotte vengono ricontrollate a ritmo periodico, e senza unicita' la stessa condizione genererebbe un allarme nuovo a ogni controllo, rendendo l'elenco inutilizzabile proprio quando serve.
- stato di implementazione: parziale. L'unicita' per sorgente c'e', e apertura e chiusura lasciano traccia nello storico. Gli stati pero' sono due (il guasto porta solo "risolto" vero o falso) e l'assegnatario non viene mai scritto: colonna e tabella delle assegnazioni restano vuote.

1.4.2.1.4 RF02.1.4 - Chiusura automatica quando la causa rientra

- enunciato: un guasto aperto perche' un nodo taceva si chiude da solo quando il nodo torna a dare notizie, senza bisogno dell'intervento di un operatore.
- **perche'**: un guasto dedotto deve essere disfatto dalla stessa deduzione che lo ha creato, altrimenti l'elenco degli allarmi si riempie di condizioni gia' rientrate che nessuno chiude.
- precondizione: il guasto e' stato aperto per deduzione e non per segnalazione.

- dipende da: RF02.3.2.
- stato di implementazione: completo. Il battito che torna richiude il guasto dedotto sulla stazione, il convoglio che riprende a muoversi richiude il proprio.

1.4.2.2 RF02.2 - Gestione dei convogli

Stato di implementazione: completo.

1.4.2.2.1 RF02.2.1 - Ritardi

Stato di implementazione: completo.

1. RF02.2.1.1 - Il ritardo si accumula quando il convoglio e' fermo e dovrebbe circolare
   - enunciato: finche' il convoglio e' trattenuto pur avendo una tappa da percorrere, il suo ritardo cresce; quando circola regolarmente non cresce.
   - dipende da: RF02.1, perche' il fermo nasce dai guasti.
   - stato di implementazione: completo.

1.4.2.2.2 RF02.2.2 - Informazioni di stato del convoglio

Stato di implementazione: completo, tutte e quattro le informazioni.

1. RF02.2.2.1 - Posizione
   - enunciato: il convoglio dichiara in modo non ambiguo dove si trova: in quale stazione e' fermo, oppure fra quali due stazioni sta viaggiando e in quale direzione.
   - esito atteso: la risposta alla domanda "il convoglio e' in stazione o in tratta?" e' sempre definita, in ogni istante della corsa.
   - **nota**: e' il **predicato centrale della specifica**: ne dipendono la conseguenza di un guasto (stazione o tratta), la modifica dell'itinerario a sistema acceso e l'anomalia del convoglio fermo.
   - stato di implementazione: completo. Il convoglio dichiara la stazione in cui e' fermo o la tratta che percorre con la direzione, e la centrale incrocia la dichiarazione con i passaggi rilevati a terra. Le due fonti possono non concordare: la telemetria dice dove il

convoglio **crede** di essere, il passaggio dei sensori conferma da dove e' uscito davvero, ma se la stazione e' spenta nessuno lo registra. In quel caso la centrale non le compone: dichiara la posizione **incerta** con il motivo (non trasmette, nessun sensore conferma la partenza, fermata davanti non dichiarata, verso ignoto), e il verso non garantito resta nullo. Il predicato ha quindi tre esiti: in stazione, in tratta fra due estremi noti, oppure *non lo so*.

2. RF02.2.2.2 - Ritardo accumulato

   - enunciato: il convoglio dichiara il ritardo accumulato dall'inizio della corsa.
   - dipende da: RF02.2.1.
   - stato di implementazione: completo.

3. RF02.2.2.3 - Stato del convoglio

   - enunciato: il convoglio dichiara come sta, con uno dei valori del vocabolario degli stati, e questa informazione e' distinta dalla posizione.
   - stato di implementazione: completo; a database la colonna dello stato accetta solo i cinque valori del vocabolario. Quei valori sono un riassunto: la telemetria porta anche condizione e fase del viaggio, ed e' quello che l'interfaccia mostra.

4. RF02.2.2.4 - Velocita' e passeggeri a bordo

   - enunciato: il convoglio dichiara la velocita' corrente e il numero di passeggeri a bordo.
   - **nota di ambito**: sono valori simulati che rendono leggibile la corsa sulla mappa; non provengono da nessuna rilevazione reale e non alimentano nessuna decisione del sistema.
   - stato di implementazione: completo, nell'ambito dichiarato.

1.4.2.2.3 RF02.2.3 - Andata e ritorno sullo stesso itinerario

- enunciato: raggiunto un estremo dell'itinerario, il convoglio inverte la marcia e lo ripercorre in senso contrario; il tempo di percorrenza di ogni tappa e' quello previsto per il verso che sta percorrendo.

- esito atteso: la corsa e' continua, senza che il convoglio scompaia e riappaia all'altro estremo.
- eccezione: l'estremo marcato come capolinea (RF01.3.4) non e' un punto in cui girarsi ma quello in cui il servizio finisce, e li' l'inversione non avviene.
- stato di implementazione: completo.

1.4.2.2.4 RF02.2.4 - Passaggi asimmetrici in stazione

- enunciato: un convoglio puo' uscire da una stazione senza esservi entrato, e puo' entrare in una stazione senza uscirne.
- precondizione: il primo caso si presenta alla stazione da cui la corsa comincia, il secondo al capolinea in cui la corsa si conclude.
- **perche' va scritto**: e' il caso che rompe l'abbinamento ingenuo fra entrate e uscite nello storico. Chi conta le coppie deve sapere che la prima e l'ultima sono spaiate per costruzione, e che questo non e' un errore di rilevazione.
- stato di implementazione: completo. L'uscita senza entrata diventa un transito puntuale (entrata e uscita coincidono), l'entrata senza uscita resta un transito aperto: al capolinea di RF01.3.4 resta aperto finche' al convoglio non viene assegnato un altro itinerario. Nello storico i due casi non sono marcati a parte.

1.4.2.3 RF02.3 - Gestione delle stazioni

Stato di implementazione: completo.

1.4.2.3.1 RF02.3.1 - Informazioni rese dalla stazione

- enunciato: la stazione sa dire quali convogli ha sui binari in questo momento, chi e' entrato e quando, chi e' uscito e quando.
- dipende da: RF02.1.1.1.1 — l'informazione locale che serve alla visualizzazione **e' la stessa** che permette alla stazione di sopravvivere a un'interruzione del collegamento. Non sono due memorie distinte, ed e' un vincolo di specifica: se fossero due potrebbero divergere.
- stato di implementazione: completo, vincolo compreso.

1.4.2.3.2 RF02.3.2 - Segnale di vita verso la centrale

- enunciato: la stazione emette un battito periodico verso la centrale, con periodo `P_bat`, anche quando non ha nulla da dichiarare.
- **perche' anche quando non c'e' niente da dire**: senza un segnale emesso a prescindere dagli eventi, il silenzio di una stazione non e' distinguibile dall'assenza di traffico su quella stazione.
- stato di implementazione: completo.

1.4.2.3.3 RF02.3.3 - Segnale di vita dei sensori verso la stazione

- enunciato: i sensori dichiarano periodicamente alla stazione di essere vivi, con periodo `P_kal`; se uno smette, la stazione lo considera guasto e lo segnala alla centrale.
- esito atteso: la stazione con un sensore muto dichiara il proprio stato come GUASTA.
- stato di implementazione: completo. La stazione mette sotto osservazione all'avvio i sensori che si aspetta (uno per verso), quindi risulta guasto anche un sensore che non ha mai battuto.

1.4.2.3.4 RF02.3.4 - Stato che la stazione dichiara di se'

- enunciato: la stazione dichiara di se' soltanto ONLINE o GUASTA.
- rimando: la motivazione e' nella sezione sul vocabolario degli stati osservabili.
- stato di implementazione: completo: il nodo dichiara due stati, la centrale ne tiene quattro.

1.4.2.4 RF02.4 - Identita' del nodo di campo

Stato di implementazione: completo.

1.4.2.4.1 RF02.4.1 - L'identita' dichiarata viene verificata

- enunciato: un nodo che entra in servizio dichiara la propria identita' alla centrale, che la verifica sull'anagrafica della rete prima di accettarne qualsiasi dato.

- **perche'**: senza verifica un nodo con un'identita' inventata inquina lo stato della centrale con la telemetria di un convoglio che non esiste.
- stato di implementazione: completo. Il nodo non manda niente finche' la centrale non gli ha risposto.

1.4.2.4.2 RF02.4.2 - Esito negativo: il nodo non entra in servizio

- enunciato: se l'identita' non e' riconosciuta, il nodo non entra in servizio e termina, invece di restare attivo senza poter comunicare.
- **perche' terminare**: a chi guarda i processi un nodo acceso ma non riconosciuto sembra sano.
- stato di implementazione: completo: il nodo termina con codice di uscita 1.

1.4.2.4.3 RF02.4.3 - Portata della dipendenza

Non e' un requisito a se': senza RF02.4 nessun nodo entra in servizio, quindi da RF02.4 dipendono per intero RF02.1, RF02.2 e RF02.3.

1.4.2.5 RF02.5 - Modifica dell'itinerario a sistema acceso

Enunciato del ramo: l'amministratore puo' cambiare le stazioni attraversate da un itinerario e i convogli assegnati mentre i convogli sono gia' in marcia, e il sistema resta coerente.

Stato di implementazione: completo — sono gestiti sia il convoglio escluso, che si dichiara fermo in attesa, sia quello che resta assegnato, che prosegue dal punto in cui la modifica lo ha colto.

1.4.2.5.1 RF02.5.1 - Convoglio escluso dall'itinerario

- enunciato: se dopo la modifica un convoglio non e' piu' assegnato ad alcun itinerario, smette di viaggiare e resta fermo in attesa di essere soppresso.
- esito atteso: il convoglio non produce piu' transiti e non avanza; resta visibile nelle schermate, perche' un convoglio fermo su un binario e' un fatto che l'operatore deve vedere.

- **punto delicato della specifica**: la notifica deve raggiungere **anche** i convogli esclusi, se no continuerebbero a percorrere l'itinerario vecchio mostrando uno stato plausibile ma falso.
- stato di implementazione: completo, punto delicato compreso.

1.4.2.5.2 RF02.5.2 - Convoglio ancora assegnato

- enunciato: se dopo la modifica il convoglio e' ancora assegnato all'itinerario, prosegue sul nuovo itinerario a partire dal punto in cui la modifica lo ha colto, conservando direzione e ritardo accumulato.
- precondizione: la stazione da cui il convoglio e' appena passato appartiene ancora al nuovo itinerario.
- esito atteso: la posizione mostrata non compie salti; il convoglio non ricomincia la corsa da capo.
- caso limite: se la stazione da cui e' appena passato non fa piu' parte dell'itinerario, il convoglio riprende dal capolinea del nuovo itinerario, perche' non esiste un punto di ripresa sensato.
- dipende da: RF02.2.2.1.
- stato di implementazione: completo, caso limite compreso (con una riga nel log del convoglio). Ricevuto l'avviso il convoglio riscarica l'itinerario senza buttare via quello che ha, cerca nella lista nuova la stazione in cui si trova e ci si riaggancia: ricalcola indice, direzione e prossima fermata, mentre ritardo, passeggeri e stato restano quelli di prima. Non pubblica nessun passaggio, perche' non si e' mosso.
- **scelta di progetto sul convoglio in mezzo a una tratta**: il percorso nuovo resta in attesa e sostituisce il vecchio quando il convoglio entra nella stazione di arrivo; riagganciarlo subito lo farebbe saltare indietro alla stazione di partenza, il salto che il requisito vieta.

1.4.2.6 RF02.6 - Anomalie dedotte dalla centrale

Enunciato del ramo: la centrale riconosce da sola le anomalie che i nodi di campo non sono in grado di segnalare, a partire dalla propria caduta.

Perche' esiste questo ramo: un nodo guasto puo' non essere in condizione di dire di esserlo. Se il sistema si fidasse solo delle segnalazioni, il caso peggiore (il nodo che smette del tutto) sarebbe anche l'unico invisibile.

Stato di implementazione: completo, tutti e tre.

1.4.2.6.1 RF02.6.1 - Stazione silenziosa oltre la soglia

- enunciato: se una stazione non da' notizie di se' per piu' di `T_sil`, la centrale la considera caduta, la mostra come OFFLINE e apre un guasto su di essa.
- dipende da: RF02.3.2, perche' senza un segnale regolare non c'e' un silenzio da misurare.
- si chiude da solo per RF02.1.4.
- stato di implementazione: completo; il guasto viene pubblicato verso il campo, quindi i convogli diretti alla stazione caduta si fermano.

1.4.2.6.2 RF02.6.2 - Convoglio fermo fuori stazione oltre la soglia

- enunciato: se un convoglio resta fermo fra due stazioni per piu' di `T_fer`, la centrale apre un allarme su di esso.
- precondizione: il convoglio non e' in una stazione e non e' soppresso.
- dipende da: RF02.2.2.1, perche' "fuori stazione" e' una domanda sulla posizione.
- stato di implementazione: completo; la tratta pero' resta percorribile (limite di RF02.1.2.2.2).

1.4.2.6.3 RF02.6.3 - Fotografia periodica dello stato

- enunciato: ogni `P_ria` la centrale rende disponibile alle schermate una fotografia completa dello stato corrente della rete.
- **a cosa serve**: a riallineare chi si e' collegato dopo che gli eventi erano gia' avvenuti, o chi ne ha persi. Non e' il meccanismo con cui le schermate si aggiornano (RF01.1.4), e' la rete di sicurezza sotto di esso.
- stato di implementazione: completo: la fotografia porta i numeri di sintesi e l'elenco completo di convogli e stazioni.

1.4.2.7 RF02.7 - Memoria storica

- enunciato: oltre allo stato corrente, il sistema conserva la traccia di cio' che e' accaduto: transiti, cambi di stato dei convogli e delle stazioni, guasti, assegnazioni degli operatori e itinerari percorsi.
- esito atteso: la storia e' consultabile anche dopo che i nodi che l'hanno prodotta sono stati spenti e riavviati.
- **regola di registrazione**: la storia registra i **cambiamenti**, non i campionamenti: un'ora di telemetria senza cambi di stato lascia una riga sola.
- stato di implementazione: parziale. Delle cinque cose ne vengono scritte tre (transiti, cambi di stato di convogli e stazioni, guasti), ai cambiamenti. Itinerari percorsi e assegnazioni degli operatori no: le quattro tabelle relative esistono nello schema ma nessuno le scrive (elenco nel capitolo *Modello*).

1.4.3 RF03 - Contratti di interazione fra le parti

Enunciato: le parti del sistema interagiscono secondo obblighi dichiarati, in modo che ciascuna possa funzionare, fallire e ripartire senza trascinare con se' le altre.

Nota di lettura: questo ramo non prescrive tecnologie. Prescrive proprieta' dell'interazione che si osservano dall'esterno: chi deve poter sopravvivere a chi, quale informazione e' fonte di verita' per cosa, che cosa una parte puo' e non puo' toccare dell'altra.

Stato di implementazione: completo.

1.4.3.1 RF03.1 - Autonomia dei nodi

- enunciato: ogni convoglio e ogni stazione sono parti autonome, avviabili, arrestabili e riavviabili una indipendentemente dall'altra.
- esito atteso: l'arresto di un nodo non ferma gli altri; la rete continua a funzionare in modo ridotto, con l'assenza segnalata come anomalia (RF02.6.1) e non come blocco generale.
- stato di implementazione: completo: ogni nodo e' un processo a se', avviato con il proprio identificativo.

1.4.3.2 RF03.2 - Disaccoppiamento fra campo e centrale

- enunciato: quando un nodo di campo comunica con la centrale non resta in attesa che la centrale sia presente per poter continuare a lavorare.
- esito atteso: con la centrale non raggiungibile, i nodi di campo continuano a rilevare, a registrare e ad accumulare cio' che dovranno consegnare; nessuna funzione locale si arresta.
- dipende da: RF02.1.1.1.1.
- stato di implementazione: completo. Fra campo e centrale c'e' sempre il broker, quindi nessuno dei due aspetta l'altro: il nodo pubblica e va avanti.

1.4.3.3 RF03.3 - Due fonti di verita' distinte per lo stesso passaggio

- enunciato: lo stesso passaggio fisico e' conosciuto per due vie indipendenti: la rilevazione a terra fatta dai sensori di stazione e la dichiarazione fatta dal convoglio. Le due vie non si sostituiscono a vicenda.
- ripartizione degli obblighi:
  - la rilevazione a terra e' la **fonte di verita' per la storia dei transiti**: esiste anche se il convoglio tace, ed e' quella che deve finire nello storico;
  - la dichiarazione del convoglio e' la **fonte di verita' per la posizione corrente**: e' disponibile prima e con piu' dettaglio, ed e' quella che alimenta il tempo reale.
- **perche' due e non una**: senza la prima si perde la storia quando il convoglio tace, senza la seconda il tempo reale fra due stazioni, dove nessun sensore vede niente.
- stato di implementazione: completo. La dichiarazione del convoglio arriva in centrale **soltanto** con la telemetria: l'annuncio del passaggio e' rivolto ai sensori di binario della stazione, non alla centrale. Cosi' il fail-stop di un sensore ha la conseguenza giusta: il transito non lo registra nessuno.

- **conseguenza**: il verso di marcia la centrale lo conosce solo dalla telemetria (ogni `P_tel`); fra un'inversione all'estremo e il frame successivo la fermata successiva viene cercata dalla parte sbagliata dell'itinerario.

1.4.3.4 RF03.4 - I sensori usano solo l'interfaccia pubblica del nodo

- enunciato: i sensori dichiarano cio' che rilevano attraverso l'interfaccia che il nodo espone verso l'esterno, senza nessun accesso privilegiato al suo stato interno.
- **perche' e' un requisito**: rende il nodo verificabile dall'esterno, con lo stesso comportamento in prova e in esercizio.
- esito atteso: una dichiarazione fatta a mano da uno strumento di prova e una fatta dal sensore reale producono nel nodo esattamente lo stesso effetto.
- stato di implementazione: completo, per i sensori di binario e di bordo; tutte le prove dei guasti si fanno da li'.

1.4.3.5 RF03.5 - L'itinerario e' fornito dalla centrale

- enunciato: il convoglio non conosce l'itinerario per conto proprio: lo riceve dalla centrale, che ne e' l'unica depositaria.
- due modalita' ammesse, entrambe valide:
  - il convoglio riceve l'itinerario completo quando entra in servizio e lo tiene per la corsa;
  - il convoglio chiede di volta in volta quale sia la prossima stazione, data la stazione in cui si trova e la direzione.
- **perche' l'unicita' della fonte conta**: se il convoglio conservasse un itinerario proprio, dopo una modifica fatta a sistema acceso esisterebbero due versioni dell'itinerario e RF02.5 non sarebbe garantibile.
- stato di implementazione: completo. Il convoglio usa la prima modalita' (itinerario intero all'avvio e di nuovo a ogni modifica); l'operazione della seconda esiste ma nessuno la usa.

1.4.4 RF04 - Accesso e protezione

Stato di implementazione: parziale — l'accesso e la protezione dei canali sono completi; l'autorizzazione per ruolo vale per le letture e per l'anagrafica ma non distingue i ruoli sui tre comandi operativi (RF04.2).

1.4.4.1 RF04.1 - Autenticazione

- enunciato: gli utenti sono registrati presso il servizio di identita' e accedono dichiarando li' le proprie credenziali; l'accesso produce un attestato che accompagna le operazioni successive.
- **requisito fondamentale**: le credenziali sono conosciute da una parte sola. Chi custodisce i dati della rete non conserva password: controlla solo che l'attestato sia autentico e non scaduto.
- l'attestato e' verificabile da solo, senza interrogare il servizio di identita' a ogni richiesta.
- l'attestato ha una durata limitata (`T_att`) e viene rinnovato senza disturbare l'utente finche' la sessione presso il servizio di identita' resta aperta (`T_ina`, `T_ses`).
- stato di implementazione: completo: la centrale controlla la firma dell'attestato.

1.4.4.2 RF04.2 - Autorizzazione per ruolo

- enunciato: ogni operazione e' permessa o negata in base al ruolo di chi la chiede: le letture sono permesse a entrambi i ruoli, la modifica dell'anagrafica della rete al solo amministratore, e ciascun comando operativo al ruolo che la consegna gli assegna — l'invio della squadra al tecnico, la soppressione della corsa all'amministratore (RF01.4).
- **requisito fondamentale**: il controllo dei permessi lo fa la parte che possiede i dati, non l'interfaccia: nascondere un comando non impedisce di mandare la stessa richiesta a mano.
- il ruolo e' scritto **dentro l'attestato** dal servizio di identita', non in una tabella della centrale: si cambia in un posto solo e vale dal primo attestato successivo.
- eccezioni dichiarate:

  - le richieste preliminari dei browser (preflight), che non portano identita';
  - le due sole letture che servono ai nodi di campo (RF03.5): i nodi non sono utenti, la loro verifica e' quella di RF02.4.

- stato di implementazione: parziale. Controllo lato centrale e ruolo letto dall'attestato ci sono, e su letture e anagrafica la ripartizione e' rispettata con le sole eccezioni qui sopra. I tre comandi operativi pero' sono concessi a entrambi i ruoli. Resta fuori anche la websocket, porta di sola lettura (limite dichiarato in *Progettazione*).

1.4.4.3 RF04.3 - Protezione dei canali

- enunciato: il sistema puo' funzionare con tutti i canali di comunicazione protetti, in modo che il contenuto scambiato fra le parti non sia leggibile ne' alterabile da un osservatore della rete.

- esito atteso: la protezione e' una condizione dell'**intero** sistema e non della singola parte: o tutti i canali sono protetti, o la garanzia non vale.

- **conseguenza da tenere presente**: un solo canale in chiaro fra canali protetti non si vede come errore di sicurezza ma come un nodo che non riesce a entrare in servizio.

- **il canale delle credenziali e' protetto sempre**: li' passano le password degli operatori, che chi le intercetta puo' riusare. Il servizio di identita' accetta solo connessioni protette, in qualunque profilo. Di conseguenza anche la pagina su cui torna l'attestato va servita protetta, e una pagina protetta non puo' chiamare indirizzi in chiaro: usare la schermata richiede il profilo di protezione su tutti i canali.

- stato di implementazione: completo. Con il profilo apposito sono protetti MQTT (porta cifrata, compresa la validazione dei nodi), l'HTTPS verso la centrale e quello delle interfacce dei nodi di campo. Le parti si riconoscono tramite un'autorita' di certificazione privata del progetto.

1.5 Requisiti non funzionali osservabili

Sono proprieta' del sistema nel suo insieme. Le tengo separate dai requisiti funzionali perche' non descrivono una funzione, ma una qualita' che ogni funzione deve avere.

#	requisito	enunciato	come si verifica
RNF01	prontezza	un evento accaduto sul campo e' visibile all'operatore entro pochi secondi	si cronometra la distanza fra l'evento provocato e la sua comparsa sulla schermata
RNF02	nessuna perdita dei transiti	nessun transito rilevato va perduto, nemmeno in presenza di interruzioni del collegamento	si contano i transiti provocati e li si ritrova tutti nello storico
RNF03	degradazione per parti	il guasto di una parte riduce le funzioni disponibili ma non ferma il sistema	si arrestano nodi uno alla volta e si controlla cosa resta funzionante
RNF04	riservatezza	il contenuto scambiato fra le parti non e' leggibile da un osservatore della rete	si osserva il traffico con la protezione attiva
RNF05	numerosita' dei nodi	il sistema regge piu' stazioni e piu' convogli contemporaneamente, senza modifiche	si avviano piu' istanze e si controlla che ciascuna sia distinguibile e indipendente
RNF06	riavviabilita'	un nodo riavviato torna in servizio da solo e si riallinea, senza interventi manuali	si arresta e si riavvia un nodo mentre il sistema e' in funzione

1.6 Scenari di verifica

Gli scenari servono a controllare i requisiti **insieme**, come li vede un utente. Ogni scenario e' scritto con la situazione di partenza, l'evento che lo innesca e il comportamento atteso passo per passo.

1.6.1 SV01 - Stazione non percorribile e successivo ripristino

situazione un convoglio sta percorrendo la tratta che porta alla stazione B; in B c'e' gia' un altro convoglio in sosta.

evento B viene dichiarata guasta.

comportamento atteso 1. B risulta GUASTA nelle schermate e compare un allarme con B come sorgente;

2. il convoglio in arrivo si ferma prima di entrare in B e passa a FERMO;
3. il convoglio gia' in sosta in B non riparte a fine sosta;
4. il ritardo di entrambi cresce finche' restano fermi;
5. il tecnico manda la squadra: B passa a MANUTENZIONE e i due convogli restano fermi, perche' una stazione in manutenzione non e' percorribile piu' di una guasta;
6. il tecnico dichiara risolto il guasto: B torna ONLINE, l'allarme risulta chiuso e conservato nella storia;
7. entrambi i convogli riprendono la corsa dal punto in cui erano, senza altri comandi.

requisiti coperti RF01.2.1, RF01.4.1, RF01.4.2, RF02.1.1.2, RF02.1.3, RF02.2.1.1.

1.6.2 SV02 - Interruzione del collegamento fra stazione e centrale

situazione la stazione A e' operativa e i convogli transitano regolarmente.

evento il collegamento fra A e la centrale viene interrotto.

comportamento atteso 1. A continua a rilevare i transiti e a rispondere alle interrogazioni locali;

2. la centrale, passato `T_sil`, mostra A come OFFLINE e apre un allarme;
3. nel frattempo transitano dei convogli da A;
4. il collegamento viene ripristinato;
5. la centrale riceve tutti i transiti avvenuti durante l'interruzione, nell'ordine giusto;
6. l'allarme aperto per silenzio si chiude da solo e A torna ONLINE.

requisiti coperti RF02.1.1.1, RF02.1.4, RF02.6.1, RF03.2, RNF02.

1.6.3 SV03 - Guasto del convoglio, in stazione e in tratta

situazione due convogli, uno fermo nella stazione C e uno a meta' della tratta D-E.

evento entrambi dichiarano un guasto di bordo.

comportamento atteso
1. per il primo, la stazione C risulta non percorribile;
2. per il secondo, e' la tratta D-E a risultare non percorribile, mentre D ed E restano operative;
3. entrambi i convogli passano a EMERGENZA e restano dove sono;
4. i due allarmi risultano segnalati dal campo e non dedotti.

requisiti coperti RF01.2.2, RF02.1.2, RF02.2.2.1.

1.6.4 SV04 - Convoglio fermo fra due stazioni

situazione un convoglio e' fermo fra due stazioni per una causa qualsiasi.

evento passa il tempo `T_fer` senza che si muova.

comportamento atteso
1. la centrale apre da sola un allarme sul convoglio, senza che nessuno lo abbia segnalato;
2. l'allarme risulta dedotto e non segnalato;
3. finche' la condizione resta, non ne compaiono altri sulla stessa sorgente.

requisiti coperti RF02.1.3, RF02.6.2, RF01.2.2.

1.6.5 SV05 - Modifica dell'itinerario a convogli in marcia

situazione un itinerario con due convogli assegnati, entrambi in marcia in punti diversi.

evento l'amministratore modifica la sequenza delle stazioni e toglie uno dei due convogli.

comportamento atteso
1. il convoglio rimasto assegnato prosegue sul nuovo itinerario dal punto in cui si trovava, senza salti di posizione e conservando il ritardo;
2. il convoglio escluso si ferma e resta in attesa di soppressione, restando visibile;
3. nessuno dei due continua a percorrere l'itinerario vecchio.

requisiti coperti RF01.3.4, RF02.5, RF03.5.

1.6.6 SV06 - Corsa completa con inversione al capolinea

situazione un convoglio all'inizio della corsa, nella stazione di partenza.

evento la corsa viene svolta per intero, andata e ritorno.

comportamento atteso

1. dalla stazione di partenza risulta una uscita senza una entrata che la preceda;
2. per ogni stazione intermedia risultano una entrata e una uscita, in quest'ordine;
3. all'estremo dell'itinerario risulta una entrata; la direzione dichiarata passa da andata a ritorno;
4. la corsa di ritorno percorre le stesse stazioni in ordine inverso;
5. lo storico contiene tutti i transiti nell'ordine reale;
6. se su quell'estremo e' marcato il capolinea non c'e' nessuna inversione: il convoglio si ferma, passa in stato capolinea e la sua entrata resta senza l'uscita corrispondente.

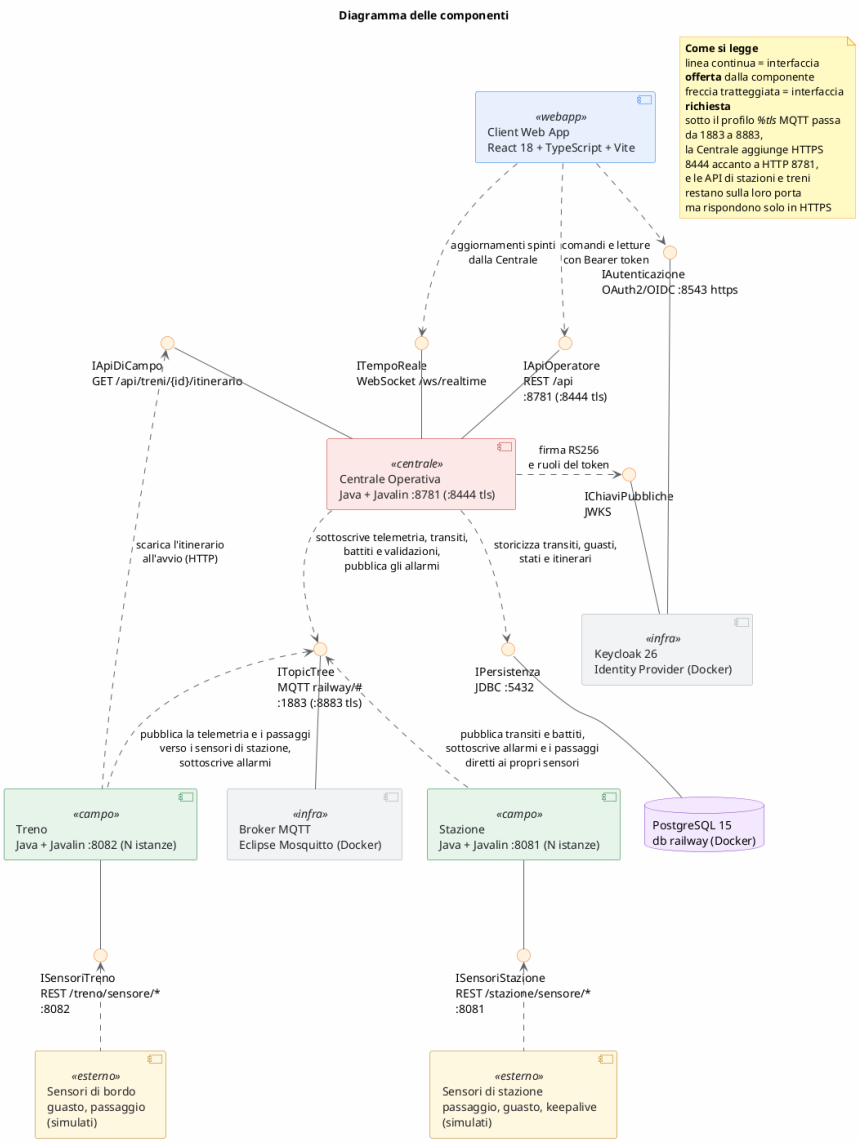
requisiti coperti RF01.5, RF02.2.3, RF02.2.4, RF02.7.

1.6.7 Copertura dei requisiti da parte degli scenari

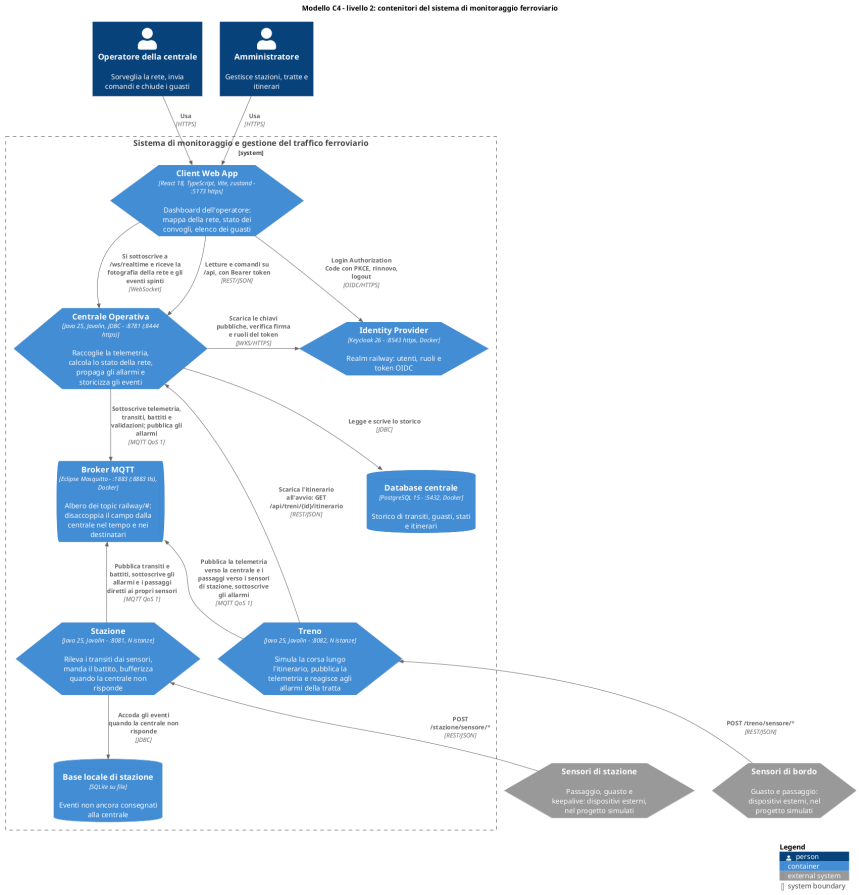
requisito	scenari che lo verificano
RF01.1	SV01, SV05
RF01.2	SV01, SV03, SV04
RF01.3	SV05
RF01.4	SV01
RF01.5	SV06
RF01.6	tutti (accesso preliminare)
RF02.1	SV01, SV02, SV03
RF02.2	SV01, SV03, SV06
RF02.3	SV02
RF02.4	preliminare a tutti
RF02.5	SV05
RF02.6	SV02, SV04
RF02.7	SV02, SV06
RF03	SV02, SV05
RF04	verifica dedicata (RF04.2, RF04.3)

2 Progettazione

2.1 Diagramma delle componenti



2.2 Diagramma dei microservizzi



2.3 Stack tecnologico

a che serve	tecnologia	dove
runtime dei nodi	Java 25, Maven (jar unico con maven-shade-plugin), nessun framework: il `main` crea gli oggetti e ne assegna i campi	Centrale, Stazioni, Treni
messaggistica campo/centrale	MQTT su Eclipse Mosquitto in Docker (1883, 8883 TLS), client Eclipse Paho 1.2.5, payload JSON con Gson	tutti i nodi

Continued on next page

Continued from previous page

a che serve	tecnologia	dove
chiamate sincrone	Javalin 6.7 (+ `ssl-plugin` per l'HTTPS), JSON con Gson	`/api` della Centrale, `/stazione` e `/treno` per i sensori
aggiornamento della dashboard	WebSocket di Javalin (`app.ws`)	`/ws/realtime` della Centrale
persistenza	PostgreSQL 15 in Docker, JDBC puro (driver `postgresql`), `schema.sql` eseguito all'avvio	solo Centrale
stato dei nodi di campo	oggetti in memoria, uno per processo; sulla stazione anche SQLite su file (`sqlite-jdbc`)	`TrainDB`, `DBLocale`, `LocalBuffer`, `PersistenzaLocale`
compiti periodici	`ScheduledExecutorService` del JDK	`Orologio` (treno, stazione), `GeneratoreEventiTempo` (centrale)
log	`java.util.logging` (con `slf4j-jdk14` per le librerie)	tutti i nodi
interfaccia utente	React 18, TypeScript, Vite, zustand	web app
identita' e autenticazione	Keycloak 26 in Docker, OAuth2 / OIDC, Authorization Code + PKCE; verifica del token con pac4j (`javalin-pac4j`, `pac4j-jwt`)	realm `railway`, `SicurezzaKeycloak`, `src/api/keycloak.ts`
autorizzazione	ruoli di realm nel token, before-handler di Javalin sui percorsi	`FiltroAutorizzazione`
configurazione	`application.properties` letto a mano, profilo con `-Dprofilo=tls` e chiavi `%tls.`	`Configurazione` in ognuno dei tre moduli
cifratura dei canali	TLS con CA privata (`gen-certs.sh`)	profilo `tls` (MQTT 8883, HTTPS 8444 sulla Centrale, HTTPS sulle stesse porte per stazioni e treni), Keycloak e web app sempre in HTTPS (8543, 5173)

Continued on next page

Continued from previous page

a che serve	tecnologia	dove
test	JUnit 5 (con il client HTTP del JDK), Vitest con Testing Library	moduli Java, web app
database dei test	H2 in memoria in modalita' PostgreSQL, creato dallo stesso `schema.sql` a ogni prova	`CentraleDiProva` della Centrale

Anche questa documentazione ha il suo piccolo stack: è scritta in org-mode ed esportata in LaTeX, e i diagrammi (stati, sequenza, classi) sono sorgenti PlantUML tenuti dentro il documento stesso e compilati in png.

2.4 Scelte progettuali e giustifiacazioni

2.4.1 Metodi di interfacciamento tra microservizzi

Il broker al posto delle chiamate REST periodiche. I nodi di campo stanno in reti private: una stazione dietro il NAT dell'ente ferroviario non è raggiungibile da fuori e un convoglio in movimento cambia rete di continuo. Con REST sarebbe la Centrale a dover contattare ogni nodo, quindi servirebbero un indirizzo stabile e una porta aperta per ciascuno; con il broker è il nodo ad aprire la connessione verso l'esterno, e sul firewall basta permettere il traffico in uscita. Il broker disaccoppia anche nel tempo — se la Centrale è giù la stazione continua a pubblicare, i messaggi con QoS 1 restano in coda e il `LocalBuffer` li ritenta, mentre un POST fallito sarebbe perso — e nei destinatari: pubblicare su `railway/alerts` raggiunge tutti i treni interessati senza che chi pubblica sappia quanti sono (RNF05), cosa che con REST costringerebbe la Centrale a tenere l'elenco aggiornato dei nodi e a chiamarli uno per uno. Questo è il grande vantaggio del metodo uno a molti del pub/sub

Perché MQTT e non AMQP o Kafka. Il traffico è fatto di messaggi piccoli e frequenti: telemetria ogni `P_tel` per convoglio, cioè circa 10.000 messaggi al secondo con diecimila treni, payload JSON di qualche centinaio di byte. MQTT ha un header fisso di 2 byte, mentre ogni richiesta HTTP ne porterebbe centinaia fra intestazioni e handshake. AMQP aggiunge exchange e instradamenti che qui non servono, Kafka è pensato per conservare e rileggere flussi enormi ed è sovradimensionato per una rete di sensori. I vincoli di tempo sono soft real-time, dell'ordine dei secondi (`P_tel`, `T_sil`): non serve una garanzia dura sulla latenza, bastano la consegna almeno-una-volta e il keepalive che si accorge della caduta.

Il resto. REST resta per rapidità implementativa dove la risposta serve subito e su richiesta (l'itinerario che il treno scarica all'avvio, le API dell'operatore); verso il browser si usa WebSocket, perché la dashboard deve ricevere aggiornamenti spontanei e il polling sprecherebbe banda su dati che spesso non cambiano. PostgreSQL sta solo sulla Centrale, dove lo storico cresce senza limite; alla stazione basta SQLite, che è un file e non un servizio. Keycloak evita di scriversi in casa utenti, password e ruoli, e parla OIDC standard: lo stesso token vale per la web app e per la Centrale.

2.4.2 Sul metodo di repository

Lo stato corrente della rete sta in RAM e non sul database

Il `TrafficLogicEngine` tiene tre mappe in memoria (treni, stazioni, guasti aperti) e le API di lettura rispondono da lì. La domanda naturale è perché non interrogare direttamente PostgreSQL.

Primo: per buona parte di quei dati il database non è un'alternativa, perché non li contiene. Posizione, velocità, ritardo corrente, passeggeri, progresso, condizione e fase del viaggio del convoglio, stato runtime e ultimo battito della stazione non hanno colonne nello schema: la mappa in RAM è il posto in cui quello stato vive, e al riavvio la rete riparte OFFLINE e lo riscopre dai messaggi del campo. I cinque campi con cui la Centrale dichiara la posizione del convoglio (`partenzaCorrente`, `arrivoCorrente`, `direzioneCorrente`, `posizioneNota`, `incertezzaPosizione`) poi sono un **giudizio** rifatto a ogni lettura, che scade dopo `T_pos`: persisterlo non avrebbe senso.

Secondo: la memoria cresce con la topologia, non con il tempo. Una entry per stazione e una per convoglio: con i numeri di una rete vera (circa 2.200 stazioni e 10.000 treni al giorno in Italia) e 400-500 byte per `Treno`, diecimila convogli stanno in circa 5 MB. Quello che cresce senza limite è lo storico, e sta su PostgreSQL.

Terzo: l'alternativa costerebbe un UPDATE per ogni telemetria, circa 10.000 al secondo con diecimila treni, per dati che invecchiano in pochi secondi. Per un sistema a nodo singolo una `ConcurrentHashMap` è la semplificazione ragionevole.

2.4.3 Sul sistema di validazione

I test hanno un database loro, non quello di lavoro Questo perchè i test che si creano i dati da soli e li cancellano appena hanno finito, se il test si interrompesse prima di cancellare i dati, tali avrebbero sporcato lo stato persis-

tente. I test della Centrale accendono una Centrale vera, `CentraleDiProva`: gli stessi oggetti collegati come nel `main`, con tre sostituzioni. Il database e' un **H2 in memoria** in modalita' PostgreSQL, nuovo per ogni prova e creato dallo stesso `schema.sql` della Centrale vera, quindi viene buttato via alla fine; in `pom.xml` la dipendenza `h2` sta in scope `test`. Il broker non c'e': la coda delle pubblicazioni e la WebSocket sono sostituite da due versioni che registrano quello che uscirebbe, e i messaggi del campo si consegnano direttamente a `IngestionService`, come farebbe il client MQTT. Keycloak non c'e': le chiavi del realm sono una coppia RSA generata sul momento, e pac4j verifica i token di prova esattamente come quelli veri. Il server Javalin invece e' vero e ascolta su una porta libera, quindi le richieste passano da HTTP, dal filtro e dal convertitore JSON come in esercizio.

Cosi' `./mvnw test` non ha bisogno di Postgres, del broker ne' di Keycloak accesi, che era il motivo per cui i test erano stati scritti: poterli lanciare senza avere il sistema in piedi.

2.5 Parametri temporali

I requisiti che parlano di tempo si riferiscono a questi parametri e non a numeri fissi. Il valore predefinito e' quello con cui il sistema e' consegnato; cambiarlo non cambia la specifica.

simbolo	significato	valore predefinito
P_{tel}	periodo con cui il convoglio dichiara la propria telemetria	1 s
P_{bat}	periodo del battito della stazione verso la centrale	10 s
P_{kal}	periodo del segnale di vita dei sensori verso la stazione	5 s
T_{sil}	silenzio oltre il quale la centrale considera caduta una stazione	30 s
T_{fer}	permanenza di un convoglio fuori stazione oltre la quale scatta l'anomalia	90 s
T_{pos}	silenzio di un convoglio oltre il quale la centrale non garantisce piu' la sua posizione	15 s
P_{ria}	periodo della fotografia completa spinta verso le schermate	10 s
T_{ric}	intervallo fra due tentativi di recupero dell'itinerario da parte del convoglio	15 s
T_{att}	durata dell'attestato di accesso, scaduta la quale va rinnovato	30 min
T_{ina}	inattivita' oltre la quale il servizio di identita' chiude la sessione	60 min
T_{ses}	durata massima di una sessione, anche se l'utente sta lavorando	10 h

Vincolo fra i parametri: deve valere `T_sil > P_bat`, con un margine che tolleri almeno un battito perso, altrimenti una singola consegna in ritardo verrebbe scambiata per una caduta. Allo stesso modo `T_fer` deve essere maggiore del piu' lungo tempo di percorrenza previsto fra due stazioni, altrimenti una corsa normale verrebbe segnalata come anomalia.

`T_fer` vale sia come durata minima dell'immobilita' prima dell'anomalia, sia come soglia oltre la quale una telemetria non aggiornata conta come immobilita'.

Vale `P_tel < T_pos < T_fer`: passato `T_pos` senza frame il convoglio diventa a posizione incerta, passato `T_fer` scatta l'allarme. Un convoglio che sparisce viene prima **dichiarato ignoto** e solo dopo **segnalato**.

Sui parametri dell'accesso vale `T_att < T_ina < T_ses`: l'attestato scade molto prima della sessione, e la sessione ha comunque una fine.

2.6 Modello

Il sistema non ha un database unico ma tre livelli di persistenza, uno per tipo di nodo. Il database centrale è quello della Centrale (PostgreSQL, schema in `ServeCentraleOperativa/src/main/resources/schema.sql`, che la Centrale esegue a ogni avvio: sono tutte `CREATE ... IF NOT EXISTS`, quindi su un database vuoto crea le tabelle e su uno già popolato non tocca niente; lo esegue anche `popola_db.sh` prima di caricare i dati demo): è lì che stanno l'anagrafica della rete e tutto lo storico. La Centrale ci parla in JDBC puro, e tutte le query stanno in una classe sola, `RailwayRepository`. La stazione ha un suo database locale, un file SQLite con lo schema di `Stazioni/src/main/resources/schema.sql`, gestito dalla classe `PersistenzaLocale`. Il treno invece tiene tutto in memoria dentro `TrainDB`, un oggetto solo per processo, e non ha database.

La differenza fra i due nodi di campo dipende da che cosa tengono in mano. Il treno non possiede niente di unico: l'itinerario lo riscarica dalla Centrale e un riavvio gli costa il viaggio in corso, non un dato. La stazione invece tiene dati che esistono solo li' finche' non li consegna (transiti e guasti accumulati a collegamento giu'), ed e' l'unica a sapere di essere guasta: se ripartisse dai valori di default si direbbe ONLINE e farebbe entrare i treni in una stazione inagibile. Per questo la persistenza locale c'e' sulla stazione e non sul treno.

2.6.1 Il database della Centrale

Le tabelle si dividono in tre gruppi.

Anagrafica (i dati che l'amministratore crea dalla dashboard):

- `Stazione`: id, nome, tipo (capolinea / partenza / normale) e, come estensione, latitudine, longitudine e numero di binari per poterle disegnare sulla mappa (i binari dalla dashboard non si scelgono: una stazione creata li' ne ha uno)

- `Tratte`: un arco fra due stazioni (`StazionePartenzaFK` -> `StazioneArrivoFK`) più il tempo nominale di percorrenza in minuti, che serve poi al calcolo del ritardo

- `Itinerari` e `Itinerario_Tratta`: l'itinerario di per sè è un id e poco altro, il percorso vero è nella tabella associativa, che lega itinerario e tratta e in più tiene la colonna `ordine`. È l'unico modo per dire che la stessa tratta può stare in più itinerari e che dentro un itinerario le tratte hanno una sequenza. L'unica colonna propria è `capolinea`

(estensione), la stazione su cui il servizio si chiude: se è nulla il percorso si ripete all'infinito, se è valorizzata il convoglio che ci arriva termina la corsa. Sta qui e non su `Stazione` perché è una proprietà del percorso: la stessa stazione può chiudere un itinerario ed essere una fermata qualunque di un altro, e infatti non va confusa con `Stazione.tipoCapolineaPartenzaoNormale`, che descrive invece la stazione in sé

- `Treni`: stato del convoglio (attivo / fermo / rotto / in manutenzione / capolinea), itinerario assegnato e posizione corrente. Da notare che `id_convoglio` è insieme chiave primaria e nome del treno (es. "IC 351"): non c'è una colonna nome separata, così il nome è univoco per costruzione, però diventa immutabile (per rinominare un convoglio va cancellato e ricreato)

- `Utenti`: anagrafica degli operatori (nome, cognome, matricola univoca, tipo), **senza password**: credenziali e ruoli stanno nel realm di Keycloak. Serve alle chiavi esterne di `Guasti` e `Storico_Assegnazioni_Guasti` verso `id_utente`; il legame con l'utente di Keycloak è la **matricola**, che viaggia nel token

Operative (le scrive il sistema, non l'amministratore):

- `Transiti`: un treno dentro una stazione, con `tempoEntrata` e `tempoUscita`. La riga nasce quando la stazione segnala l'entrata e viene chiusa alla segnalazione dell'uscita, quindi finché `tempoUscita` è null il treno è ancora in stazione. I due istanti sono quelli scritti dalla stazione nel messaggio, non quelli di arrivo: con lo store and forward i transiti accodati tornano tutti insieme. La colonna `ritardoMinuti` (estensione) congela il ritardo **nell'istante** del passaggio

- `Guasti_Pervenuti_da_treni_o_Staz`: il guasto aperto, con tipo, severità, sorgente (STAZIONE o TRENO più l'id), messaggio, timestamp di apertura e di risoluzione e l'operatore che se ne sta occupando

- `eventi_stazioni`: log degli eventi di stazione (heartbeat perso, guasto, manutenzione), estensione

Le estensioni (`ritardoMinuti`, le coordinate delle stazioni, `capolinea` degli itinerari e il valore `capolinea` nel CHECK di `Treni.stato`) stanno tutte in `schema.sql`, quindi un database creato da `schema.sql` (a mano o da `popola_db.sh`) le ha gia'.

Storici: sono tabelle separate in sola aggiunta, ognuna con la sua chiave `SERIAL` e un campo `ts_storicizzazione`. La scelta è stata di non versionare le tabelle operative con dei flag ma di copiare la riga in una tabella gemella quando lo stato cambia: le query della dashboard sul dato corrente restano semplici e lo storico non le appesantisce. `Storico_Transiti` porta la stessa colonna `ritardoMinuti` di `Transiti`, ed è quella che la pagina Transiti legge (un valore nullo vale zero).

Le tabelle di storico sono otto, ma ne vengono scritte quattro, tutte dal `RailwayRepository`:

tabella	scritta	quando
`Storico_Transiti`	sì	a ogni entrata e a ogni uscita, con il ritardo di quel momento
`Storico_Guasti`	sì	all'apertura e alla chiusura di un guasto
`Storico_Stato_Treni`	sì	a ogni cambio di stato del convoglio
`Storico_Stato_Stazioni`	sì	a ogni cambio di stato della stazione, manutenzione compresa
`Storico_Itinerari`	no	la tabella la crea `schema.sql`, nessuno ci scrive
`Storico_Itinerari_Tratte`	no	idem: è la gemella dell'associativa
`Storico_Assegnazioni_Guasti`	no	servirebbe a dire quale operatore ha preso in carico un guasto, ed è il pezzo di RF02.1.3 che resta scoperto
`Storico_Interventi_Manutenzione`	no	l'intervento è già raccontato dallo storico di stato della stazione

Non c'è nessuna tabella per i singoli frame di telemetria, per scelta: sarebbero migliaia di righe al secondo per un dato che invecchia in pochi secondi (vedi *Sul metodo di repository*).

2.6.2 I db locali dei nodi di campo

Il treno non ha un db locale: la fotografia di dove si trova il convoglio (direzione, stato, stazione corrente, prossima stazione e tratta) è l'oggetto `TrainDB` in memoria, la stessa che viene spedita in telemetria, per il motivo detto sopra.

La stazione ne ha di più perchè deve sopravvivere alla disconnessione, e il suo schema è davvero eseguito: all'avvio `PersistenzaLocale` apre il file `Stazioni/data/stazione-{id}.db` e ci lancia dentro le `CREATE TABLE IF NOT EXISTS` di `schema.sql`.

- `Stato`: nome, regione, se è funzionante e i timestamp dell'ultimo guasto. È la riga che fa ripartire GUASTA una stazione che si era guastata prima di spegnersi
- `Tratte`: copia locale delle tratte che toccano la stazione (stessi id del db centrale). La tabella viene creata ma resta vuota: il nodo non ragiona per tratte, il grafo della rete è roba della Centrale
- `Guasto_Locale_Momentaneo` e la associativa `Tratta_Guasto` (vuota): il registro dei guasti aperti della stazione, quelli dichiarati dal nodo e quelli con cui la Centrale la chiude. Una riga si chiude con il RESOLVED della Centrale oppure, per i guasti dei sensori, quando i sensori tornano a battere. L'allarme con cui la stazione trattiene un treno appena entrato non apre righe. Al riavvio dal registro si ricostruisce anche se la stazione era guasta solo per i sensori
- `Treno`: i treni presenti al momento, con tempo di entrata e di partenza. Chi ha il tempo di partenza a NULL è ancora fermo sui binari, ed è da questa query che al riavvio si ricostruisce la mappa dei presenti
- `bufferChache`: la coda degli eventi non ancora consegnati alla Centrale. Nel codice è la classe `LocalBuffer`, il buffer FIFO di uno schema produttore/consumatore, in cui ogni elemento porta il canale di destinazione ("transit" o "alert"). Ci passano tutti i transiti e gli allarmi: la riga si cancella appena il broker ha preso l'evento

Il criterio: sul disco va solo quello che al riavvio non si sa più ricavare. Stato operativo, treni sui binari e coda vengono riletti; connessione MQTT, validazione dell'ID e keepalive dei sensori si rifanno da capo (rileggere battiti vecchi di ore farebbe dichiarare guasti sensori sani).

2.7 Topic

La comunicazione fra i nodi passa da un broker Mosquitto (localhost:1883, oppure 8883 con TLS sotto il profilo `tls`, cioè lanciando i nodi con `-Dprofilo=tls`). Ogni nodo ha una sola connessione, fatta con il client Eclipse Paho e chiusa dentro la classe `ClientMqtt`. Tutti i topic stanno sotto il prefisso `railway/` e sono organizzati per nodo: `railway/train/{id}/...` e `railway/station/{id}/...`. Ogni nodo pubblica sul proprio topic, la Centrale si iscrive con la wildcard `+` (per esempio `railway/train/+/telemetry`) e riceve i messaggi di tutti senza sapere quanti nodi ci sono. Il payload è sempre JSON.

Topic del treno, tutti sotto il prefisso `railway/train/{id}/`:

suffisso	flusso	contenuto
validation	treno -> centrale	richiesta di validazione dell'id all'avvio
validation-response	centrale -> treno	esito: se l'id non è a db il nodo non entra
telemetry	treno -> centrale	posizione, velocità, progresso, stato, condizione e fase del viaggio, passeggeri, fermate e direzione (il ritardo no: lo calcola la centrale da stazione e progresso)

Il treno pubblica anche `railway/treno-simulatoriSensori/{id}`, che alla Centrale non arriva: è il modo con cui la simulazione mette il convoglio davanti ai **sensori di binario** della stazione. Le stazioni si iscrivono con `railway/treno-simulatoriSensori/+` e tengono solo i messaggi con il proprio `stazioneId`.

Topic della stazione, sotto il prefisso `railway/station/{id}/`:

suffisso	flusso	contenuto
validation	stazione -> centrale	come sopra, lato stazione
validation-response	centrale -> stazione	esito della validazione
heartbeat	stazione -> centrale	battito periodico, il silenzio vale come guasto
transit	stazione -> centrale	passaggio del treno visto dai sensori

Le coppie `validation` / `validation-response` sono l'unico scambio richiesta-risposta: la Centrale ascolta tutte le richieste con un solo filtro (`railway/train/+/validation`), cerca l'id a database e risponde sul topic del mittente.

Fuori da questi due gruppi c'è un unico topic senza id, `railway/alerts`: il canale broadcast dei guasti, a cui sono iscritti tutti e tre i tipi di nodo, perché un guasto di stazione interessa tutti i treni. Il tipo di messaggio sta nel campo `tipoEvento`. Negli alert che genera la Centrale mette il marcatore `"origine": "CENTRALE"`, con cui scarta l'eco dei propri messaggi; la stazione, al contrario, reagisce solo agli alert che lo portano.

La qualità del servizio non è la stessa per tutti. `railway/alerts` viaggia a QoS 1, cioè con la conferma del broker, e ogni nodo si collega con una sessione persistente (`cleanSession=false` e un client-id fisso: `treno-<ID>`,

stazione-<ID>, centrale-operativa): gli alert arrivati mentre un nodo era staccato il broker glieli consegna alla riconnessione, perché nessuno li ripete. Per questo i destinatari si registrano **prima** di collegarsi. Tutto il resto (telemetria, battiti, passaggi, transiti, validazioni) va a QoS 0: si ripete da solo o, per i transiti, resta nella coda della stazione finché non viene consegnato. Nessun nodo pubblica dal thread delle callback di Paho: i messaggi in uscita passano da una coda (`CodaPubblicazioni` sul treno e sulla Centrale, `LocalBuffer` sulla stazione) svuotata da un thread suo, perché una pubblicazione a QoS 1 aspetta la conferma che porta proprio quel thread.

Un messaggio `GUASTO` porta `sorgenteTipo` (STAZIONE o TRENO), `sorgenteId`, `severita`, `messaggio` e `timestamp`, più un campo facoltativo `tipoGuasto` con cui **chi segnala dichiara di che guasto si tratta**. La stazione lo valorizza in due casi: `sensore_offline`, dedotto in automatico quando scade il keepalive di un sensore di binario, e `binario_guasto`, dichiarato dall'operatore tramite la CLI della stazione (endpoint `/stazione/sensore/guasto`) quando a guastarsi non è l'intera stazione ma un singolo binario o scambio — la stessa CLI, con l'altra scelta, dichiara `stazione_guasta`. Con questo campo la Centrale classifica l'allarme senza indovinarlo dal testo; se manca, il tipo lo decide il tipo di sorgente.

Un messaggio `RITARDO` è l'unico del canale che non parla di guasti: la Centrale comunica al convoglio il ritardo che gli ha misurato. Porta `target` (il treno destinatario, come `STOP` e `ITINERARIO_AGGIORNATO`), `ritardoMinuti` con il totale, `variazioneMinuti`, `causa` (`ARRIVO_IN_RITARDO` se il tempo si è perso in linea, `SOSTA_PROLUNGATA` se in banchina) e `sorgenteId`, la stazione del passaggio in cui è stato misurato. Serve perché la consegna vuole il convoglio **"in grado di ricevere informazioni (guasti o ritardi)"**, e l'orario di servizio ce l'ha solo la Centrale. A bordo il numero si ferma al display. L'alert parte solo quando il ritardo misurato cambia.

Un messaggio `STATO_STAZIONE` dice **in che stato la Centrale ha messo** una stazione. Porta `sorgenteId` e `stato` (uno dei quattro del vocabolario) e parte da tutti i punti in cui lo stato lo decide lei: l'invio della squadra, la PUT della stazione, il ritorno a ONLINE alla chiusura dell'allarme e quello dichiarato dal nodo con il battito, se non le resta nessun allarme aperto. Serve perché la MANUTENZIONE non nasce da nessun `GUASTO` del campo: senza questo annuncio i treni non la vedrebbero.

Ne segue che il `RESOLVED` di un guasto **non è mai** il permesso di ripartire: sulla stazione può esserci ancora la squadra o un secondo allarme aperto, e quel messaggio lo pubblica anche il nodo di terra quando i sensori tornano a battere. A liberare i treni è soltanto lo `STATO_STAZIONE` che dichiara la

stazione ONLINE.

Lo stesso passaggio fisico attraversa tre canali in fila: su `railway/treno-simulatoriSensori/{id}` (MQTT) il convoglio **passa**, con `POST /stazione/sensore/treno` (REST) il sensore di binario **rileva**, su `railway/station/{id}/transit` (MQTT) la stazione **certifica** alla Centrale. Se il sensore è morto il transito non lo registra nessuno, ed è per questo che una stazione senza sensori è una stazione guasta (RF03.3).

2.8 Vocabolario degli stati osservabili

2.8.1 Stato del convoglio

stato	significato
FERMO	e' in sosta in una stazione, oppure e' trattenuto e non puo' ripartire
$\text{IN}_{\text{VIAGGIO}}$	sta percorrendo una tratta elementare fra due stazioni
EMERGENZA	e' aperto un guasto che lo riguarda; la corsa e' sospesa dove si trovava
SOPPRESSO	la corsa e' stata annullata: il convoglio non viaggia piu' e non riprendera'
CAPOLINEA	e' arrivato all'estremo che chiude il servizio: la corsa e' finita, il convoglio sta bene

2.8.2 Stato operativo della stazione

La stazione e' vista da due punti di osservazione, che dispongono di informazione diversa. Non e' una contraddizione: e' la conseguenza di che cosa ciascuno dei due puo' sapere.

stato	chi lo puo' dichiarare	significato
ONLINE	la stazione, la centrale	la stazione e' viva, raggiungibile e percorribile
GUASTA	la stazione, la centrale	la stazione non e' percorribile: i convogli non ci entrano e non ne escono
OFFLINE	solo la centrale	la stazione non da' piu' notizie di se'
MANUTENZIONE	solo la centrale	la squadra e' sul posto e sta lavorando: non e' percorribile, come GUASTA

2.8.3 Stato composto mostrato dall'interfaccia

I cinque valori dello stato del convoglio sono un **riassunto**: `TrainDB.stato()` li calcola dalla condizione (T6) e dalla fase del viaggio (T5), e nel calcolo si perde qualcosa. FERMO vale per il convoglio in sosta al binario, per quello trattenuto al binario e per quello trattenuto a meta' tratta; EMERGENZA nasconde la fase in cui il convoglio si e' fermato. Un elemento pero' non sta mai in uno stato solo: sta in uno stato per ognuno dei suoi automi, tutti nello stesso momento, ed e' questo **stato composto** che l'interfaccia stampa, un badge per automa.

Per il convoglio i valori arrivano dalla telemetria, che oltre al riassunto porta `condizione` e `faseViaggio`; il verso c'era gia'.

automa	che cosa si stampa
T6 condizione	REGOLARE, EMERGENZA o SOPPRESSO
T5 fase del viaggio	una delle sei fasi: $\text{SENZA}_{\text{ITINERARIO}}$, $\text{IN}_{\text{STAZIONE}}$, $\text{IN}_{\text{VIAGGIO}}$, $\text{BLOCCATO}_{\text{INSTAZIONE}}$, $\text{BLOCCATO}_{\text{INTRATTA}}$, $\text{FERMO}_{\text{CAPOLINEA}}$
T7 verso di marcia	ANDATA o RITORNO, solo nelle quattro fasi in cui il convoglio percorre l'itinerario: senza itinerario un verso non c'e', e al capolinea T7 si e' chiusa

Il convoglio trattenuto da una stazione guasta davanti a lui, per esempio, si legge REGOLARE, $\text{BLOCCATO}_{\text{INTRATTA}}$, ANDATA. Finche' un convoglio non ha trasmesso dopo l'avvio della centrale resta il valore della colonna Treni.stato (M13).

Per la stazione lo stato composto e' quello di M12 e, solo in MANUTENZIONE, anche quello che il nodo dichiara nel battito (S10): negli altri stati la

centrale accetta quello che il nodo dichiara e i due coincidono, in MANUTENZIONE no (`accettaDalCampo`). Da OFFLINE il nodo non batte, e il suo ultimo stato non si stampa.

2.9 Diagrammi degli stati

Come si leggono. Nessun componente è una macchina sola: il treno ne ha 11, la stazione 11, la centrale 13, e ogni diagramma ritaglia una variabile o un flusso. I richiami alle altre macchine sono le giunzioni fra un ritaglio e l'altro, e sono quattro:

`T2 - evento` **in testa all'etichetta** il **trigger** nasce altrove, ma l'arco resta qui: partenza e arrivo sono stati di questo diagramma.

`(T11)` **in coda, dopo la barra** l'**effetto** cambia una variabile di un'altra macchina, dove lo stesso fatto è disegnato come transizione vera.

riquadro grigio `<<esterna>>` **raggiunto da una freccia tratteggiata** l'arco **esce**: quel riquadro non è uno stato ma il segnaposto di un'altra macchina, spesso in un altro processo.

`[... T9]` **dentro una guardia** la condizione **legge** una variabile di un'altra macchina.

Nessuno stato è condiviso fra due macchine: ogni stato è il valore di una variabile che appartiene a una macchina sola, e i nomi che si ripetono (`Ascolto`, `Attesa`, `Errore`) sono omonimi locali. `ONLINE` e `GUASTA` stanno sia in S10 sia in M12 perché sono due copie in processi diversi, che devono poter divergere.

Le tre mappe (T0, S0, M0) non sono macchine a stati ma l'indice del gruppo: i riquadri sono macchine e non stati, e la freccia vuol dire "avvia" o "alimenta", mai "transita in". Il processo non lascia mai T1, e infatti in T1 le altre macchine stanno **dentro** lo stato "Servizio operativo".

2.9.1 Del Treno

- nota : gli stati del treno sono simulati, al posto delle simulazioni degli eventi nella realta tali dovrebbero essere implementati attraverso l'ascolto dei sensori di bordo

- il componente e' stato astratto in 11 macchine, ognuna con la sua sezione; la mappa qui sotto dice chi avvia chi

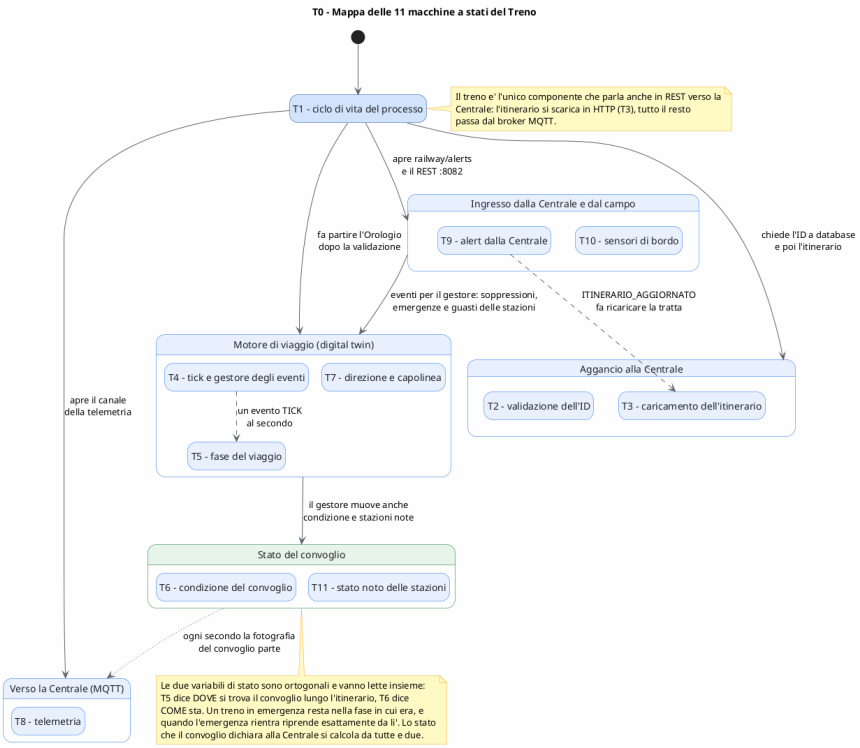
- si leggono in quattro gruppi: l'aggancio alla centrale (T2 e T3), il motore di viaggio che simula il convoglio (T4, T5, T7), i canali verso e dalla centrale (T8, T9, T10) e lo stato del dominio (T6, T11)

- le due variabili di stato non vanno confuse: T5 dice DOVE sta il convoglio lungo l'itinerario (faseViaggio), T6 dice COME sta (condizione: regolare, in emergenza, soppresso). Sono due campi distinti del TrainDB e cambiano per cause diverse, ed e' esattamente l'ortogonalita' che la specifica chiede in RF02.2.2.1 e RF02.2.2.3. Lo stato che il convoglio dichiara fuori (FERMO, $\text{IN}_{\text{VIAGGIO}}$, EMERGENZA, SOPPRESSO, CAPOLINEA) non e' una terza variabile: TrainDB.stato() lo calcola da quelle due

- nel codice le macchine che cambiano lo stato del convoglio sono classi con soltanto guardie (metodi che leggono) e mosse (metodi che scrivono): AutomaViaggio per T5 e T7, AutomaCondizione per T6, AutomaStazioniNote per T11. Le muove solo GestoreEventi (T4, dove sono descritte le tre fasi di ogni evento), con gli eventi di tre generatori: GeneratoreEventiTempo (T4 e T3), TrainGateway (T9) e TrainIngestion (T10)

2.9.1.1 mappa delle macchine

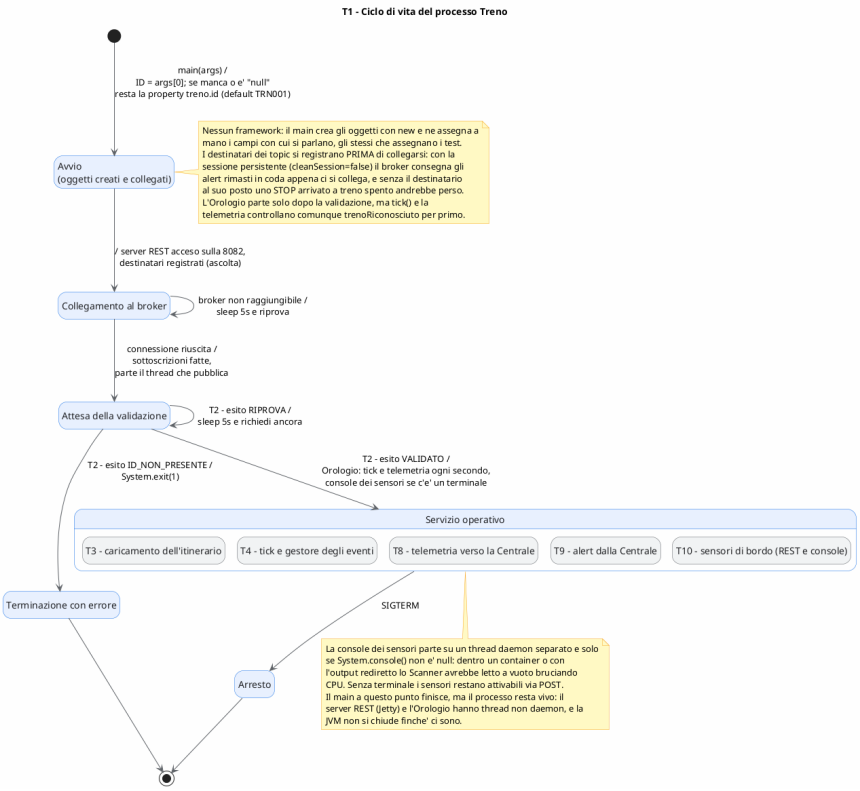
- requisiti coperti: nessuno in proprio, e' l'indice delle altre undici: i requisiti veri stanno scritti sopra le singole macchine



2.9.1.2 T1 ciclo di vita del processo treno

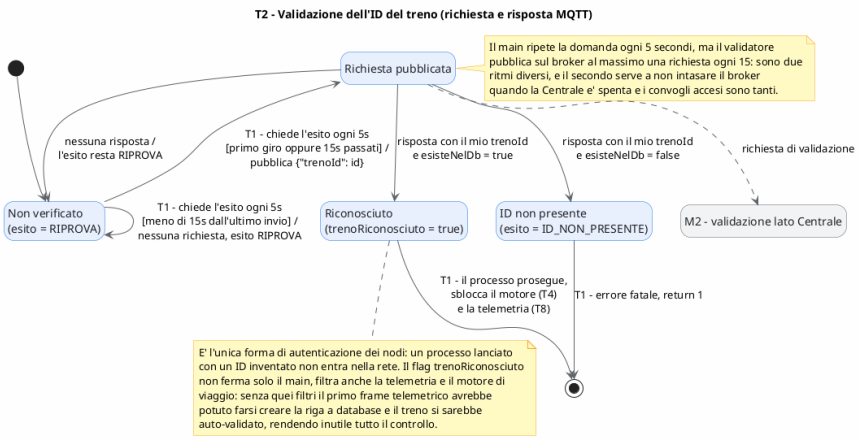
- e' la macchina radice: legge l'ID dagli args o dalla property, crea gli oggetti e ne assegna i campi, accende il server REST e il client MQTT, aspetta il via libera della centrale e solo allora fa partire l'Orologio, che da quel momento fa girare il tick del viaggio e la telemetria (main)

- requisiti coperti: RF02.4.1 e RF02.4.2 parzialmente, qui c'e' solo la guardia all'avvio e la scelta di terminare invece di restare acceso senza poter comunicare; RF03.1 parzialmente, il convoglio e' un processo a se', avviabile e arrestabile da solo. La conversazione vera e' in T2



2.9.1.3 T2 validazione dell'id sul database centrale

- richiesta e risposta su MQTT: il processo prosegue solo se la centrale conferma che l'ID esiste a database, altrimenti termina con codice 1 (TrainDatabaseValidator)
- requisiti coperti: RF02.4.1 e RF02.4.2 completi per la parte del treno, l'altra meta' della conversazione e' M2 lato centrale

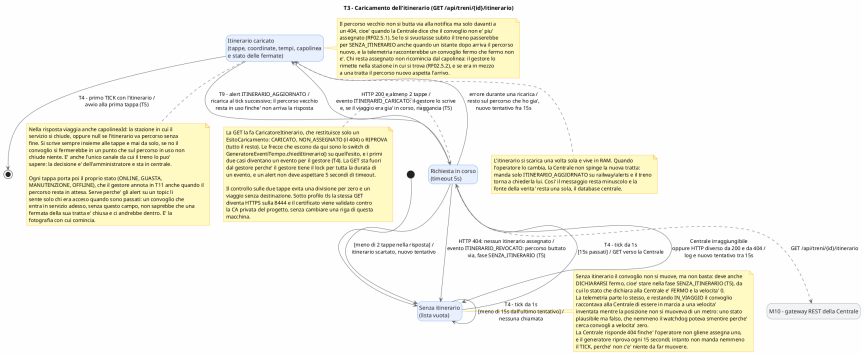


2.9.1.4 T3 caricamento dell'itinerario

- unica conversazione REST del treno verso la centrale: GET /api/treni/{id}/itinerario, con retry ogni 15s e ricarica completa quando l'operatore cambia la tratta. La chiamata la fa CaricatoreItinerario, chiamato da GeneratoreEventiTempo fuori dal gestore degli eventi (puo' durare fino a 5 secondi), e l'esito diventa un evento: $\text{ITINERARIO}_{\text{CARICATO}}$ con il percorso, oppure $\text{ITINERARIO}_{\text{REVOCATO}}$ per il 404. Che cosa farne lo decide GestoreEventi (T4)

- finche' l'itinerario manca il convoglio resta nella fase $\text{SENZA}_{\text{ITINERARIO}}$ (T5), che e' quella da cui parte il processo e quella in cui lo mette il 404: si dichiara fermo, se no continuerebbe a mandare telemetria da treno in marcia stando immobile

- requisiti coperti: RF03.5 completo lato treno, il convoglio non conosce l'itinerario per conto proprio ma lo riceve dalla centrale, che ne resta l'unica depositaria; RF02.5.2 completo, il gestore decide che cosa fare del percorso appena scaricato e AutomaViaggio.fermataDiRiaggancio() rimette il convoglio nella stazione in cui si trova invece di far ripartire il viaggio dalla prima tappa; il 404 (convoglio sganciato) e' l'unico caso in cui l'itinerario in memoria viene buttato via



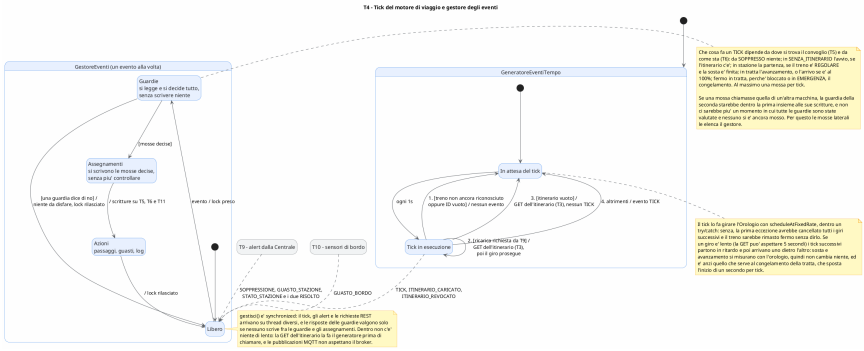
2.9.1.5 T4 tick e gestore degli eventi

- il battito da 1 secondo del digital twin e il punto da cui passano tutte le mosse delle macchine del convoglio. GeneratoreEventiTempo.tick() fa tre controlli e poi manda l'evento TICK; GestoreEventi.gestisci() lo gestisce insieme a tutti gli altri eventi (l'itinerario di T3, gli alert di T9, il guasto di bordo di T10), uno alla volta

- ogni evento attraversa tre fasi fisse: guardie, assegnamenti, azioni. Le guardie di tutte le macchine coinvolte si valutano prima di scrivere qualsiasi cosa (il punto in cui si ritrovera' il convoglio si calcola in variabili locali): se una dice di no si esce senza aver toccato niente, quindi non resta mai una mossa a meta'. Le azioni (passaggi, guasti, log) partono dopo le scritture e vedono lo stato gia' coerente

- le macchine non si chiamano mai fra loro: quando un evento ne muove piu' d'una, come l'arrivo in stazione che installa anche l'itinerario rimasto in attesa, e' il gestore a elencarle, e le loro guardie stanno tutte nella prima fase

- requisiti coperti: nessuno per intero, e' l'orchestratore: RF02.5.1 parzialmente (da soppresso il TICK non muove piu' niente), RF02.5.2 parzialmente (da qui parte la ricarica, e all'arrivo in stazione viene installato l'itinerario rimasto in attesa perche' il treno era a meta' tratta) e RF02.1.2.2 parzialmente (l'EMERGENZA congela l'avanzamento, pero' congela il convoglio e non l'infrastruttura, quindi RF02.1.2.2.1 e RF02.1.2.2.2 restano scoperti)



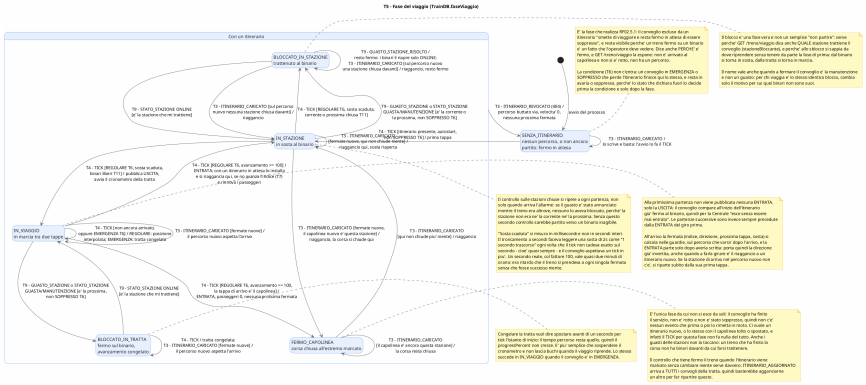
2.9.1.6 T5 fase del viaggio

- il cuore della simulazione, cioe' dove si trova il convoglio lungo l'itinerario: comprende il trattenimento per stazione chiusa e l'attesa di un percorso quando l'itinerario non c'e'

- le fasi ammesse sono sei, e sono i valori che `TrainDB.faseViaggio` puo' assumere: $\text{SENZA}_{\text{ITINERARIO}}$, $\text{IN}_{\text{STAZIONE}}$, $\text{IN}_{\text{VIAGGIO}}$, $\text{BLOCCATO}_{\text{INSTAZIONE}}$, $\text{BLOCCATO}_{\text{INTRATTA}}$ e $\text{FERMO}_{\text{CAPOLINEA}}$. Il blocco e' diviso in due perche' le due situazioni si comportano in modo diverso: al binario il treno aspetta e basta, a meta' tratta ha un avanzamento da congelare, e allo sblocco riprende la sosta oppure la marcia

- la scrive solo AutomaViaggio, e solo quando glielo chiede il gestore degli eventi (T4). La condizione del convoglio (T6) e' un'altra variabile: qui la si legge nelle guardie, ma nessuna mossa di questa macchina la scrive, quindi un blocco o un avvio non possono cancellare un'EMERGENZA

- requisiti coperti: RF02.2.2.1 completo (dove si trova il convoglio); RF02.2.1.1 completo insieme alla Centrale: il convoglio trattenuto resta fermo pur avendo una tappa da percorrere, e il ritardo lo fa crescere la Centrale (M3), che dalla telemetria vede la sosta allungarsi o l'avanzamento fermo mentre l'orario corre; RF02.1.1.2.1 e RF02.1.1.2.2 completi insieme a T9 e T11: non entra nella stazione non percorribile e riparte solo al ripristino, non per scadenza di un tempo; RF02.5.1 completo per la parte del convoglio, che con la fase $\text{SENZA}_{\text{ITINERARIO}}$ smette davvero di viaggiare invece di dichiararsi in marcia stando fermo

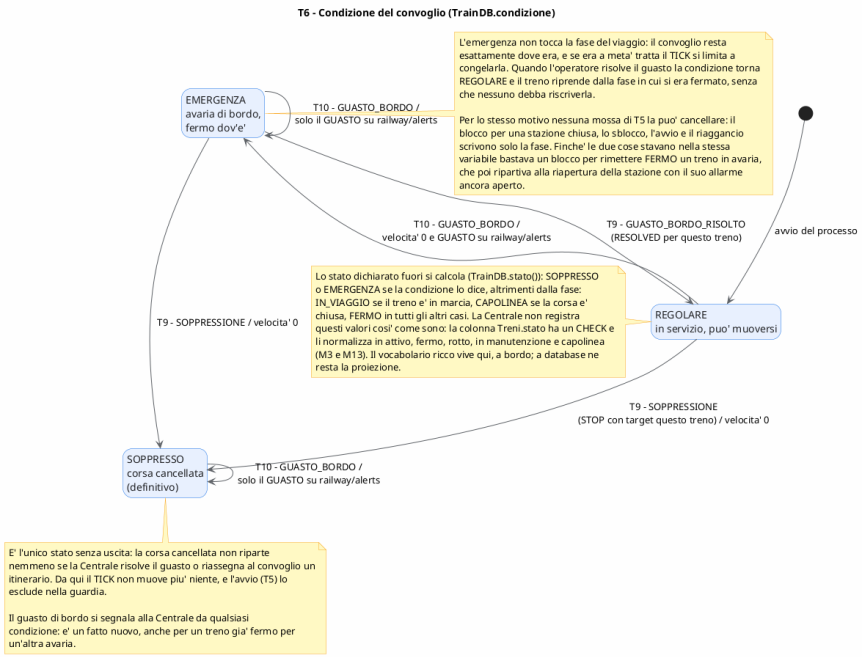


2.9.1.7 T6 condizione del convoglio

- macchina di dominio: come sta il convoglio, cioe' `TrainDB.condizione` (REGOLARE, EMERGENZA, SOPPRESSO). Dove si trova lo dice T5, e le due variabili non cambiano mai con lo stesso evento: la condizione la scrive solo AutomaCondizione, e le mosse di T5 la leggono soltanto

- lo stato che il convoglio dichiara fuori (telemetria e REST) non e' una terza variabile: `TrainDB.stato()` lo calcola da questa macchina e da T5. Con EMERGENZA o SOPPRESSO vale la condizione, che vince su tutto; da REGOLARE si legge la fase: $\text{IN}_{\text{VIAGGIO}}$ da' $\text{IN}_{\text{VIAGGIO}}$, $\text{FERMO}_{\text{CAPOLINEA}}$ da' CAPOLINEA, tutto il resto FERMO. Sulla centrale questi cinque valori vengono normalizzati in quelli ammessi dalla colonna Treni.stato (M13)

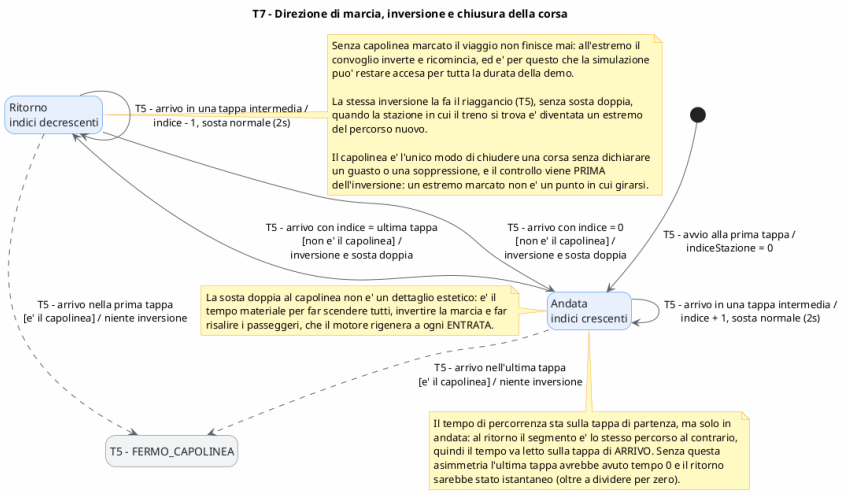
- requisiti coperti: RF02.2.2.3 completo, e' lo stato del convoglio tenuto distinto dalla posizione che sta in T5, come la specifica richiede; RF02.1.2 parzialmente, qui nasce l'EMERGENZA ma la registrazione in centrale (RF02.1.2.1) sta in M6 e M13; RF02.5.1 parzialmente, SOPPRESSO e' lo stato definitivo del convoglio fermato in attesa di soppressione



2.9.1.8 T7 direzione di marcia e capolinea

- andata e ritorno sullo stesso itinerario: all'estremo la direzione si inverte e la sosta raddoppia, e il tempo di percorrenza si legge sulla tappa opposta; se pero' quell'estremo e' il capolinea dell'itinerario non ci si gira, la corsa finisce li'

- requisiti coperti: RF02.2.3 completo (raggiunto il capolinea la marcia si inverte e ogni tappa usa il tempo di percorrenza del verso che sta percorrendo) e RF02.2.4 parzialmente, perche' e' qui che nascono i passaggi spaiati dei due capolinea; e' inoltre di supporto a RF02.2.2.1, perche' tiene coerenti indice, direzione e tempo di percorrenza, senza i quali la posizione riportata sarebbe sbagliata

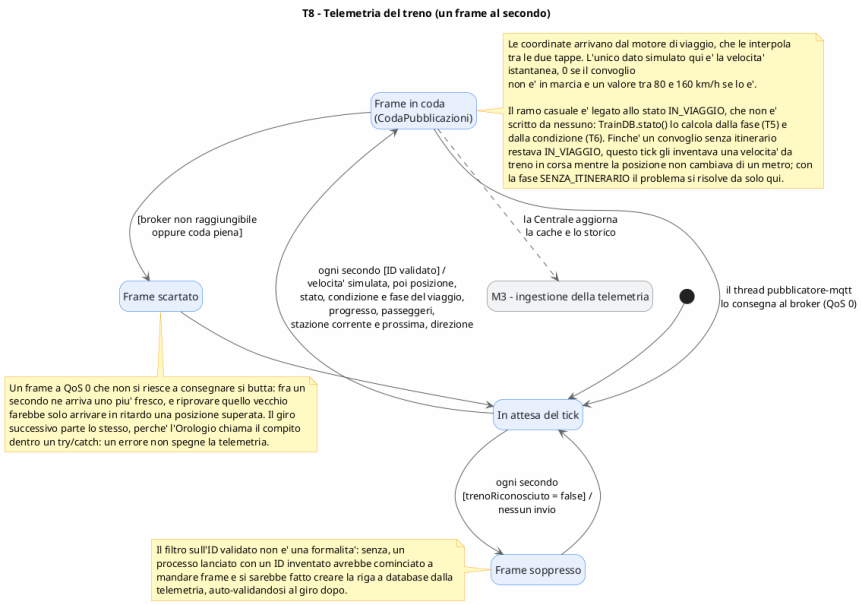


2.9.1.9 T8 telemetria verso la centrale

- il frame ogni secondo con posizione, velocita', progresso, stato e passeggeri: e' cio' che alimenta la mappa dell'operatore (TrainElab, fatto girare dall'Orologio). Il frame non si pubblica direttamente: entra nella CodaPubblicazioni e lo consegna al broker il suo thread

- oltre allo stato, riassunto calcolato da `TrainDB.stato()`, il frame porta `condizione` (T6) e `faseViaggio` (T5): dal riassunto non si torna indietro (FERMO vale sia in sosta al binario sia a meta' tratta), e con i due campi l'interfaccia stampa lo stato composto (T6, T5 e il verso di T7)

- requisiti coperti: RF02.2.2.1 e RF02.2.2.4 parzialmente (posizione, velocita' e passeggeri: qui c'e' la sorgente del dato, mentre la correttezza e la visualizzazione si chiudono in M3 e M11; il ritardo invece nel frame non c'e': la centrale lo rifa' a ogni frame in M3 a partire dalla stazione o dalla quota di tratta dichiarate, lo misura al passaggio in M5 e lo comunica al convoglio in T9), RF03.3 parzialmente perche' la dichiarazione del convoglio e' la fonte di verita' per la posizione corrente, quella che alimenta il tempo reale

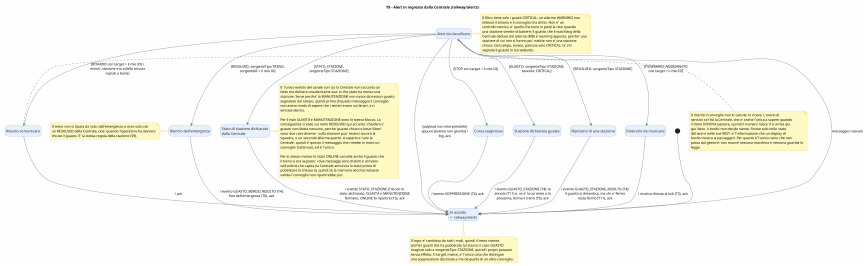


2.9.1.10 T9 alert in ingresso dalla centrale

- il treno in ascolto su railway/alerts: soppressioni, guasti delle stazioni, stati dichiarati dalla centrale, ripristini, itinerari aggiornati e il ritardo che la Centrale gli comunica (TrainGateway.riceviAlert)

- il listener non decide come reagire: filtra gli alert che riguardano questo treno e li traduce in eventi per il gestore (T4). Se la stazione e' la corrente o la prossima, se il convoglio si blocca o riparte, lo decidono le guardie di T5 e T6

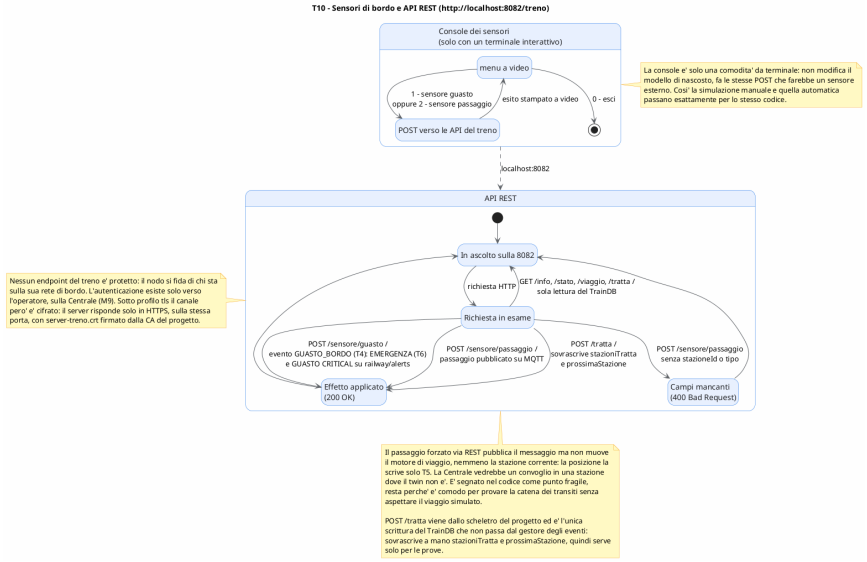
- requisiti coperti: RF02.5.1 completo lato treno (lo STOP con il mio target), RF02.1.1.2.1, RF02.1.1.2.2 e RF02.2.1.1 parzialmente (porta al gestore gli eventi da cui nascono il blocco e lo sblocco, decisi ed eseguiti in T5), RF02.1.2 parzialmente (dal RESOLVED si esce dall'emergenza), RF02.5.2 parzialmente (qui arriva solo l'avviso, la ricarica e' T3), RF01.4.1 lato treno perche' e' da qui che il convoglio viene a sapere della squadra di manutenzione; e' anche il punto in cui si chiude la richiesta della consegna di un convoglio capace di ricevere "guasti o ritardi", perche' il ramo RITARDO e' la seconda meta'



2.9.1.11 T10 sensori di bordo e api rest

- i sensori di bordo non toccano il modello: passano sempre dalle API REST della 8082, sia dalla console interattiva sia da un chiamante esterno

- requisiti coperti: RF03.4 completo lato treno, i sensori di bordo non toccano il modello ma passano sempre dall'interfaccia pubblica del nodo, ed e' cio' che rende il nodo verificabile dall'esterno; RF02.1.2.1 parzialmente, e' il punto in cui il guasto del convoglio nasce, mentre la registrazione e le conseguenze sono lato centrale



2.9.1.12 T11 stato delle stazioni noto al treno

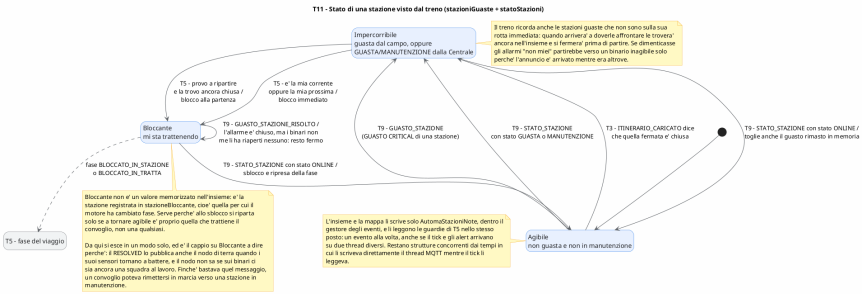
- macchina di entita': la storia di una stazione nella memoria del convoglio, cioe' il motivo per cui il treno non riparte verso un binario

ancora inagibile

- le memorie sono due e restano separate apposta: l'insieme delle stazioni che il treno ha visto dichiararsi guaste (dal campo) e la mappa dello stato che la centrale ha dichiarato per ciascuna. Una stazione puo' avere il guasto risolto e i tecnici ancora sul posto, e in quel caso il convoglio non deve ripartire

- di conseguenza le due memorie non hanno lo stesso peso: la chiusura di un allarme non e' un permesso di ripartire, mentre lo `STATO_STAZIONE` agibile lo e' - ed e' l'unico, tanto che cancella anche il guasto rimasto nell'insieme

- requisiti coperti: RF02.1.1.2.1, RF02.1.1.2.2 e RF02.1.1.2.3 completi insieme a T5 e T9: e' la memoria che fa fermare il convoglio anche quando l'annuncio era arrivato mentre stava altrove, e quella che gli fa trovare la fotografia gia' giusta se si accende dopo



2.9.2 Della Stazione

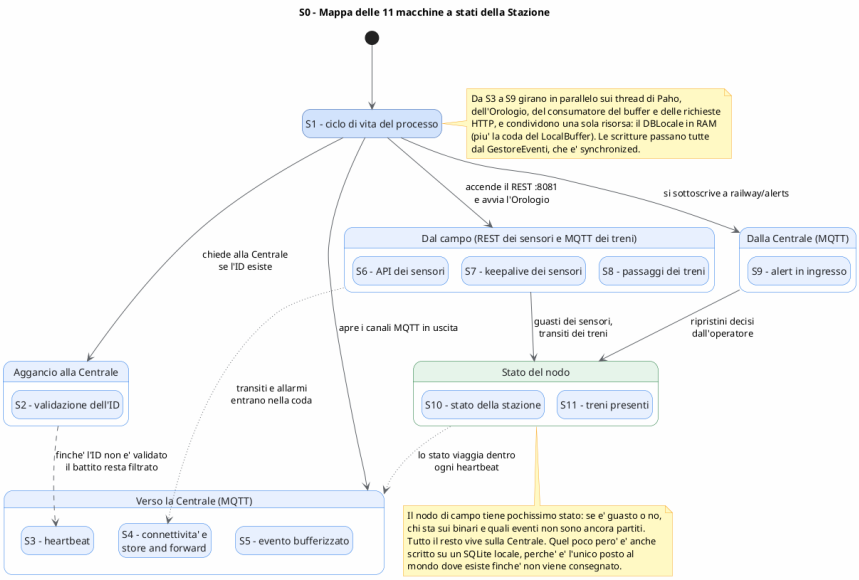
- il componente e' stato astratto in 11 macchine, ognuna con la sua sezione; la mappa qui sotto dice chi avvia chi

- si leggono in quattro gruppi: l'avvio e l'aggancio alla centrale (S1 e S2), i canali verso la centrale con il caching locale (S3, S4, S5), cio' che arriva dal campo e dall'operatore (S6, S7, S8, S9) e lo stato del nodo (S10, S11)

- la stazione e' il nodo che deve sopravvivere alla caduta della rete: S4 e S5 sono esattamente la parte che il testo chiede, cioe' il caching locale con reinvio (RF02.1.1.1.1 per il non perdere niente, RF02.1.1.1.2 per il consegnarlo nell'ordine giusto)

2.9.2.1 mappa delle macchine

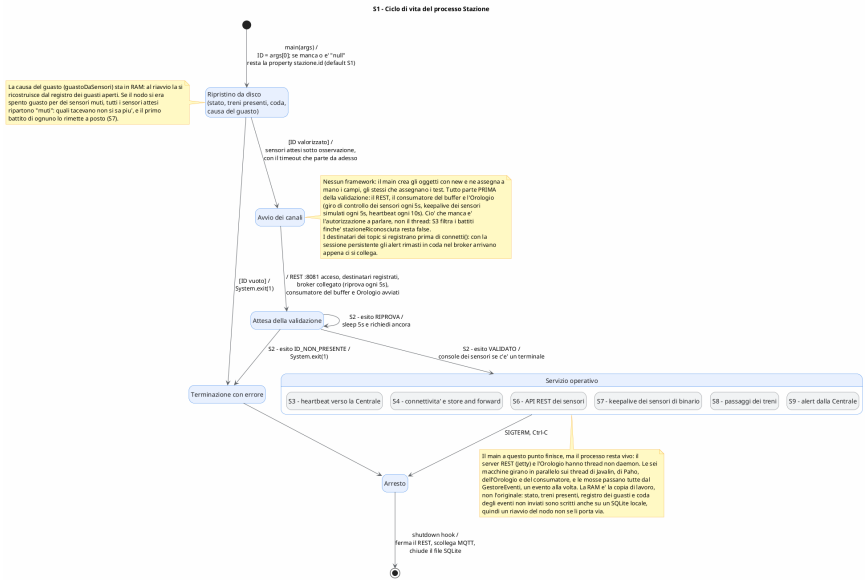
- requisiti coperti: nessuno in proprio, e' l'indice delle altre undici: i requisiti veri stanno scritti sopra le singole macchine



2.9.2.2 S1 ciclo di vita del processo stazione

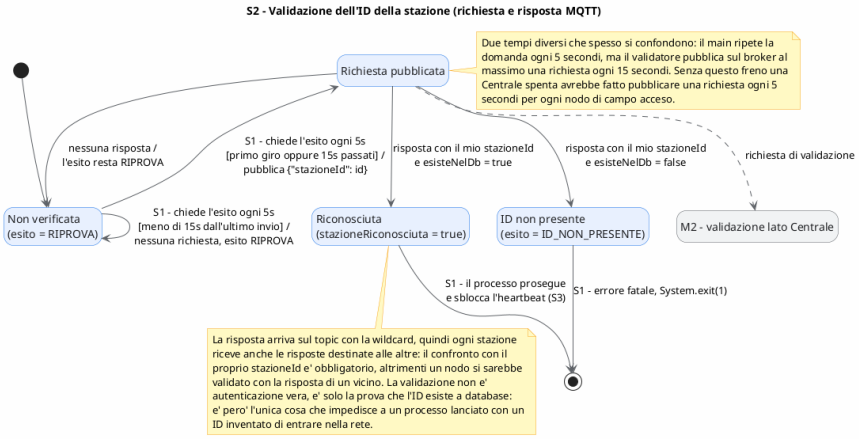
- e' la macchina radice: legge l'ID dagli args o dalla property, rimette il nodo dov'era leggendo il db locale, accende il server REST dei sensori, si collega al broker, fa partire l'Orologio e solo alla fine aspetta il via libera della centrale (main)

- requisiti coperti: RF02.4.1 e RF02.4.2 parzialmente, come in T1 qui c'e' solo la guardia all'avvio e la terminazione del nodo non riconosciuto; RF03.1 parzialmente, ogni stazione e' un processo a se'. La conversazione vera e' in S2



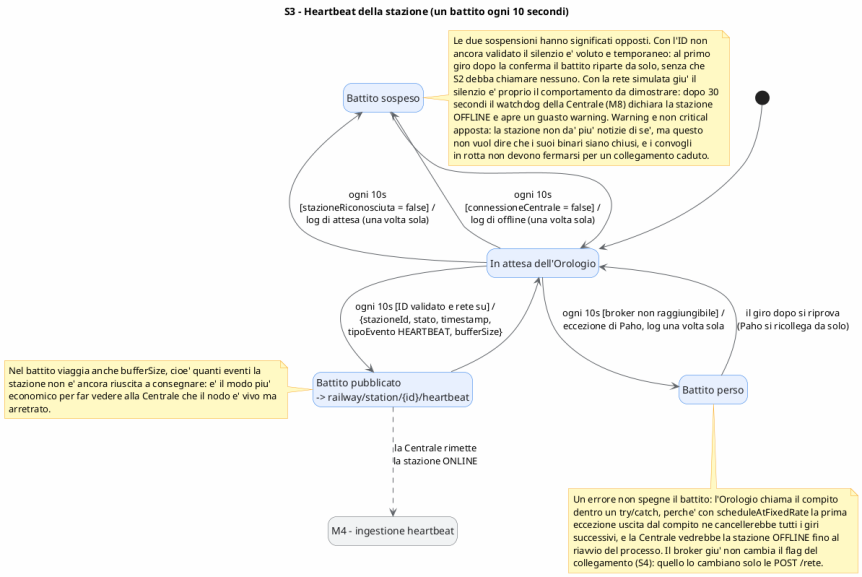
2.9.2.3 S2 validazione dell'id sul database centrale

- stesso pattern del treno, richiesta e risposta su MQTT: il nodo parte solo se la centrale conferma che l'ID esiste a database (StationDatabaseValidator)
- requisiti coperti: RF02.4.1 e RF02.4.2 completi per la parte della stazione, l'altra meta' della conversazione e' M2 lato centrale



2.9.2.4 S3 heartbeat verso la centrale

- il battito ogni 10 secondi tiene la stazione ONLINE lato centrale, e il suo silenzio e' proprio cio' che il watchdog legge come caduta del nodo (HeartbeatGenerator, fatto girare dall'Orologio)
- il battito non passa dal buffer: vale solo adesso, e rimandarne uno vecchio di minuti direbbe alla centrale che la stazione era viva in un momento in cui non lo era. Se non parte si perde, e dieci secondi dopo ne parte un altro. Legge lo stato e non muove nessun automa, quindi non passa nemmeno dal gestore degli eventi
- requisiti coperti: RF02.3.2 completo lato nodo, il battito parte ogni P_{bat} anche quando non c'e' niente da dichiarare, ed e' proprio questo che rende il silenzio di una stazione distinguibile dall'assenza di traffico; da solo non gestisce niente, e' il segnale il cui silenzio M8 legge come caduta (RF02.6.1)



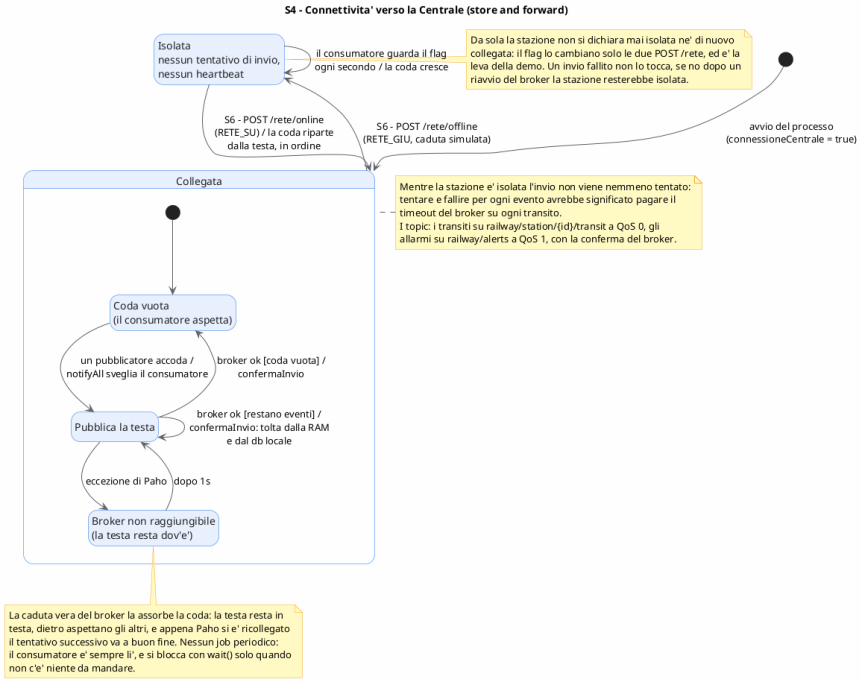
2.9.2.5 S4 connettivita' verso la centrale e store and forward

- lo stato del collegamento e lo svuotamento della coda: e' il caching locale con reinvio richiesto dal testo, con le due POST /rete per simulare la caduta durante la demo

- la coda e' uno schema produttore/consumatore: i produttori (i due pubblicatori, chiamati dal gestore degli eventi) accodano e tornano subito; il consumatore e' un thread solo che guarda la testa, la pubblica e solo se il broker l'ha presa la toglie. Non c'e' nessun job periodico che passa a svuotarla: riparte da sola appena si puo'

- il flag del collegamento lo cambiano soltanto le due POST /rete (AutomaConnessione). Una caduta vera del broker non passa da qui: la assorbe la coda, perche' la testa che non riesce a partire resta dov'e' e si riprova ogni secondo mentre Paho si ricollega

- requisiti coperti: RF02.1.1.1.1 e RF02.1.1.1.2 completi (nessun dato perduto durante l'interruzione e ordine di consegna preservato) e RF03.2 completo lato stazione, il nodo continua a rilevare e a registrare mentre la centrale non c'e': e' esattamente il caching locale dei dati con reinvio in massa alla ripresa della connessione chiesto dal testo

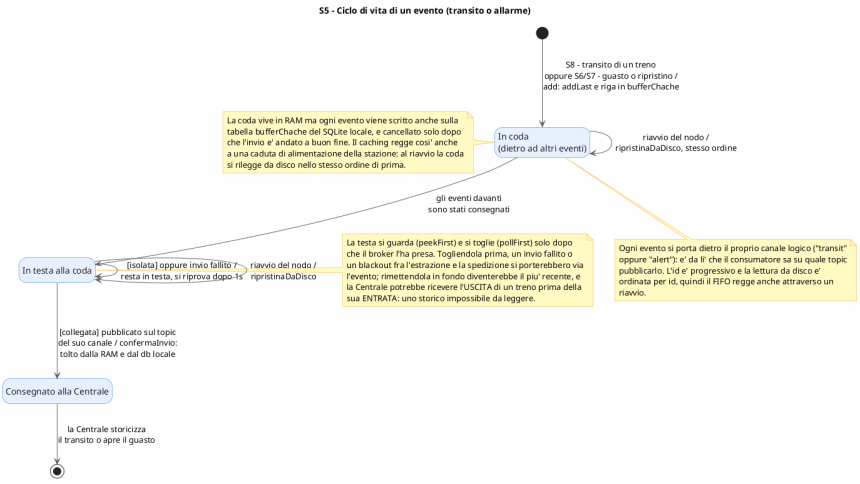


2.9.2.6 S5 ciclo di vita di un evento della stazione

- macchina di entita': la storia di un singolo transito o allarme dalla generazione alla consegna. L'evento esce dalla coda solo dopo che il

broker l'ha preso: fino a quel momento resta in testa, e quelli dietro aspettano

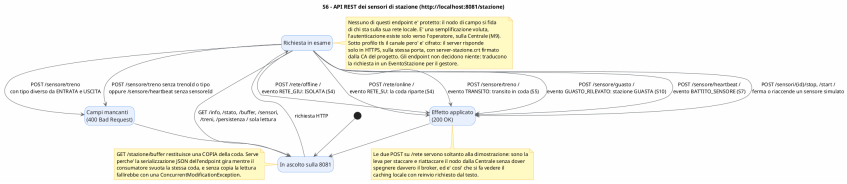
- requisiti coperti: RF02.1.1.1.1 e RF02.1.1.1.2 completi, e' il dettaglio sul singolo evento di quello che S4 fa sull'intera coda: nessun dato prodotto durante la caduta va perso, e il fatto che la testa si tolga solo a invio riuscito e' cio' che impedisce a una uscita di precedere la propria entrata



2.9.2.7 S6 api rest dei sensori di stazione

- i sensori di binario parlano REST, come deciso nel contratto di integrazione: qui entrano transiti, guasti, keepalive e le leve di simulazione della rete

- requisiti coperti: RF03.4 completo lato stazione (i sensori di binario dichiarano cio' che rilevano solo attraverso l'interfaccia pubblica del nodo), RF02.3.1 parzialmente (e' la porta da cui entrano i transiti, la storicizzazione e' in M5), RF02.3.3 parzialmente (qui arrivano i segnali di vita dei sensori) e RF02.1.1.2 parzialmente, perche' da qui si dichiara il guasto che rende la stazione non percorribile

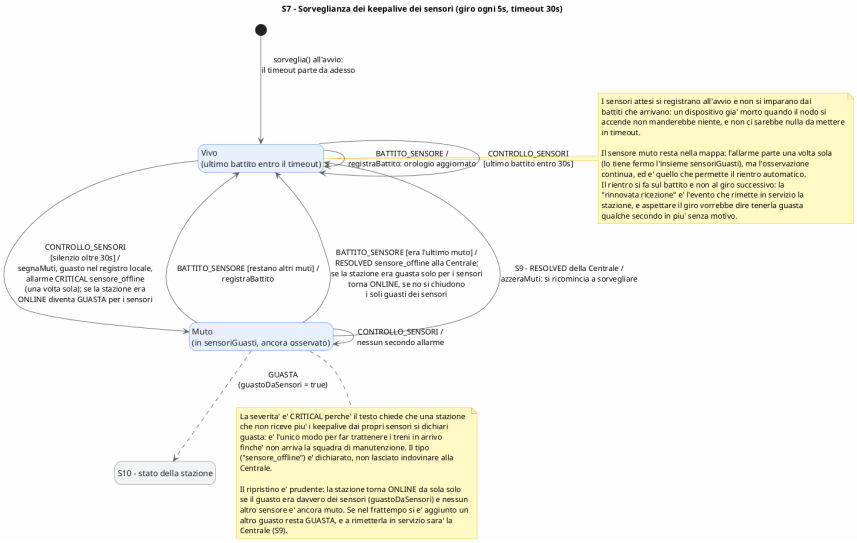


2.9.2.8 S7 sorveglianza dei sensori di binario

- il fail-stop dei sensori: ogni sensore deve farsi vivo con una POST /stazione/sensore/heartbeat entro il timeout (30s), e un giro di controllo ogni 5 secondi guarda chi ha smesso. Il giro lo scandisce GeneratoreEventiTempo (evento CONTROLLO$_{\text{SENSORI}}$), il battito lo porta StationIngestion (evento BATTITO$_{\text{SENSORE}}$), e le decisioni le prende GestoreEventi muovendo AutomaSensori e, quando serve, AutomaStatoStazione (S10)

- il sensore muto non viene dimenticato: resta sotto osservazione anche da guasto, ed e' il ritorno del suo battito, non il giro di controllo, a rimettere la stazione operativa

- requisiti coperti: RF02.3.3 completo, il sensore che smette di dichiararsi vivo oltre la soglia porta la stazione a GUASTA, che e' esattamente l'esito atteso dal requisito, e il rientro automatico e' il seguito

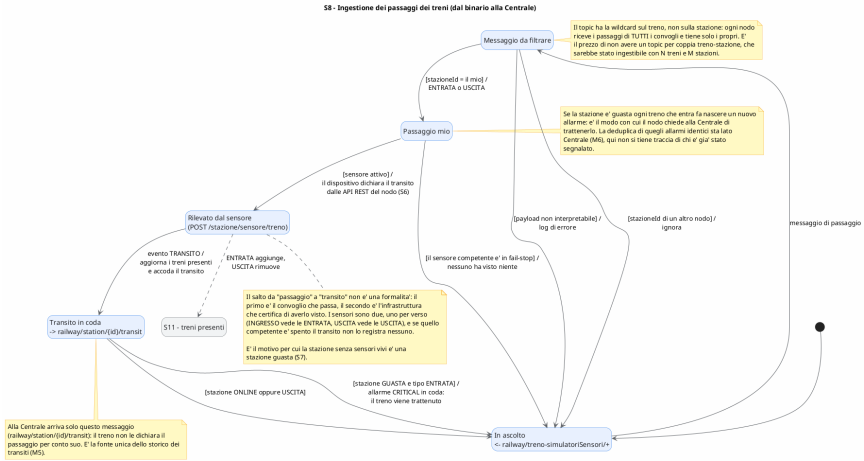


2.9.2.9 S8 ingestione dei passaggi dei treni

- consumo di railway/treno-simulatoriSensori/+ con filtro sul proprio ID, consegna al sensore di binario competente e transito ufficiale verso la centrale (StationGateway.riceviPassaggio, SimulatoreSensori.rilevaPassaggio, StationIngestion.rilevaTransitoTreno, GestoreEventi.suTransito)

- il convoglio annuncia il proprio passaggio ma non lo registra nessuno finche' un sensore non lo vede: fra il messaggio e il transito ci sono un dispositivo e una chiamata REST, ed e' li' che il fail-stop del sensore ha effetto

- requisiti coperti: RF02.3.1 parzialmente (produce il transito ufficiale che poi M5 storicizza), RF03.3 parzialmente perche' e' la rilevazione a terra, cioe' la fonte di verita' per la storia dei transiti, RF03.4 perche' il passaggio entra nel nodo solo dalla sua interfaccia pubblica, e RF02.1.1.2.3 parzialmente: se la stazione e' GUASTA e il convoglio entra lo stesso, pubblica l'alert che lo trattiene



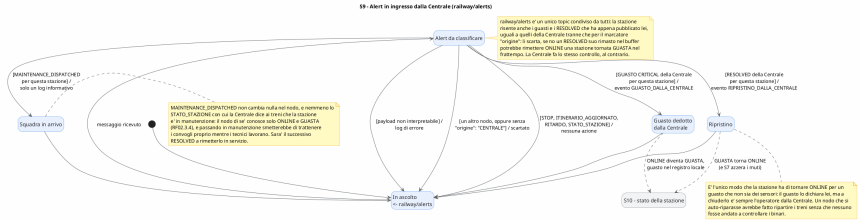
2.9.2.10 S9 alert in ingresso dalla centrale

- la stazione in ascolto su railway/alerts: la riguardano solo gli alert che parlano di lei e che portano la firma della centrale ("origine": "CENTRALE"), cioe' il ripristino, il guasto dedotto dalla centrale e l'invio della squadra; tutto il resto del topic condiviso, compresa l'eco dei suoi stessi allarmi, viene scartato

- il listener non decide come reagire: traduce l'alert in un evento per il gestore (RIPRISTINO$_{\text{DALLACENTRALE}}$ o GUASTO$_{\text{DALLACENTRALE}}$), che muove S10 e S7

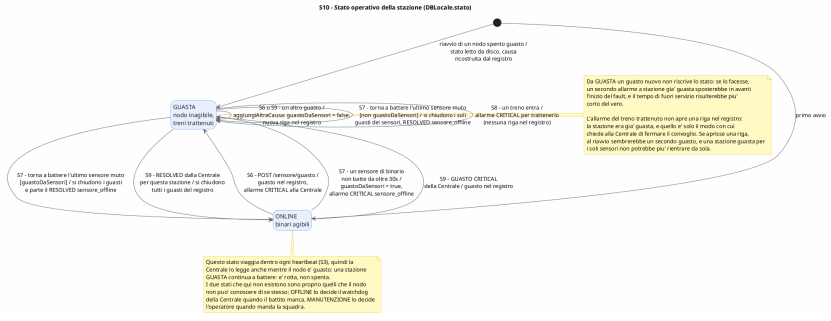
- requisiti coperti: RF02.1.1.2.2 parzialmente, la stazione da sola non si ripara: torna percorribile solo con il RESOLVED deciso dalla centrale in M10; RF02.1.2.2.1 lato campo, perche' e' da qui che il nodo

apprende di essere stato chiuso da un convoglio guasto sui suoi binari; RF01.4.1 lato campo, perche' e' qui che arriva l'invio della squadra di manutenzione: il nodo lo registra e basta, il suo stato locale resta GUASTA ed e' quello che continua a trattenere i treni per tutta la durata dell'intervento



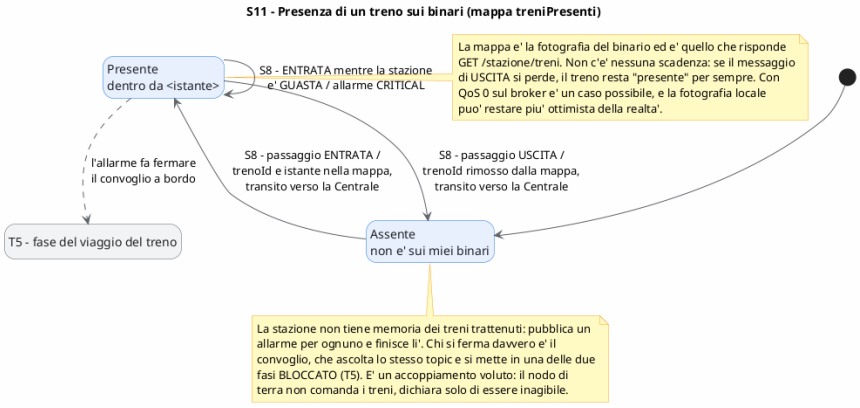
2.9.2.11 S10 stato operativo della stazione

- gli stati ONLINE e GUASTA come li vede il nodo: sono due e non quattro come nella cache della centrale (M12), perche' OFFLINE e MANUTENZIONE la stazione non puo' conoscerli di se stessa
- insieme allo stato l'automa tiene la causa del guasto (guastoDaSensori) e il registro dei guasti ancora aperti (la tabella $\text{Guasto}_{\text{LocaleMomentaneo}}$ del db locale). La causa decide chi puo' rimettere in servizio la stazione: se e' guasta soltanto per i sensori muti basta il ritorno dei loro battiti, per qualsiasi altro guasto serve il RESOLVED della centrale
- la scrive solo AutomaStatoStazione, e solo quando glielo chiede il gestore degli eventi
- requisiti coperti: RF02.3.4 completo, il nodo dichiara di se' soltanto ONLINE e GUASTA, mentre OFFLINE e MANUTENZIONE sono giudizi che si possono dare solo da fuori e li tiene la centrale (M12)



2.9.2.12 S11 presenza di un treno sui binari

- macchina di entita': la fotografia dei treni fermi sui binari, quella che risponde alla GET /stazione/treni
- requisiti coperti: RF02.3.1 completo lato nodo (quali convogli ci sono sui binari in questo momento, con l'ora di entrata e di uscita) e RF01.1.3 parzialmente, e' la fotografia che risponde alla GET /stazione/treni, tenuta in RAM e rispecchiata sulla tabella Treno del db locale



2.9.3 Del Server Centrale

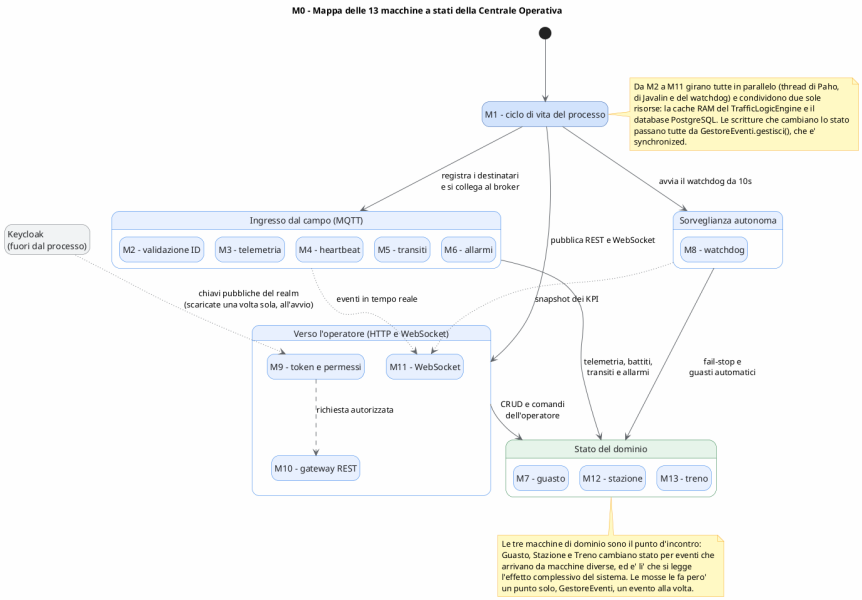
- nota: ci sono anche due macchine relative a stazione e treno sarebbero la proiezione lato della centrale operativa
- il componente e' stato astratto in 13 macchine, ognuna con la sua sezione; la mappa qui sotto dice chi avvia chi
- si leggono in tre gruppi: quelle che consumano i topic MQTT dal campo (M2-M6), quella che sorveglia la rete da sola (M8) e quelle che servono l'operatore (M9-M11). Le ultime tre (M7, M12, M13) sono lo stato del dominio, cioe' il punto dove si vede l'effetto di tutte le altre
- M8 e' l'unica macchina che non reagisce a un messaggio ma a un silenzio: e' quella che realizza RF02.6, le anomalie che i nodi di campo non sono in grado di segnalare da soli
- nel codice lo stesso schema del treno e della stazione: le tre macchine di dominio sono classi che offrono soltanto guardie e mosse (AutomaGuasto per M7, AutomaStazione per M12, AutomaTreno per

M13), a farle muovere e' un punto solo, GestoreEventi, e gli eventi glieli portano tre generatori: IngestionService (i messaggi MQTT del campo, M3-M6), GeneratoreEventiTempo (il tick del watchdog, M8) e RestApiGateway (i comandi dell'operatore, M10). Le anagrafiche (CRUD) e la validazione degli id (M2) non cambiano lo stato di nessuna macchina e restano fuori dal gestore

- ogni evento si gestisce in tre fasi: le guardie (si legge la cache e, quando serve, il database, e si decide tutto senza scrivere niente), gli assegnamenti (prima il database, in una transazione, poi le mosse degli automi sulla cache) e le azioni (WebSocket verso il browser e MQTT verso il campo). Il database va prima apposta: se il commit fallisce l'eccezione esce prima delle mosse, e la cache non mostra niente che sul database non ci sia. gestisci() e' synchronized, perche' gli eventi arrivano dal thread di Paho, da quelli di Javalin e dal watchdog

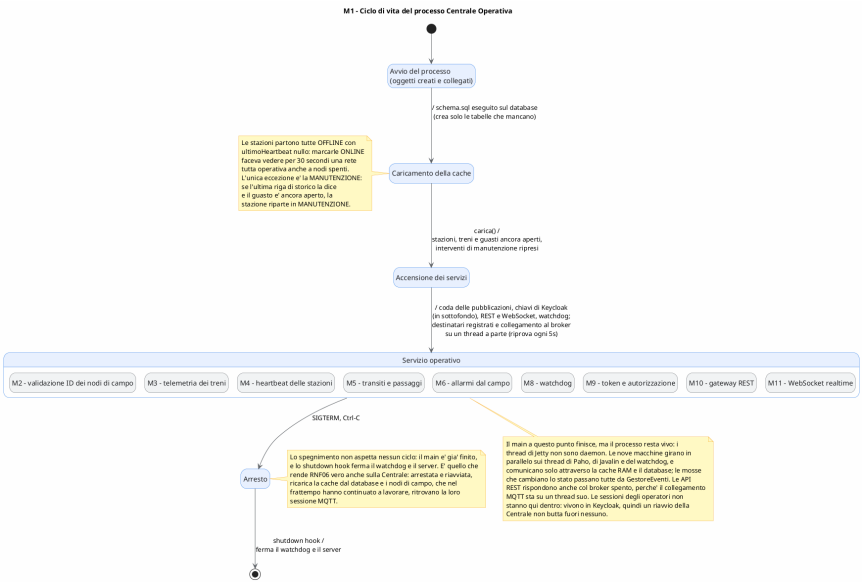
2.9.3.1 mappa delle macchine

- requisiti coperti: nessuno in proprio, e' l'indice delle altre tredici: i requisiti veri stanno scritti sopra le singole macchine



2.9.3.2 M1 ciclo di vita del processo della centrale

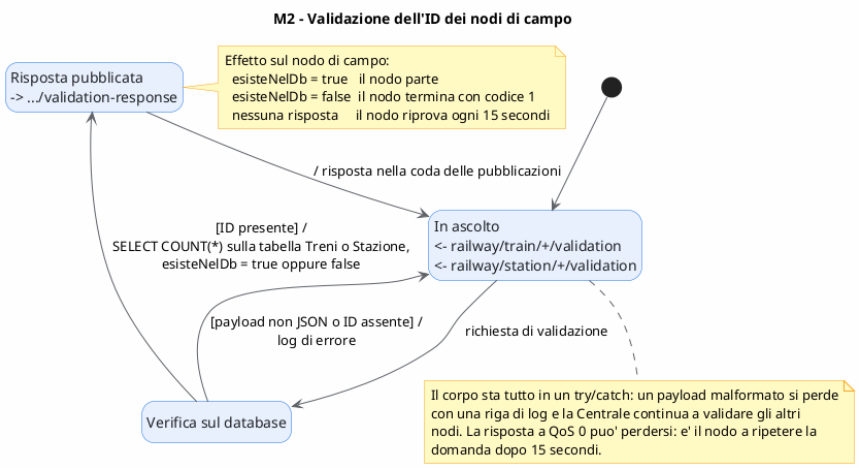
- e' la macchina radice: il main crea gli oggetti e li collega assegnandone i campi, riempie la cache dal database e poi accende in quest'ordine la coda delle pubblicazioni, la verifica dei token, il server HTTP e il watchdog; il collegamento al broker parte per ultimo, su un thread a parte (main)
- l'ordine conta: la cache si carica prima di aprire MQTT e le API REST, cosi' il primo messaggio che arriva trova gia' tutti i treni e le stazioni; e i destinatari dei topic si registrano prima di collegarsi, perche' con la sessione persistente gli alert arrivati a Centrale spenta vengono consegnati appena ci si collega
- requisiti coperti: nessuno in modo diretto, e' l'infrastruttura che fa partire tutte le altre: se non parte lei cadono tutti i requisiti, ma da sola non ne implementa nessuno



2.9.3.3 M2 validazione dell'id dei nodi di campo

- lato centrale della verifica dell'id gia' vista nei diagrammi del treno e della stazione (ExistIdForEdge / ExistIdForEdgeStazione)
- requisiti coperti: RF02.4.1 e RF02.4.2 completi lato centrale, e' il pezzo che va davvero a leggere l'anagrafica e chiude la conversazione aperta

da T2 e S2



2.9.3.4 M3 ingestione della telemetria dei treni

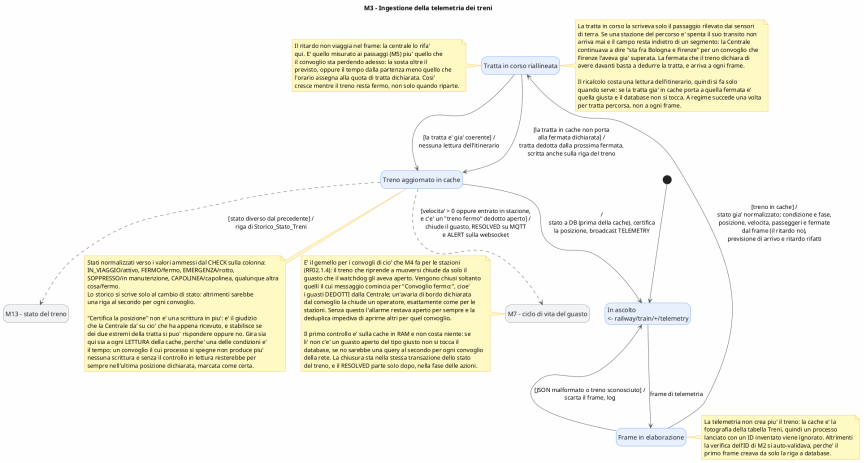
- consumo del topic railway/train/+/telemetry, aggiornamento della cache e storicizzazione solo al cambio di stato: IngestionService.onTelemetry traduce il frame nell'evento TELEMETRIA, GestoreEventi.suTelemetria decide e scrive, AutomaTreno.telemetria muove la copia in cache

- e' anche il punto in cui la centrale si accorge che un convoglio dato per fermo ha ripreso a muoversi, e chiude da sola il guasto che si era aperta

- dal frame si copiano in cache anche `condizione` e `faseViaggio` (T6 e T5): non vanno a database e non cambiano nessuna decisione della centrale, servono solo allo stato composto della web app

- a ogni frame si rifa' anche il ritardo, che nel frame non c'e': e' quello misurato ai passaggi (M5) piu' quello che il convoglio sta perdendo adesso, cioe' la sosta oltre il previsto oppure, in tratta, il tempo trascorso dalla partenza meno quello che l'orario assegna alla quota di tratta dichiarata (PrevisioneArrivi.aggiornaRitardo). Cosi' il ritardo di un treno trattenuto cresce finche' resta fermo, e al passaggio successivo la stima lascia il posto alla misura dello stesso tempo. Il convoglio che compare fermo nella prima stazione senza esserci entrato apre li' la sua sosta: e' la partenza della corsa, da cui esce senza ENTRATA. A corsa finita (capolinea, soppressione, convoglio sganciato dall'itinerario) il ritardo resta quello raggiunto

- requisiti coperti: RF02.2.2.1 e RF02.2.2.3 completi lato centrale (posizione e stato in cache), RF02.2.1.1 completo lato centrale (il ritardo del convoglio fermo dove non dovrebbe cresce a ogni frame), RF02.2.2.2 parzialmente (qui si rifanno la previsione di arrivo e la quota di ritardo in corso, la misura la fa M5 al passaggio), RF02.1.4 completo per i convogli (il guasto dedotto si richiude quando la causa rientra, gemello di quello che M4 fa per le stazioni), RF02.7 parzialmente perche' storicizza solo al cambio di stato e non a ogni frame, che e' la regola di registrazione chiesta dal requisito, RF01.1.1 e RF01.1.2 parzialmente (il dato sta qui, la spinta al browser e' M11), RF02.1.2.1 parzialmente (normalizza lo stato nei valori ammessi da M13)



2.9.3.5 M4 ingestione degli heartbeat delle stazioni

- consumo del topic railway/station/+/heartbeat e chiusura automatica dei guasti aperti dal watchdog quando il battito torna: IngestionService.onHeartbeat traduce il battito nell'evento HEARTBEAT, GestoreEventi.suHeartbeat decide e scrive, AutomaStazione muove la copia in cache

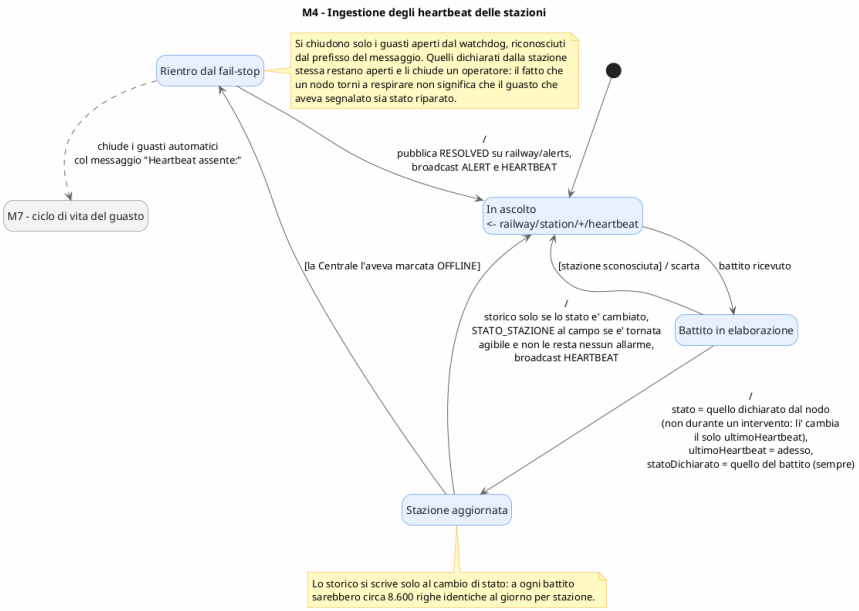
- il battito dice che il nodo e' vivo, non in che stato si trova: durante un intervento aggiorna il solo istante di ultimo contatto, perche' dal campo la stazione continua a dichiararsi GUASTA e il primo battito cancellerebbe la manutenzione (M12). E' la guardia AutomaStazione.accettaDalCampo

- lo stato scritto nel battito si annota sempre in `Stazione.statoDichiarato`,

anche quando la guardia lo rifiuta: e' lo stato di S10, che la web app mostra accanto a MANUTENZIONE

- quando il nodo si dichiara di nuovo sano e non gli resta aperto nessun allarme, il ritorno viene annunciato al campo con `STATO_STAZIONE`, l'unico messaggio che rimette in marcia i convogli trattenuti (T9)

- requisiti coperti: RF02.3.2 completo lato centrale e RF02.1.4 completo: il battito che torna riporta la stazione ONLINE e chiude da solo, senza intervento di un operatore, il guasto che il watchdog aveva dedotto dal silenzio



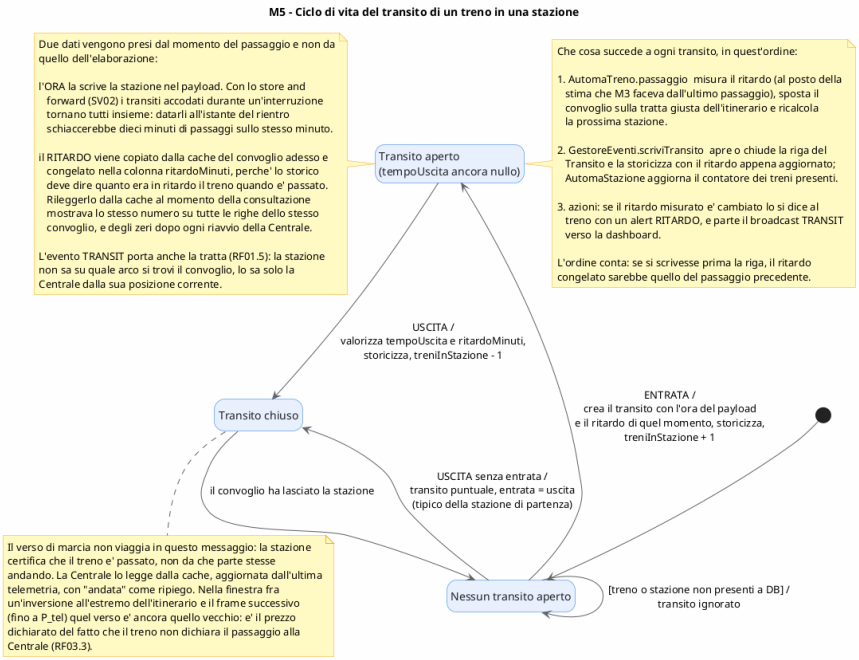
2.9.3.6 M5 ingestione dei transiti

- ciclo di vita del transito di un treno in una stazione: la notizia arriva da una strada sola, quella del sensore di terra (onTransit, evento PASSAGGIO), e viene usata per due cose diverse, aggiornare la posizione del convoglio sull'itinerario e scrivere il transito con il suo storico (GestoreEventi.suPassaggio)

- l'ora e il ritardo scritti nel transito sono quelli del passaggio e non quelli dell'elaborazione: sono due dati che invecchiano, e lo storico deve conservarli come erano

- il passaggio e' il momento in cui il ritardo si misura: all'ingresso si confronta l'ora vera con quella fissata all'uscita precedente e si apre la sosta, all'uscita si conta la sosta oltre il previsto (doppia agli estremi dell'itinerario) e si fissa l'orario per la fermata dopo. La misura prende il posto della quota che M3 aveva stimato dall'ultimo passaggio, quindi lo stesso tempo non si conta due volte, e solo lei arriva al treno con l'alert RITARDO

- requisiti coperti: RF02.3.1 e RF01.5 completi lato centrale (convoglio, stazione, tratta e orario di entrata e di uscita, a db e in cache), RF03.3 parzialmente perche' e' la via della rilevazione a terra, quella che fa la storia dei transiti, mentre l'altra via e' la telemetria (M3), RF02.7 e RF02.2.2.1 parzialmente (la posizione sulla tratta e la prossima stazione si aggiornano al passaggio)



2.9.3.7 M6 ingestione degli allarmi dal campo

- railway/alerts e' un topic condiviso, quindi la centrale deve filtrare i messaggi che ha pubblicato lei stessa (IngestionService.onAlert, marcatore "origine"); del campo interessano due eventi: il GUASTO che un nodo segnala ($\text{GUASTO}_{\text{SEGNALATO}}$) e il RESOLVED con cui un nodo

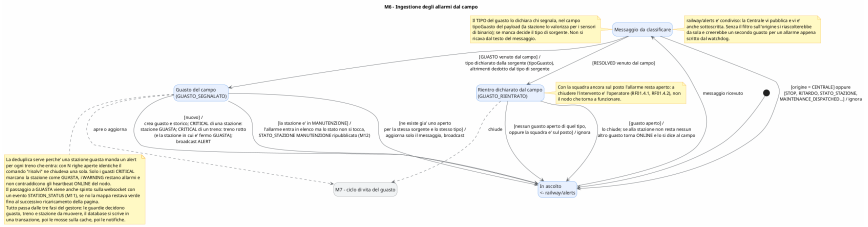
dichiara rientrato un guasto che aveva segnalato lui ($\text{GUASTO}_{\text{RIENTRATO}}$), oggi mandato dalla stazione quando i sensori tornano a battere

- il tipo del guasto lo dichiara la sorgente nel campo `tipoGuasto`

- una stazione guasta manda un alert per ogni treno che entra: la deduplica tiene un solo guasto aperto per sorgente e tipo, e la marcatura della sorgente passa dalla guardia di M12, che durante un intervento non la riporta a GUASTA

- se a guastarsi e' un convoglio fermo in una stazione, la stazione diventa non percorribile anche lei: il gestore apre su di lei un guasto dedotto e lo pubblica sul campo

- requisiti coperti: RF02.1.2.1 completo (il guasto dichiarato dal campo finisce a db), RF02.1.2.2.1 completo (il convoglio guasto in stazione chiude la stazione), RF02.1.1.2 completo per la parte di stazione (l'alert CRITICAL con sorgenteTipo STAZIONE la porta a GUASTA), RF01.4.1 per la parte di difesa dell'intervento in corso, RF01.2.2 parzialmente perche' e' qui che un guasto viene marcato come segnalato dal campo e non dedotto; la storia del guasto e' M7



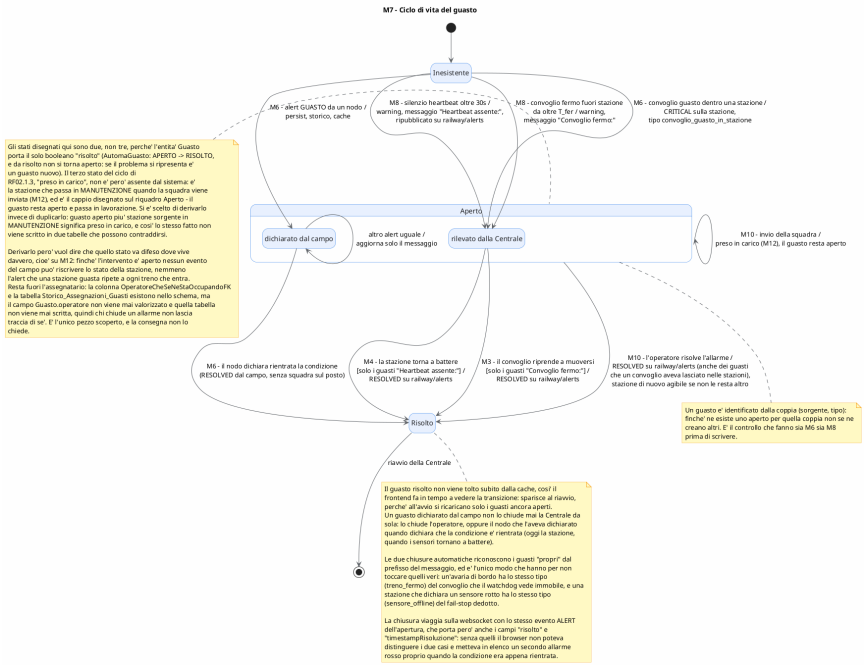
2.9.3.8 M7 ciclo di vita del guasto

- macchina di entita': la storia di un singolo guasto, su cui agiscono M3, M4, M6, M8 e M10

- le due chiusure automatiche (stazione che torna a battere, convoglio che riparte) riguardano solo i guasti dedotti dalla centrale, riconosciuti dal prefisso del messaggio: quelli dichiarati dal campo li chiude un operatore

- requisiti coperti: RF02.1.3 parzialmente (l'unicita' del guasto aperto per sorgente e la traccia su $\text{Storico}_{\text{Guasti}}$ ci sono, da qualunque parte arrivi il guasto; mancano lo stato intermedio "preso in carico" e l'assegnatario,

vedi la nota nel diagramma) e RF01.2.1 parzialmente, perche' e' il dato che poi la visualizzazione legge



2.9.3.9 M8 watchdog della centrale

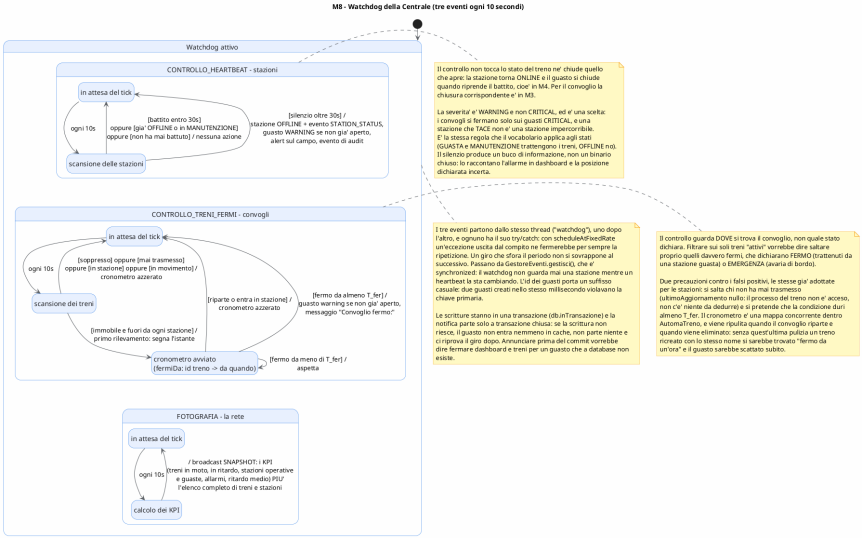
- i tre controlli ogni 10 secondi che rilevano cio' che i nodi di campo non possono segnalare da soli, cioe' la propria caduta. GeneratoreEventiTempo scandisce soltanto il tempo e manda tre eventi ($\text{CONTROLLO}_{\text{HEARTBEAT}}$, $\text{CONTROLLO}_{\text{TRENIFERMI}}$, FOTOGRAFIA); che cosa succede a ogni scatto lo decide GestoreEventi, con le stesse tre fasi di tutti gli altri eventi

- il secondo controllo tiene un cronometro per convoglio (in AutomaTreno): l'anomalia non e' "adesso e' fermo" ma "e' fermo da almeno T_{fer}", e senza memoria il watchdog poteva solo guardare l'istante presente

- il terzo controllo non manda soltanto i numeri: la fotografia comprende l'elenco di treni e stazioni, che e' cio' che le permette di riallineare davvero le schermate

- prima si scrive e si chiude la transazione, poi si annuncia: un guasto annunciato prima del commit puo' fermare i convogli per qualcosa che a database non esiste

- requisiti coperti: RF02.6.1, RF02.6.2 e RF02.6.3 completi: la stazione silenziosa oltre T_{sil}, il convoglio fermo fuori stazione oltre T_{fer} e la fotografia dello stato ogni P_{ria}, cioe' tutto cio' che i nodi di campo non sono in grado di segnalare da soli; RF01.1.5 e RF01.2.2 parzialmente. RF02.1.2.2.2 resta solo sfiorato: il convoglio fermo fra due stazioni fa aprire un allarme, ma la tratta non viene resa non percorribile



2.9.3.10 M9 ciclo di vita del token e autorizzazione delle api rest

- autenticazione OAuth2/OpenID Connect delegata a Keycloak e filtro dei permessi tecnico/amministratore su ogni chiamata: e' l'architettura base del PDF, non la variante 2

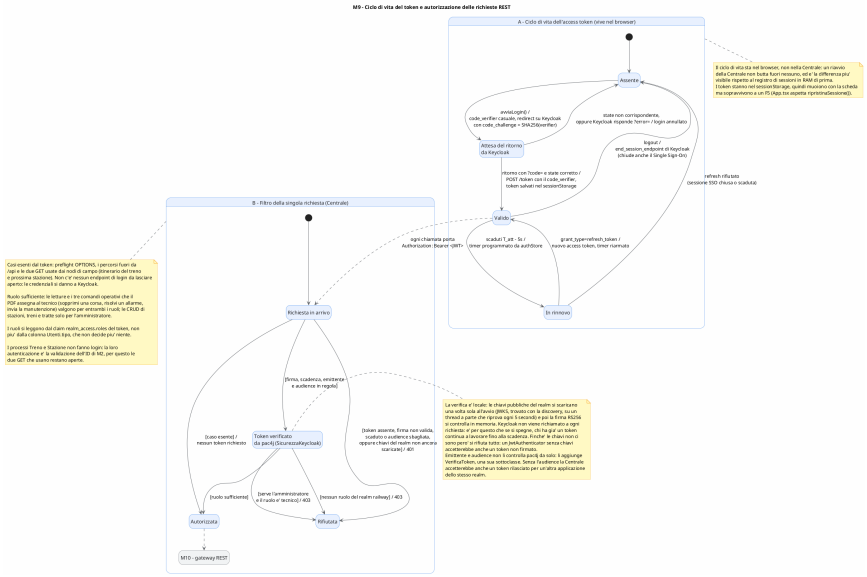
- la Centrale non tiene sessioni: quello che ha un ciclo di vita e' il token, e vive nel browser

- la macchina si legge in due meta': in alto il token visto dalla web app (assente, valido, scaduto, rinnovato), in basso la singola richiesta vista dalla Centrale

- la validita' del token (firma, scadenza, emittente, audience) la stabilisce pac4j dentro `SicurezzaKeycloak`, a cui il filtro chiede per ogni richi-

esta; il filtro decide solo chi puo' fare cosa. Il filtro e' un before-handler di Javalin su /api/*: se rifiuta scrive la risposta e ferma gli handler successivi, quindi l'endpoint non viene nemmeno chiamato

- requisiti coperti: RF04.1 completo e RF04.2 parziale (le eccezioni dichiarate nella specifica ci sono tutte — le richieste preliminari del browser e le due sole letture di cui hanno bisogno i nodi di campo — ma i tre comandi operativi restano aperti a entrambi i ruoli), e RF01.6 completo lato centrale: il controllo dei permessi e' fatto qui, dalla parte che possiede i dati, non nascondendo i comandi nell'interfaccia

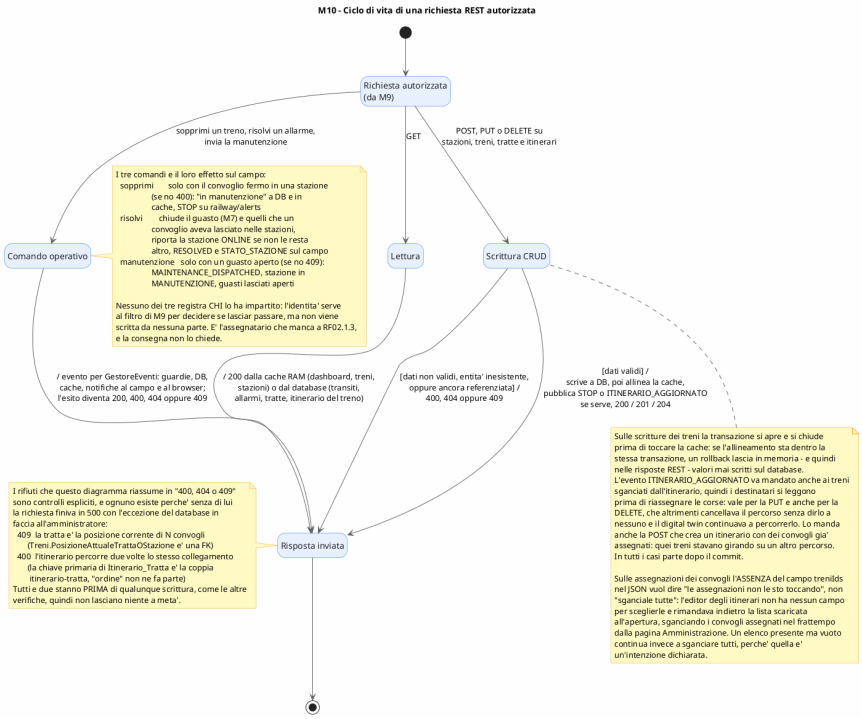


2.9.3.11 M10 gateway rest, crud e comandi operativi

- ciclo di vita di una richiesta gia' autorizzata: letture, crud, comandi e notifiche verso il campo (RestApiGateway)

- requisiti coperti: RF01.3 completo insieme a M9, RF01.5 completo per le letture dello storico, RF03.5 completo lato centrale perche' e' da qui che l'itinerario viene servito ai convogli, RF02.5.1 completo e RF02.5.2 parzialmente (da qui partono STOP e $\text{ITINERARIO}_{\text{AGGIORNATO}}$, ma il punto da cui si riprende lo decide il treno in T3), RF01.3.5 parzialmente perche' l'integrita' alle eliminazioni e' affidata ai vincoli del db. Dei tre comandi operativi i due che riguardano il guasto sono completi: RF01.4.1 apre l'intervento e lo stato MANUTENZIONE dura fino alla

chiusura (vedi la nota sotto e M12), RF01.4.2 lo chiude e sblocca chi era trattenuto (M7). E' completo anche RF01.4.3: la precondizione "convoglio fermo in stazione" e' la guardia del comando, valutata dal gestore degli eventi prima di scrivere



2.9.3.12 M11 canale websocket realtime

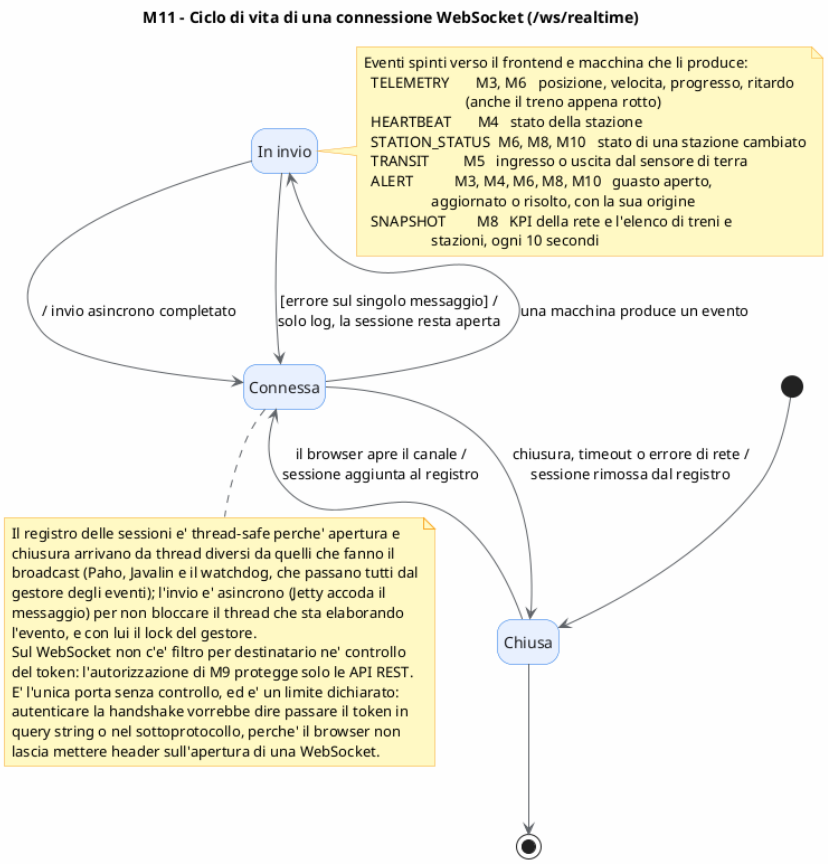
- punto di uscita comune di quasi tutte le altre macchine verso la dashboard (RealtimeWebSocket per le connessioni, NotificatoreRealtime per il vocabolario degli eventi): a chiamarlo e' il gestore degli eventi, nella fase delle azioni

- sotto profilo tls il browser apre `wss://localhost:8444/ws/realtime`, e la porta HTTPS della Centrale parla solo HTTP/1.1 (`ssl.http2 = false` in `ServerRest`): con l'HTTP/2 acceso Firefox prova a far passare la WebSocket dentro quella connessione, che Jetty 11 non accetta

- ogni evento TELEMETRY porta, oltre allo stato normalizzato, `condizione`, `faseViaggio` e `direzione` dalla cache, scritti sempre, a null quando non si sanno (anche nell'evento che parte senza telemetria, il treno rotto

annunciato dalla centrale). L'HEARTBEAT porta `statoDichiarato`; SNAPSHOT e REST li hanno gia', perche' serializzano gli stessi oggetti della cache

- requisiti coperti: RF01.1 e RF01.2 completi, e in particolare RF01.1.4: l'aggiornamento e' spinto nel momento in cui l'evento accade e non ottenuto da interrogazioni a orologio del browser; RF02.6.3 parzialmente, perche' e' anche il canale su cui viaggia la fotografia periodica



2.9.3.13 M12 stato runtime della stazione

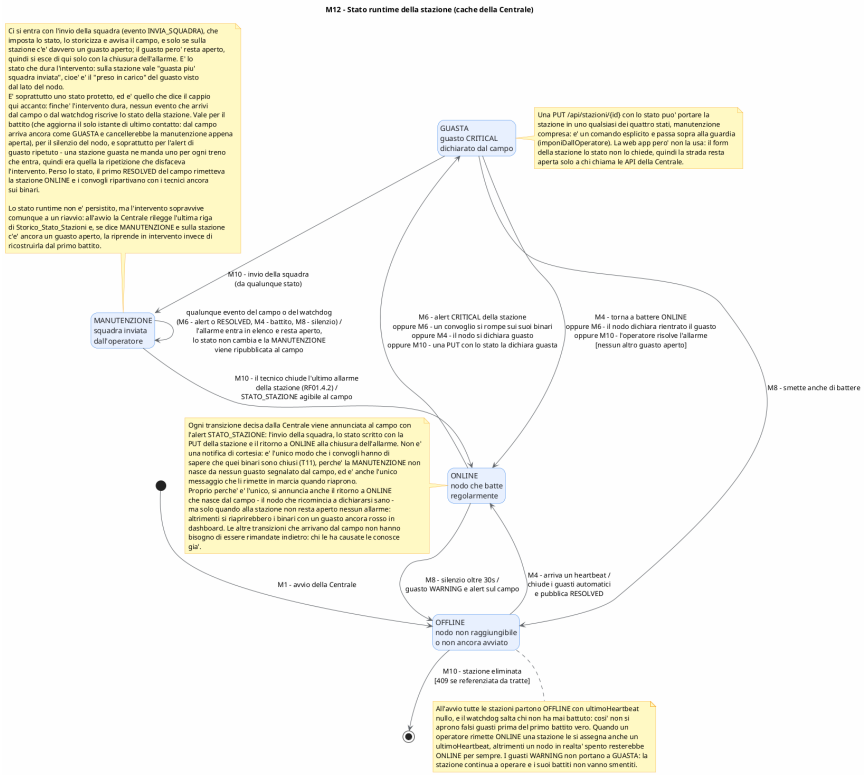
- gli stati ONLINE / OFFLINE / GUASTA / MANUTENZIONE come li vede la centrale: stanno solo in cache, sul db ne resta la traccia nello storico

- requisiti coperti: RF02.3.4 completo lato centrale per tutti e quattro

gli stati, compreso l'OFFLINE che il nodo non puo' conoscere di se'; RF01.4.1 per la MANUTENZIONE, che dura quanto l'intervento e non quanto la chiamata REST che la apre (vedi la nota). RF02.6.1 e RF01.1.3 parzialmente

- lo stato si scrive in una classe sola, `AutomaStazione`, ed e' li' che vive la regola dell'intervento, nella guardia `accettaDalCampo`: gli eventi del campo non escono dalla MANUTENZIONE, i comandi dell'operatore si'. Le mosse le chiama solo GestoreEventi

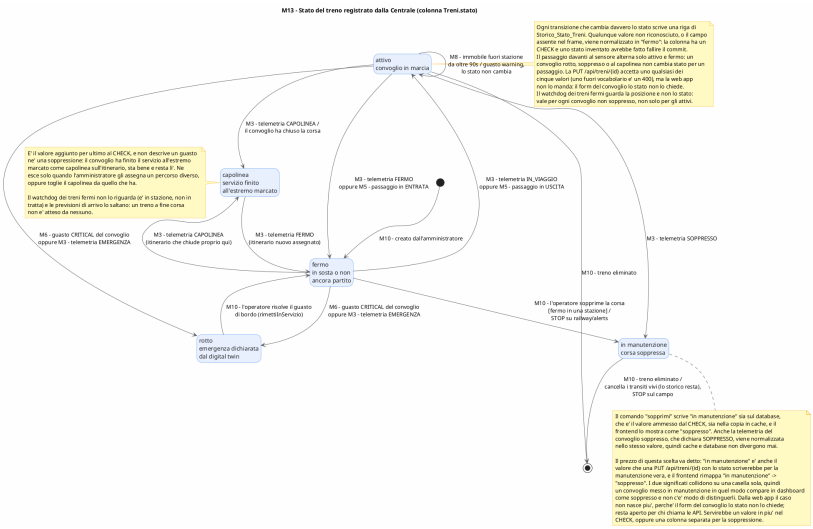
- MANUTENZIONE e' l'unico stato in cui questa macchina e S10 restano a lungo diverse, ed e' per questo che la web app accanto a MANUTENZIONE stampa anche `statoDichiarato` (vedi *Stato composto mostrato dall'interfaccia*)



2.9.3.14 M13 stato del treno

- proiezione sul database centrale della macchina a stati del convoglio: i cinque valori ammessi dal CHECK sulla colonna Treni.stato

- requisiti coperti: RF02.2.2.3 completo, e' la proiezione a db dello stato del convoglio, con il CHECK sulla colonna Treni.stato che tiene solo i cinque valori del vocabolario; RF02.7 parzialmente



2.9.4 Riepilogo requisiti - macchine a stati

È la lettura al contrario delle righe "requisiti coperti" che stanno sopra ogni diagramma del capitolo *Progettazione*: qui si parte dal requisito e si trovano le macchine, lì si parte dalla macchina e si trovano i requisiti. T = treno, S = stazione, M = centrale, e i numeri sono quelli delle macchine a stati, non quelli dei grafi G1. . . G8 di questa specifica.

Un requisito è in **coperto da** quando le macchine elencate lo soddisfano per intero, in **parzialmente** quando ci contribuiscono senza chiuderlo da sole. Le righe vuote in entrambe le colonne sono i requisiti che il sistema realizzato non soddisfa: sono lasciate apposta, perché una tabella che non le mostra racconta una copertura che non c'è.

Questa tabella però non va letta come lo **stato di implementazione** scritto accanto a ogni requisito: lì la domanda è "il sistema lo fa?", qui è "quale macchina lo mostra?", e le due risposte non coincidono sempre. Un paio di requisiti sono realizzati per intero senza che nessuna macchina li chiuda, perché a tenerli sono i vincoli del database (RF01.3.5) o il modo in cui il dato viene scritto (RF02.2.4). Dove i due giudizi divergono vale quello scritto accanto al requisito.

requisito	coperto da	parzialmente	nota
RF01.1.1	M11	T8, M3	T8 e M3 producono la posizione, M11 è il canale che la spinge alla mappa
RF01.1.2	M11	T8, M3	
RF01.1.3	M11, M12	S11	i presenti sui binari li sa il nodo (S11), gli stati la centrale (M12)
RF01.1.4	M11		il websocket: l'evento arriva quando accade, non a interrogazione
RF01.1.5	M11	M8	i numeri di sintesi li calcola il watchdog ogni 10 secondi
RF01.2.1	M11	M7	
RF01.2.2	M6, M8, M11		M6 è la porta del segnalato, M8 quella del dedotto; l'origine arriva all'elenco nel campo `origine`
RF01.3	M9, M10		M9 decide chi può, M10 esegue
RF01.3.5		M10	l'integrità alle eliminazioni la tengono i vincoli del db, non una macchina
RF01.4.1	M10, M12, S9		il comando parte da M10, il nodo lo riceve in S9 e M11 ne annuncia il cambio di stato; MANUTENZIONE dura fino alla chiusura del guasto
RF01.4.2	M10, M7, T9, S9		la chiusura sblocca chi era trattenuto e chiude in storico l'intervallo aperto da RF01.4.1; chi l'ha chiuso non viene registrato, ma non e' richiesto
RF01.4.3	M10, T6, T9		la precondizione "fermo in stazione" e' la guardia del comando
RF01.5	M10, M5		M5 scrive lo storico, M10 lo serve alla dashboard
RF01.6	M9		
RF02.1.1.1.1	S4, S5	S7	è il caching locale con reinvio in massa
RF02.1.1.1.2	S4, S5		la testa si toglie solo a invio riuscito: è ciò che conserva l'ordine
RF02.1.1.2.1	T5, T9, T11	S8	vale per il guasto e per la manutenzione: per chi viaggia sono la stessa cosa
RF02.1.1.2.2	T5, T9, T11	S9, M10	riparte quando la stazione torna agibile, non alla chiusura del singolo allarme
RF02.1.1.2.3	T11	S8, T3	lo stato di ogni fermata arriva anche dentro l'itinerario, quindi lo sa anche chi si accende dopo
RF02.1.2.1	M6, M7, M13	T10, M3	

Continued on next page

Continued from previous page

requisito	coperto da	parzialmente	nota
RF02.1.2.2.1	M6	M7, M12, S9, T9	M6 chiude la stazione in cui il convoglio si e' rotto, S9 e T9 la ricevono dal campo
RF02.1.2.2.2		M8	M8 apre solo un allarme sul convoglio, la tratta resta percorribile
RF02.1.3		M7, M6, M8	l'unicità per sorgente c'è ed è ciò che tiene leggibile l'elenco; mancano lo stato "preso in carico" e l'assegnatario
RF02.1.4	M4, M3	M8	il battito che torna chiude il guasto dedotto sulla stazione (M4), il convoglio che riprende a muoversi chiude il proprio (M3): in tutti e due i casi solo quelli con il prefisso della centrale
RF02.2.1.1	T5, M3	T9, T11	T5 tiene fermo il convoglio, M3 ne fa crescere il ritardo a ogni frame
RF02.2.2.1	T5, M3	T7, T8, M5	il cardine della specifica: sei macchine ci si appoggiano
RF02.2.2.2	M5	M3, T9	il ritardo lo misura la centrale al passaggio (M5) e lo comunica al convoglio (T9); M3 lo stima a ogni frame fra un passaggio e l'altro e rifa' la previsione di arrivo
RF02.2.2.3	T6, M13	M3	lo stato è tenuto distinto dalla posizione, come chiede il requisito
RF02.2.2.4	T8		valori simulati, non alimentano nessuna decisione
RF02.2.3	T7		
RF02.2.4		T5, T7	i passaggi spaiati nascono agli estremi; l'entrata definitiva senza uscita e' la fase $\text{FERMO}_{\text{CAPOLINEA}}$
RF02.3.1	S11, M5	S6, S8	
RF02.3.2	S3, M4	M8	S3 emette il battito, M8 ne misura il silenzio
RF02.3.3	S7	S6	
RF02.3.4	S10, M12		due stati sul nodo, quattro in centrale: è la asimmetria voluta dal requisito
RF02.4.1	T2, S2, M2	T1, S1	i due lati della stessa conversazione MQTT
RF02.4.2	T1, S1		il nodo non riconosciuto termina con codice 1, non resta acceso

Continued on next page

Continued from previous page

requisito	coperto da	parzialmente	nota
RF02.5.1	T9, T5, M10	T4, T6	la fase $\text{SENZA}_{\text{ITINERARIO}}$ e' il "resta fermo in attesa": senza di essa il convoglio escluso continuava a dichiararsi in marcia
RF02.5.2	T3	T4, T9, M10	il convoglio si riaggancia nella stazione in cui si trova; a meta' tratta il percorso nuovo aspetta l'arrivo
RF02.6.1	M8	S3, M12	
RF02.6.2	M8		
RF02.6.3	M8	M11	M8 scatta la fotografia, M11 la porta al browser
RF02.7		M5, M3, M13	si registrano i cambiamenti e non i campionamenti, ma solo per transiti, stati e guasti: assegnazioni degli operatori e itinerari percorsi non vengono mai scritti
RF03.1	T1, S1, M1		tre processi distinti, avviabili e arrestabili uno per volta
RF03.2	S4	S5, S7	
RF03.3	M5	T8, S8	le due vie sono la telemetria (T8) e la rilevazione a terra (S8, M5), e restano distinte
RF03.4	T10, S6		i sensori entrano solo dalle API REST del nodo
RF03.5	M10	T3	
RF04.1	M9		
RF04.2		M9, M10	il controllo è sul lato che possiede i dati, non sull'interfaccia; sui tre comandi operativi il ruolo però non viene distinto
RF04.3			non è una macchina: è il profilo `%tls` sui canali, vedi la sezione *Topic*

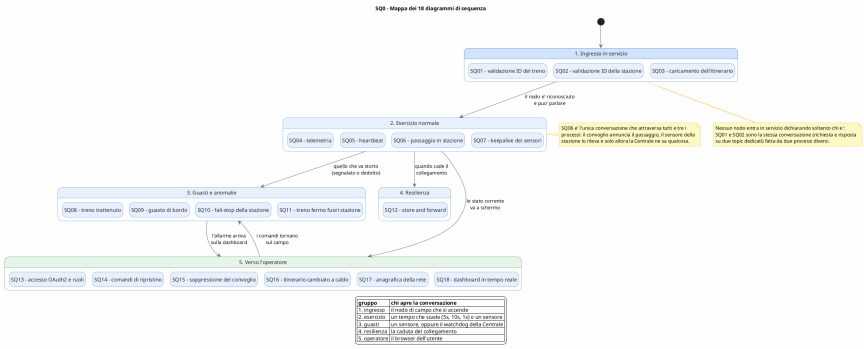
2.10 Diagrammi di sequenza

Note generali:

`RailwayRepository`, non l'ho disegnato come partecipante a se' perche' avrebbe aggiunto una corsia a quindici figure senza dire niente di nuovo; chi lo chiama e per che cosa si vede in `C5`

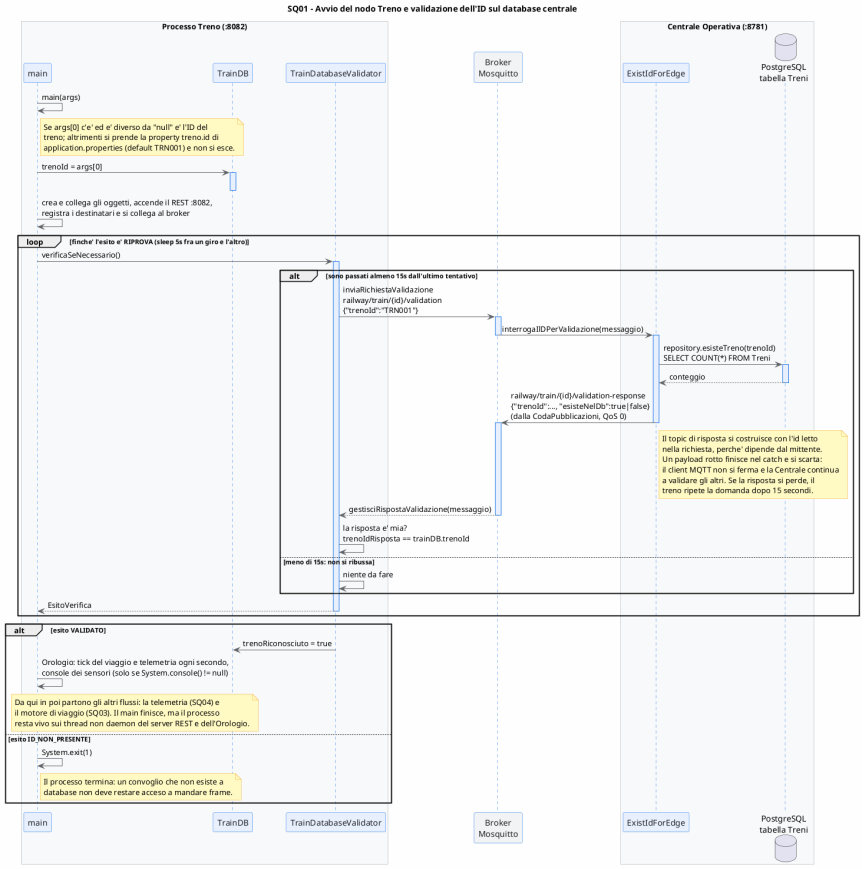
2.10.1 mappa dei diagrammi

- non e' una conversazione: e' l'indice delle altre diciotto, raggruppate per il momento in cui si presentano nella vita del sistema
- i cinque gruppi rispondono a una domanda sola: chi apre la conversazione. Un tempo che scade, un sensore, il watchdog della centrale oppure il browser dell'utente
- requisiti coperti: nessuno in proprio, i requisiti veri stanno sopra i singoli diagrammi



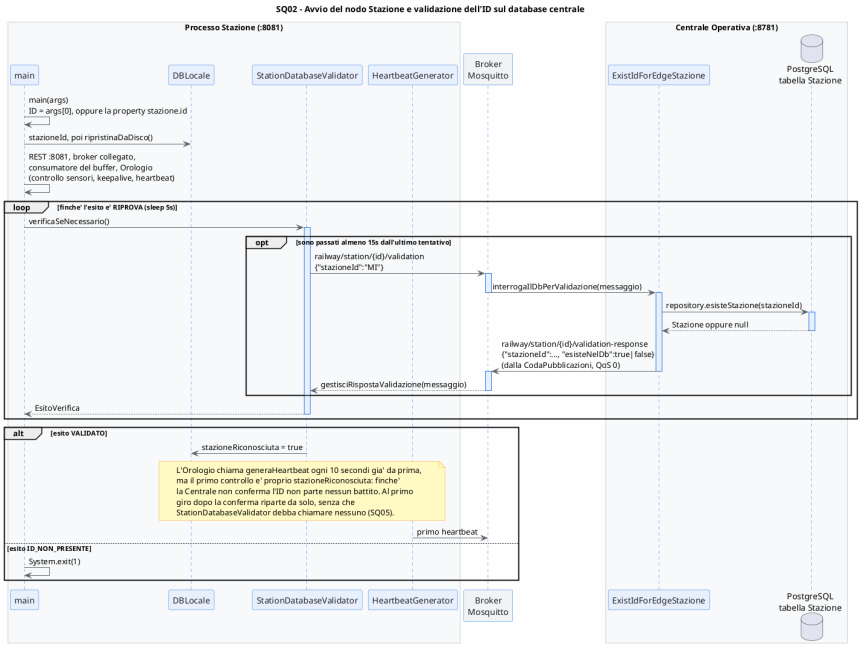
2.10.2 SQ01 avvio del nodo treno e validazione dell'id

- richiesta e risposta su due topic dedicati: il processo prosegue solo se la centrale conferma che l'id esiste a database, altrimenti termina con codice 1
- i tre esiti sono VALIDATO, $\text{ID}_{\text{NONPRESENTE}}$ (uscita) e RIPROVA (nuovo tentativo dopo 5 secondi, con una richiesta MQTT nuova al massimo ogni 15)
- il topic di risposta lo costruisce la centrale a partire dall'id che trova nella richiesta, perche' dipende da chi ha chiesto: la centrale ascolta tutte le richieste con un solo filtro (railway/train/+/validation) e risponde a ciascun treno sul suo topic
- requisiti coperti: RF02.4.1 e RF02.4.2 completi per la parte del treno, RF02.4.3 per la portata della dipendenza; e' la conversazione che le macchine T1, T2 e M2 raccontano da un lato solo



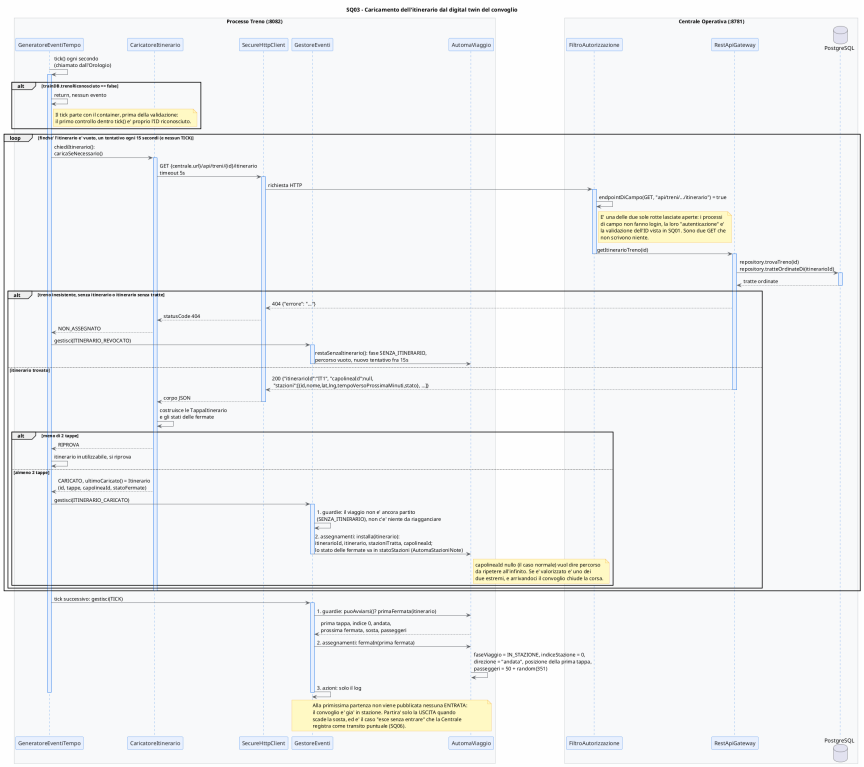
2.10.3 SQ02 avvio del nodo stazione e validazione dell'id

- stessa conversazione di SQ01 fatta dall'altro tipo di nodo: due coppie di topic separate e due classi diverse lato centrale, perche' le tabelle interrogate sono due
- il dettaglio che vale la pena guardare e' l'aggancio con l'heartbeat: l'Orologio lo fa scattare ogni 10 secondi gia' da prima della validazione, ma il battito resta sospeso finche' non arriva la conferma
- requisiti coperti: RF02.4.1 e RF02.4.2 completi per la parte della stazione, l'altra meta' e' M2 lato centrale



2.10.4 SQ03 caricamento dell'itinerario del convoglio

- e' l'unica chiamata REST che un nodo di campo fa verso la centrale: il digital twin scarica la propria mappa di viaggio e senza quella non si muove
- la rotta e' lasciata aperta dal filtro di autorizzazione perche' i processi di campo non fanno login: la loro autenticazione e' la validazione dell'id di SQ01
- in caso di errore (centrale spenta, treno senza itinerario, itinerario con meno di due stazioni) si riprova ogni 15 secondi, non si esce
- la GET la fa il generatore del tick, fuori dal gestore degli eventi; la risposta diventa un evento, e solo il gestore scrive il percorso nel `TrainDB`
- requisiti coperti: RF03.5 completo, RF03.1 parzialmente perche' e' il punto in cui l'autonomia del nodo si appoggia comunque alla centrale



2.10.5 SQ04 telemetria del convoglio fino alla mappa

- il percorso completo di un frame: Orologio da 1 secondo, coda delle pubblicazioni, topic della telemetria, gestore degli eventi della centrale, cache in RAM, storico solo al cambio di stato, websocket, store del frontend

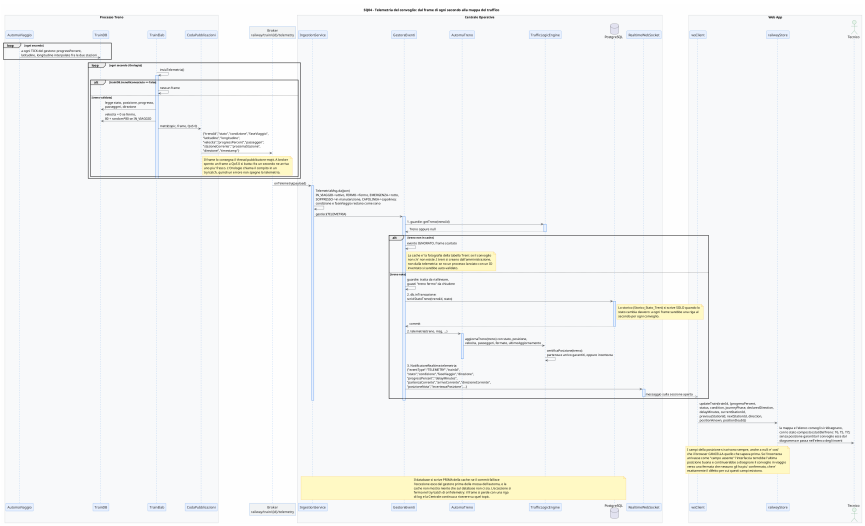
- sono quattro salti e tre rappresentazioni diverse dello stesso convoglio (TrainDB, cache della centrale, riga di tabella, oggetto Train del frontend): e' la ragione di tutte le normalizzazioni che si vedono nel codice

- in centrale il frame e' un evento TELEMETRIA come gli altri: IngestionService lo traduce, GestoreEventi decide nella fase delle guardie, scrive il database in una transazione e poi muove AutomaTreno sulla cache, e solo alla fine avvisa il browser. Se la transazione fallisce l'eccezione esce prima della mossa, e la cache non mostra niente che sul database non ci sia

- quello che arriva alla mappa non e' la posizione che il convoglio dichiara

ma quella che la centrale **garantisce**: fra le due c'e' il passo `certificaPosizione`, che incrocia la telemetria con l'ultimo passaggio rilevato a terra e, quando i conti non tornano, manda al browser un'incertezza dichiarata al posto di una coppia di stazioni inventata

- requisiti coperti: RF01.1.1 e RF01.1.2 completi, RF02.2.2.1 e RF02.2.2.3 per posizione e stato dichiarato; il canale che spinge il dato a schermo e' M11



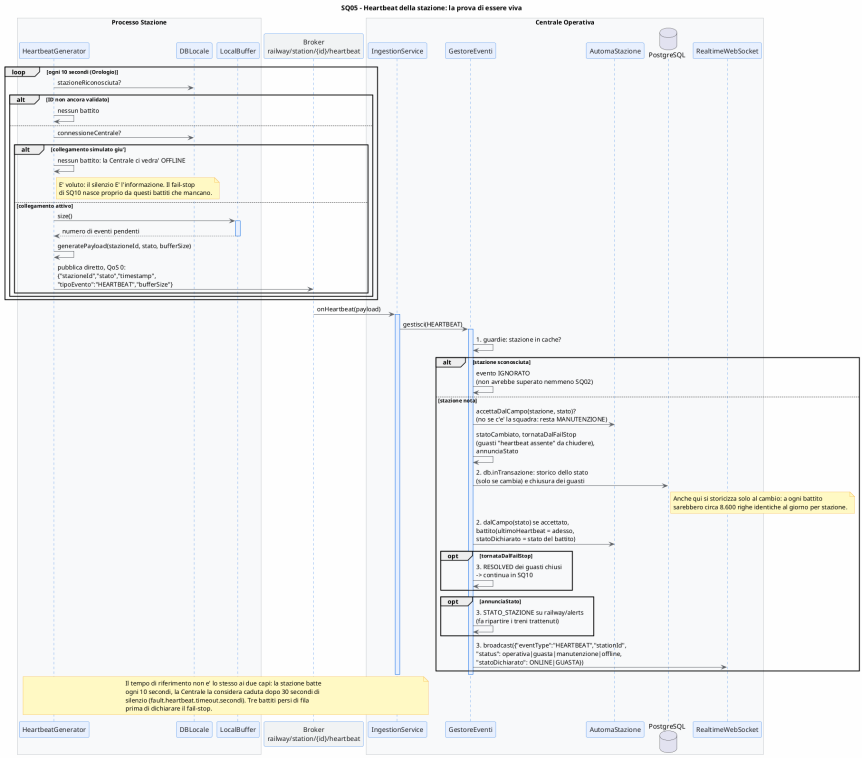
2.10.6 SQ05 heartbeat della stazione

- il battito parte solo se la stazione e' stata riconosciuta e se il collegamento risulta attivo: due controlli in cascata dentro lo stesso metodo, che l'Orologio chiama ogni 10 secondi. Non passa dal buffer: un battito vecchio non dice niente di vero

- in centrale il battito e' un evento HEARTBEAT per il gestore degli eventi, che decide se accettare lo stato dichiarato (non con la squadra sul posto), chiude i guasti del fail-stop se la stazione torna dal silenzio e annuncia al campo lo stato solo quando la stazione e' di nuovo agibile e pulita

- i due tempi non coincidono apposta, la stazione batte ogni 10 secondi e la centrale la considera caduta dopo 30: tre battiti persi di fila prima del fail-stop

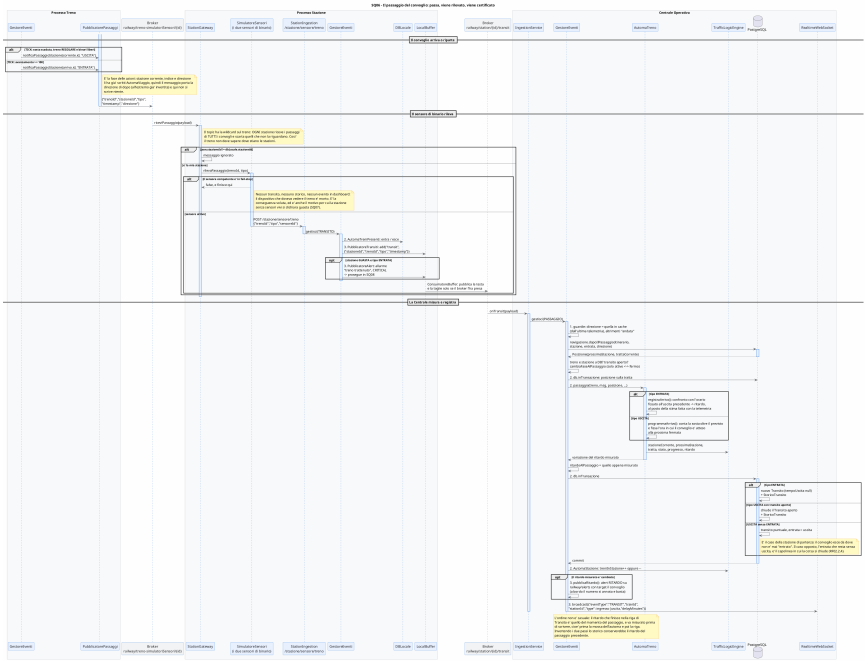
- il silenzio non e' un errore da gestire, e' l'informazione: e' quello che fa scattare SQ10
- requisiti coperti: RF02.3.2 completo, RF02.3.1 parzialmente per le informazioni che il nodo rende di se'



2.10.7 SQ06 passaggio in stazione, dal binario alla centrale

- e' il diagramma piu' importante del capitolo: fa vedere la catena con cui un fatto fisico diventa un dato, e i tre salti di responsabilita' che la compongono
- il convoglio **passa** (MQTT verso i sensori), il sensore di binario **rileva** (REST verso il proprio nodo), la stazione **certifica** alla centrale (MQTT); nessuno dei tre fa il lavoro dell'altro
- in centrale la notizia e' un evento PASSAGGIO per il gestore degli eventi, e serve a due cose in quest'ordine: prima la mossa di AutomaTreno misura il ritardo e sposta il convoglio sull'itinerario, poi il gestore scrive la riga del transito con quel ritardo gia' aggiornato

- il treno non dichiara il passaggio alla centrale: un secondo testimone dello stesso fatto renderebbe innocuo il fail-stop del sensore
- requisiti coperti: RF03.3 parzialmente (qui c'e' la via della rilevazione a terra, l'altra e' la telemetria di SQ04), RF03.4 perche' il passaggio entra nel nodo solo dalle sue API pubbliche, RF01.5 parzialmente perche' e' qui che nasce lo storico consultabile, RF02.2.4 per i passaggi spaiati degli estremi

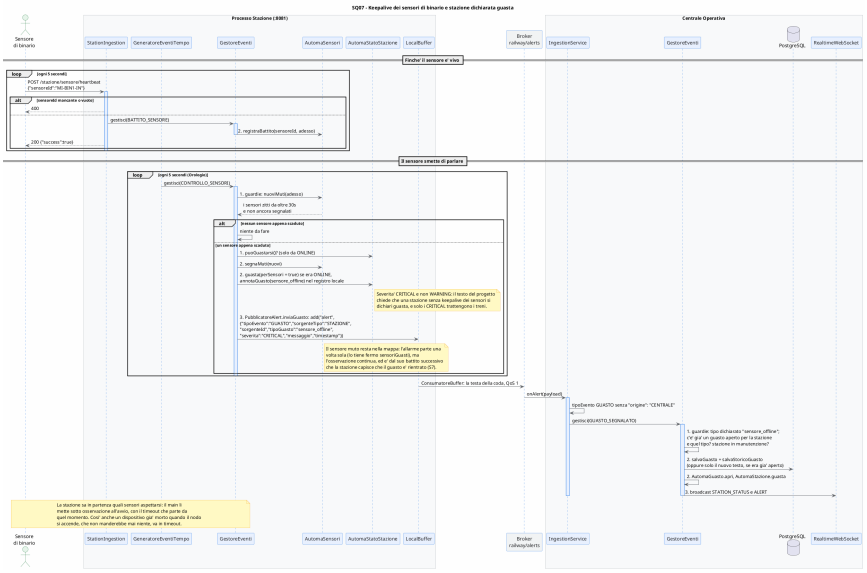


2.10.8 SQ07 keepalive dei sensori di binario

- i sensori parlano con il nodo solo attraverso le sue API REST: non toccano mai il modello interno, ed e' una scelta esplicita
- la stazione sa in partenza quali sensori aspettarsi (li mette sotto osservazione il main, con il timeout che parte dall'avvio): un sensore gia' morto quando il nodo si accende risulta guasto lo stesso, perche' non si aspetta il suo primo battito per cominciare a contare
- la severita' e' CRITICAL e non warning, perche' e' quello che porta la stazione in stato guasta e quindi trattiene i convogli; il tipo del guasto ($\text{sensore}_{\text{offline}}$) lo dichiara la stazione, la centrale non lo deve indovinare

- lato stazione il battito e il giro di controllo sono due eventi per il gestore ($\text{BATTITO}_{\text{SENSORE}}$ e $\text{CONTROLLO}_{\text{SENSORI}}$); lato centrale l'allarme e' un evento $\text{GUASTO}_{\text{SEGNALATO}}$

- requisiti coperti: RF02.3.3 completo, RF03.4 completo per il contratto dei sensori



2.10.9 SQ08 convoglio trattenuto da una stazione guasta

- due controlli in due momenti diversi: l'alert blocca subito il convoglio se la stazione guasta e' la corrente o la prossima, altrimenti la stazione resta annotata fra quelle guaste e sara' il controllo alla ripartenza a trattenerlo

- quel secondo controllo e' anche quello che copre chi era gia' fermo in quella stazione quando il guasto e' arrivato

- mentre e' bloccato a meta' tratta il cronometro della tratta viene congelato, cosi' la percentuale di avanzamento non cresce; il ritardo non lo conta il convoglio: la Centrale lo vede crescere a ogni frame di telemetria (M3), perche' la sosta si allunga o l'avanzamento resta fermo mentre l'orario corre, e lo misura al passaggio successivo (M5)

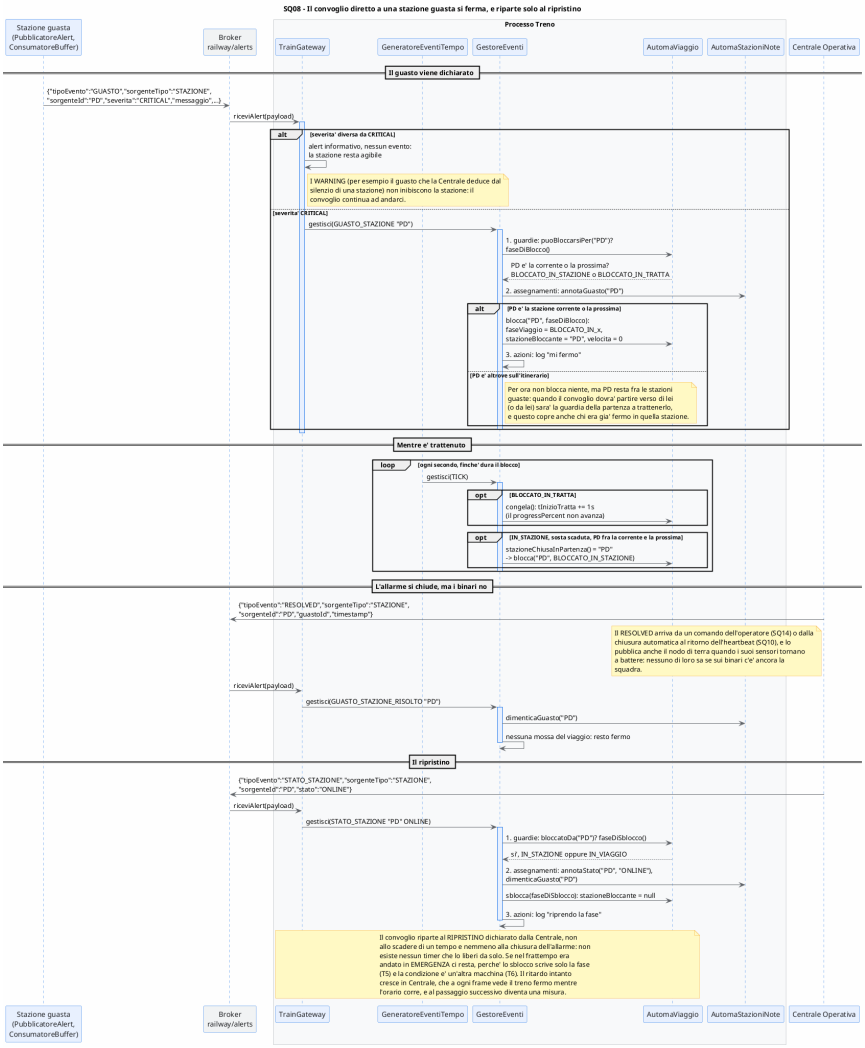
- riparte al ripristino e non allo scadere di un tempo, e non basta che l'allarme si chiuda: il permesso di ripartire e' solo lo `STATO_STAZIONE`

ONLINE della Centrale, perche' un guasto chiuso non vuol dire che sui binari non ci sia ancora la squadra

- lo stesso blocco lo provoca anche la manutenzione, che pero' non arriva da qui ma dalla Centrale con l'alert `STATO_STAZIONE` (SQ14): il convoglio non distingue i due casi, per lui quei binari non si attraversano e basta

- lato treno ogni alert diventa un evento per il gestore, che prima valuta le guardie (la stazione e' la corrente o la prossima? il treno e' in una fase che si puo' bloccare?) e solo dopo scrive

- requisiti coperti: RF02.1.1.2.1, RF02.1.1.2.2 e RF02.1.1.2.3 completi, RF02.2.1.1 perche' il convoglio trattenuto resta fermo e la Centrale ne fa crescere il ritardo finche' non riparte

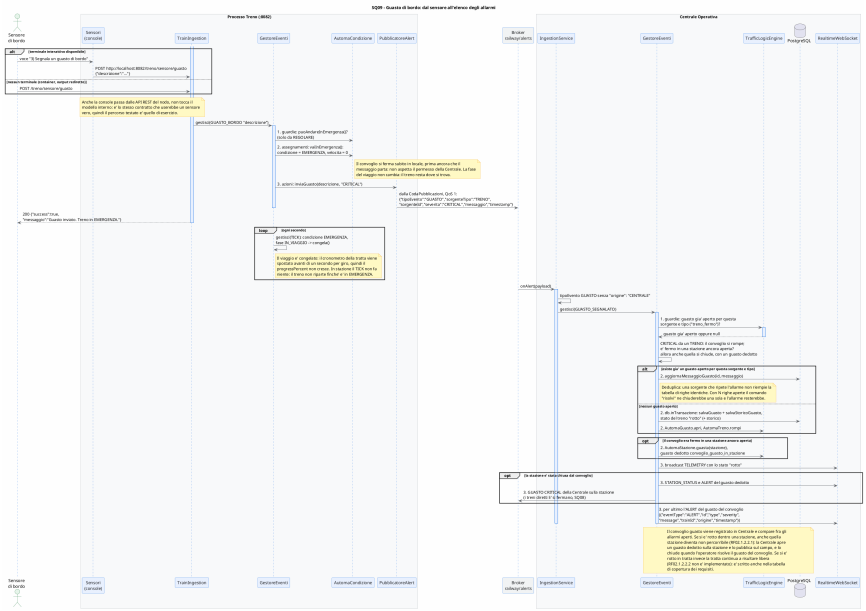


2.10.10 SQ09 guasto rilevato dai sensori di bordo

- il convoglio si ferma in locale prima ancora che il messaggio parta: non aspetta il permesso della centrale, ed e' il senso dell'autonomia del nodo
- lato centrale la parte interessante e' la deduplica, perche' una sorgente che ripete l'allarme riempirebbe la tabella di righe identiche e il comando risolvi ne chiuderebbe una sola
- e' anche il diagramma dove si vedono le conseguenze sull'infrastruttura:

se il convoglio si rompe dentro una stazione, la centrale chiude anche quella stazione con un guasto dedotto; se si rompe in tratta, la tratta continua a risultare percorribile, ed e' il limite dichiarato

- requisiti coperti: RF02.1.2.1 completo, RF02.1.2.2.1 completo, RF02.1.3 parzialmente per l'unicita' per sorgente, RF01.2.1 per l'elenco degli allarmi aperti; RF02.1.2.2.2 non e' coperto ed e' segnato nella nota



2.10.11 SQ10 fail-stop della stazione e chiusura automatica

- e' il giro completo del cardine per assenza: la centrale non riceve piu' niente, deduce il guasto, lo scrive e lo rimanda sul campo

- il guasto dedotto dal silenzio e' **warning e non critical**, quindi i convogli NON si fermano: e' la differenza fra "questa stazione e' chiusa" e "di questa stazione non so piu' niente", e sono due fatti diversi che meritano due reazioni diverse. Quello che il silenzio produce e' un buco di informazione — l'allarme in dashboard e i convogli che ne dipendono dichiarati a posizione incerta — non un binario inibito

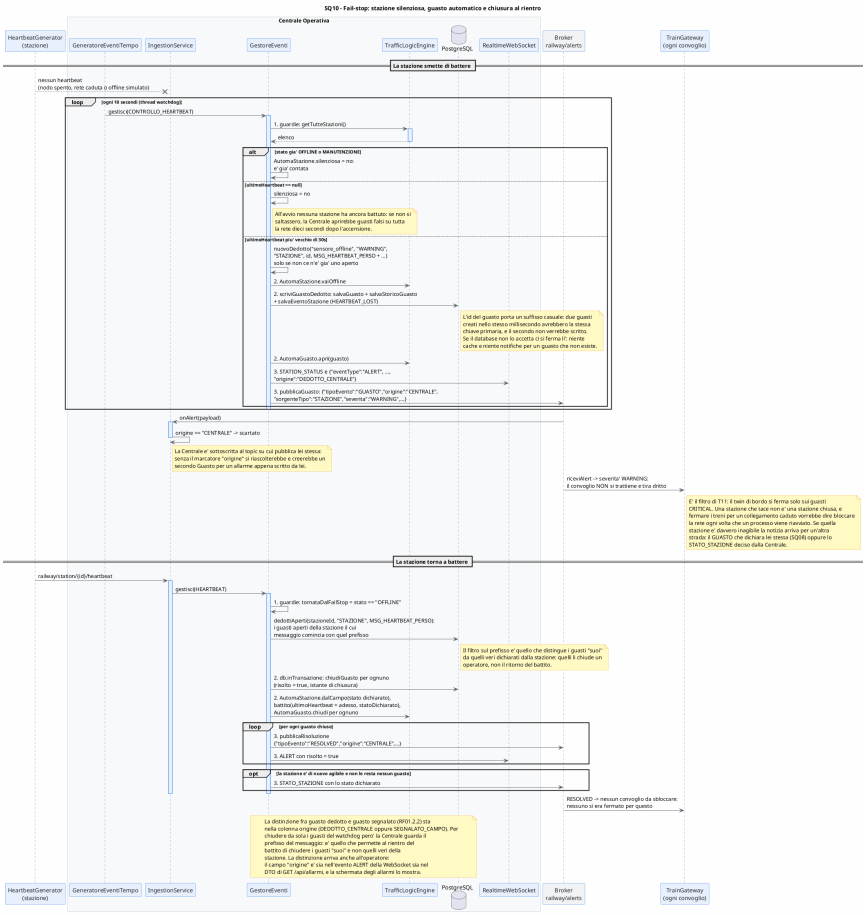
- due dettagli reggono tutto il meccanismo: il marcatore sull'origine, che impedisce alla centrale di riascoltarsi e creare un secondo guasto per un

allarme scritto da lei, e il prefisso nel messaggio, che le permette di riconoscere i guasti suoi e chiuderli da sola senza toccare quelli dichiarati dalla stazione

- il silenzio lo scandisce il tempo: GeneratoreEventiTempo manda al gestore un $\text{CONTROLLO}_{\text{HEARTBEAT}}$ ogni 10 secondi, e il ritorno del battito e' un evento HEARTBEAT come gli altri. Tutte e due le strade passano dalle tre fasi del gestore

- le stazioni che non hanno mai battuto vengono saltate, se no dieci secondi dopo l'accensione la centrale aprirebbe guasti falsi su tutta la rete

- requisiti coperti: RF02.6.1 completo, RF02.1.4 completo per la chiusura automatica quando la causa rientra, RF01.2.2 per la distinzione fra guasto segnalato e dedotto, che arriva all'operatore nel campo "origine"

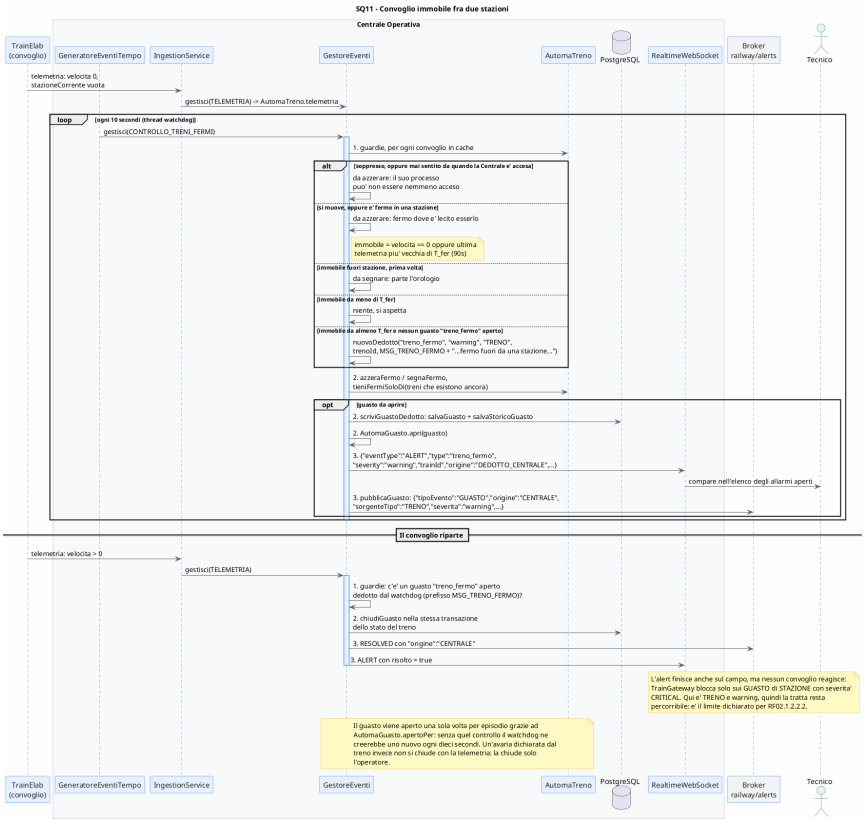


2.10.12 SQ11 convoglio fermo fuori stazione

- stesso watchdog di SQ10 ma con severita' warning: e' un'anomalia da guardare, non un fail-stop, e infatti non blocca nessuno
- l'alert finisce comunque sul campo, ma nessun convoglio reagisce perche' il blocco scatta solo sui guasti di stazione con severita' critica
- l'anomalia non scatta al primo giro in cui il convoglio risulta immobile: la prima volta parte un orologio (da quando e' fermo), e il guasto si apre solo se l'immobilita' dura almeno T_{fer}. Il controllo sul guasto gia' aperto e' quello che evita di crearne uno nuovo ogni dieci secondi
- il guasto dedotto si chiude da solo: al primo frame di telemetria in cui il convoglio si muove di nuovo (o risulta in una stazione) il gestore lo

chiude e lo annuncia, come fa il battito per SQ10

- requisiti coperti: RF02.6.2 completo, RF02.1.4 per la chiusura automatica; il fatto che la tratta resti percorribile e' il limite gia' segnalato per RF02.1.2.2.2

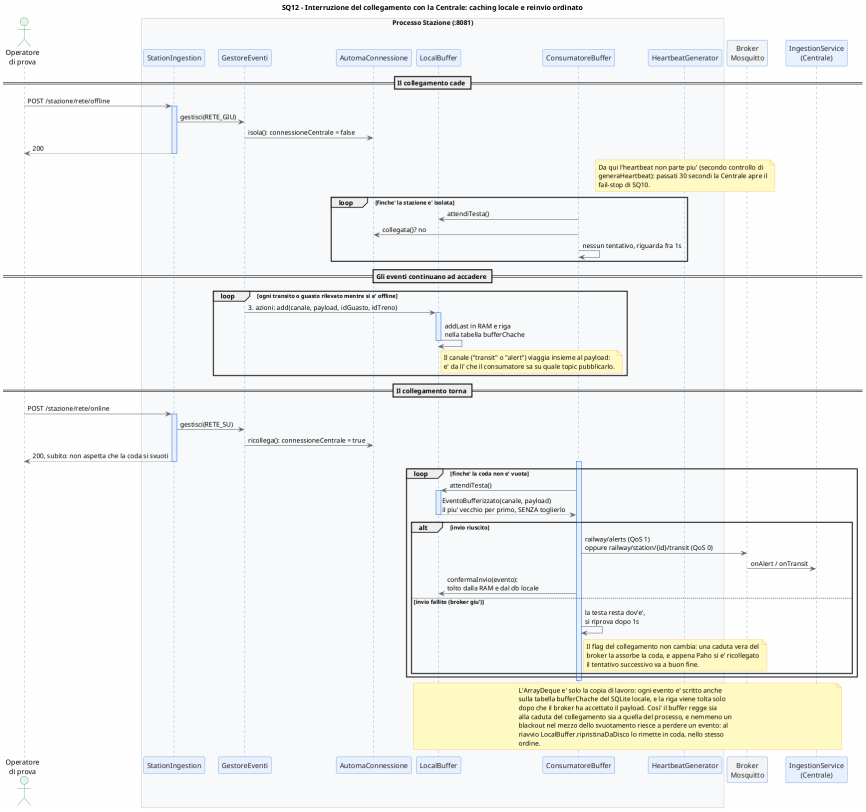


2.10.13 SQ12 store and forward durante l'interruzione del collegamento

- mentre il collegamento e' giu' non si tenta nemmeno l'invio: gli eventi entrano in coda come sempre, insieme al nome del canale su cui andranno pubblicati, e il consumatore non prova a svuotarla

- il punto vero e' che la testa si toglie solo dopo che il broker l'ha presa: se l'invio fallisce l'evento resta davanti a tutti, perche' toglierlo prima lo perderebbe e rimetterlo in fondo lo renderebbe il piu' recente, e la centrale riceverebbe una uscita prima della relativa entrata

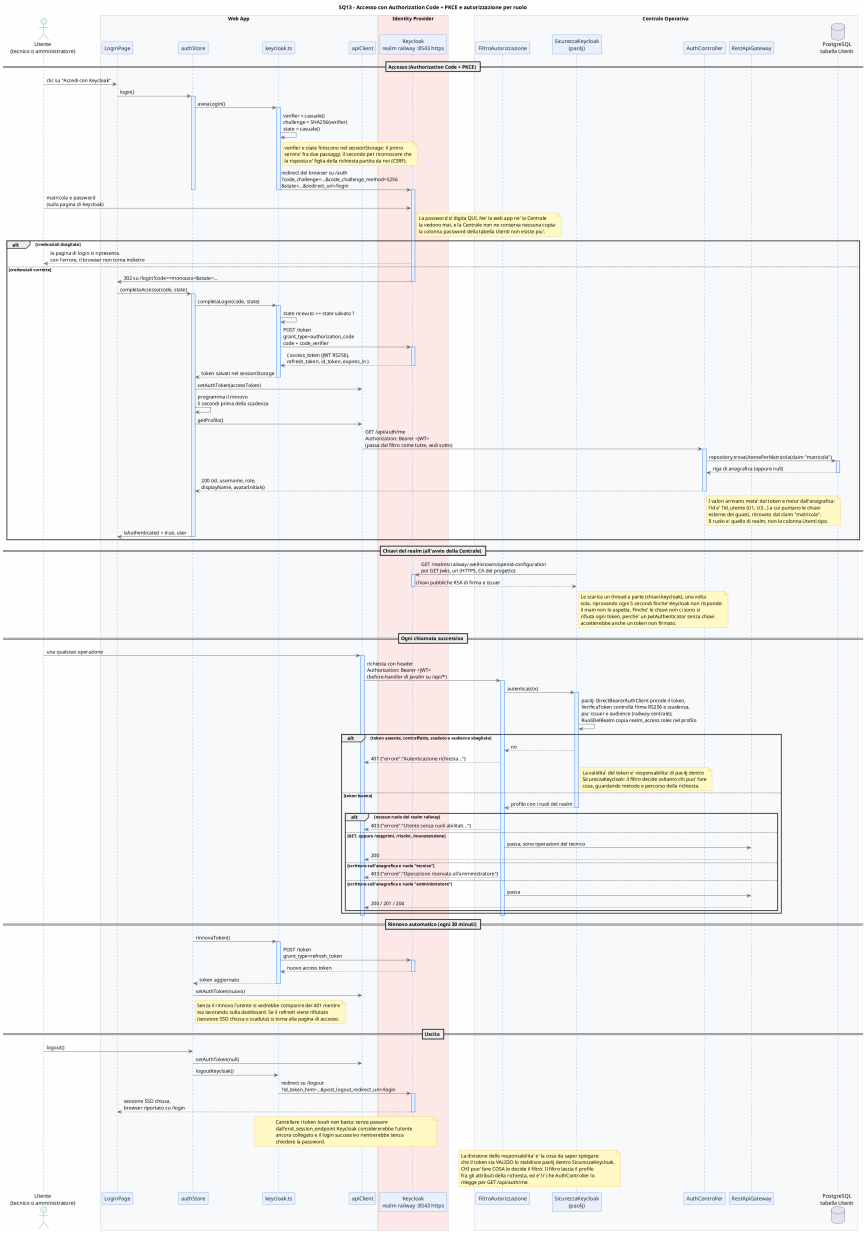
- il buffer sta in RAM ma ha una copia su SQLite: sopravvive sia alla caduta del collegamento sia a quella del processo, e la riga sul disco viene cancellata solo dopo che l'invio e' riuscito
- non c'e' nessun job periodico che passa a svuotare la coda: il consumatore e' sempre li', e riparte da solo appena il collegamento torna
- requisiti coperti: RF02.1.1.1.1 e RF02.1.1.1.2 completi, RF03.1 parzialmente per l'autonomia del nodo



2.10.14 SQ13 accesso OAuth2, token e permessi per ruolo

- la password si digita sulla pagina di Keycloak e non sulla nostra: la web app e la Centrale non la vedono mai
- il codice che torna nella URL da solo non basta, serve anche il $\text{code}_{\text{verifier}}$ che non e' mai uscito dalla memoria della pagina: e' PKCE, ed e' quello che permette a una SPA di usare l'Authorization Code senza $\text{client}_{\text{secret}}$

- la Centrale verifica la firma del token in locale con le chiavi pubbliche del realm, scaricate una volta sola all'avvio (pac4j dentro Javalin): non richiama Keycloak a ogni chiamata
- i tre esiti del filtro sono quelli osservabili dall'esterno: passa, 401 se il token manca o non vale, 403 se il ruolo non basta
- il logout non e' cancellare il token: senza passare dall'$\text{end}_{\text{sessionendpoint}}$ Keycloak terrebbe viva la sessione SSO e il rientro successivo non chiederebbe la password
- requisiti coperti: RF01.6 e RF04.1 completi, RF04.2 parziale perche' il ruolo non viene distinto sui tre comandi operativi; RF04.3 non e' una macchina ne' una sequenza, e' il profilo tls sui canali



2.10.15 SQ14 comandi di ripristino dell'operatore

- due comandi diversi che si chiudono con lo stesso effetto sul campo: l'annuncio che la stazione e' di nuovo agibile, che libera i convogli trattenuti, per questo stanno nello stesso diagramma

- l'invio della squadra non chiude niente: apre l'intervento (la stazione passa in MANUTENZIONE) e lo tiene aperto; a chiuderlo e' la chiusura dell'allarme, che e' l'unico punto in cui la stazione torna ONLINE, e solo se non le resta nessun altro guasto aperto

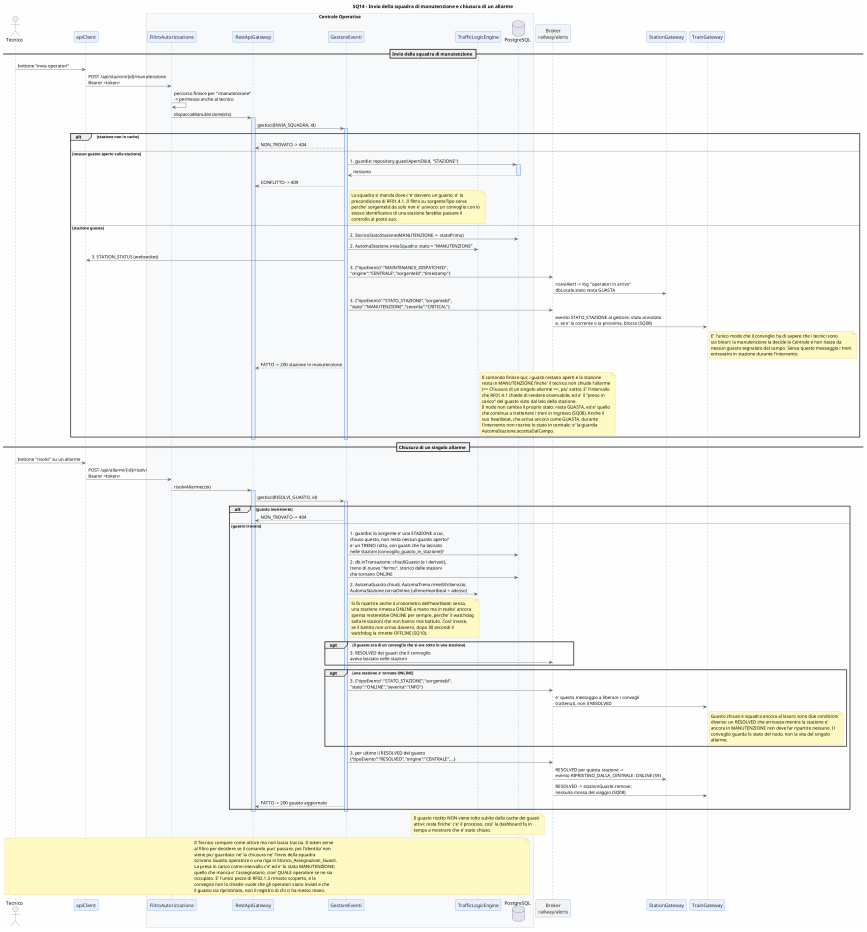
- tutti e due sono eventi per il gestore della centrale ($\text{INVIA}_{\text{SQUADRA}}$ e $\text{RISOLVI}_{\text{GUASTO}}$): RestApiGateway traduce la richiesta in un evento e l'esito del gestore nella risposta HTTP (404, 409, 200)

- quando la stazione torna ONLINE il cronometro dell'heartbeat riparte: senza, una stazione rimessa operativa a mano ma in realta' ancora spenta resterebbe operativa per sempre

- il filtro sul tipo di sorgente serve perche' l'identificativo da solo non e' univoco fra convogli e stazioni

- di tutti e due i comandi si vede l'effetto ma non l'autore: nessuno dei due scrive chi lo ha impartito

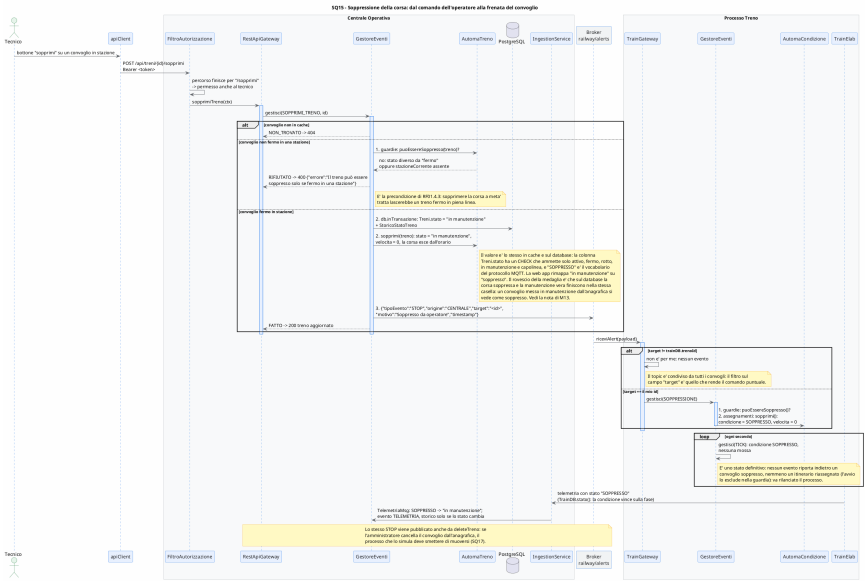
- requisiti coperti: RF01.4.1 completo (il comando porta la stazione in MANUTENZIONE, lo storicizza e avvisa il campo, e lo stato dura fino alla chiusura del guasto, vedi M12), RF01.4.2 completo (la chiusura sblocca chi era trattenuto e chiude in storico l'intervallo aperto dal comando precedente), RF02.1.3 parzialmente: il ciclo di vita del guasto si apre e si chiude, e l'intervallo di lavorazione e' leggibile come MANUTENZIONE sulla stazione, ma l'assegnatario non viene mai registrato



2.10.16 SQ15 soppressione di un convoglio

- il comando si accetta solo con il convoglio fermo in una stazione: e' una guardia di AutomaTreno, valutata dal gestore prima di scrivere qualsiasi cosa, e se dice di no l'operatore riceve un 400 con il motivo
- sul database e in cache va lo stesso valore, "in manutenzione", perche' e' quello che il vincolo della colonna ammette ("SOPPRESSO" e' il vocabolario del protocollo MQTT); la web app lo mostra come "soppresso"
- sul campo il messaggio e' broadcast e il filtro sul destinatario e' quello che lo rende puntuale

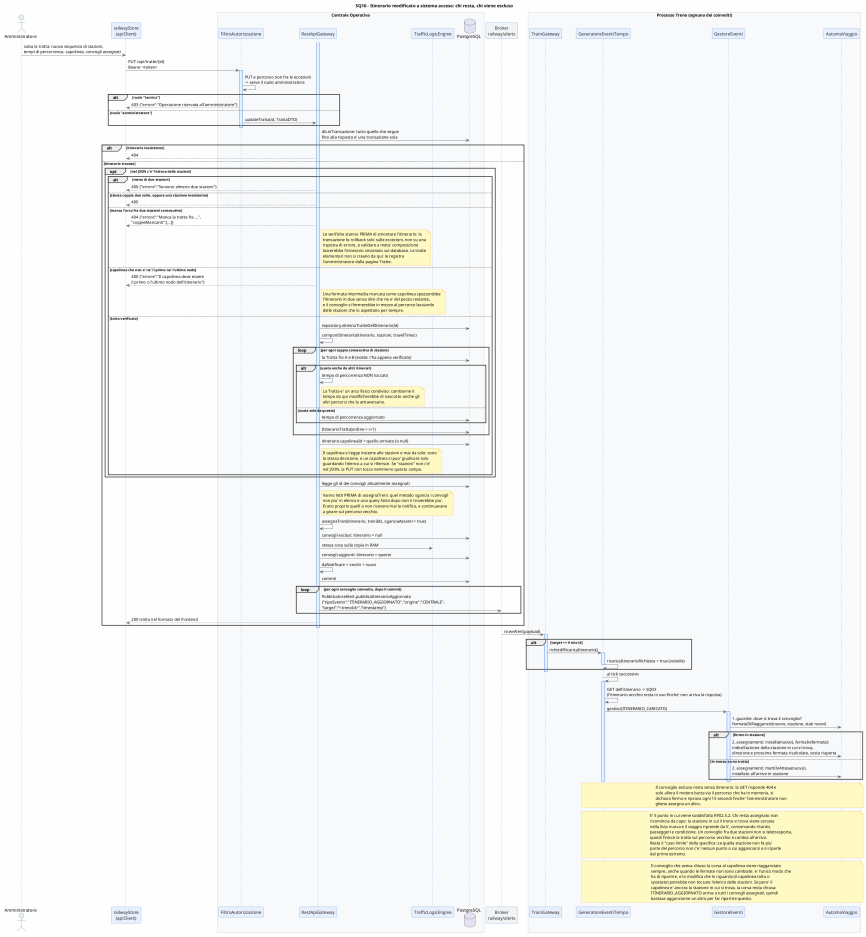
- lo stato e' definitivo per il processo del convoglio: nessun ramo del motore riporta indietro un convoglio soppresso
- requisiti coperti: RF01.4.3 completo: l'effetto e' quello richiesto (il convoglio passa a SOPPRESSO e non viaggia piu') e la precondizione "il convoglio e' fermo in una stazione" e' la guardia del comando



2.10.17 SQ16 modifica dell'itinerario a sistema acceso

- gli identificativi dei convogli assegnati vanno letti prima di riassegnare: chi viene sganciato non risulterebbe piu' da nessuna query e non riceverebbe mai la notifica, continuando a girare sul percorso vecchio
- vengono avvisati sia i convogli esclusi sia quelli appena agganciati
- chi resta assegnato si riaggancia al percorso nuovo nella stazione in cui si trova, e se era fra due stazioni lo fa appena arriva alla successiva; chi e' stato escluso resta senza itinerario e riprova ogni 15 secondi
- e' anche la strada con cui si rimette in servizio un convoglio che ha chiuso la corsa: togliere o spostare il capolinea e' una modifica dell'itinerario come le altre, e l'avviso arriva a tutti i convogli assegnati
- il tempo di percorrenza di una tratta condivisa con altri itinerari non viene toccato, se no si cambierebbero di nascosto anche gli altri percorsi

- requisiti coperti: RF02.5.1 completo e RF01.3.4 per la composizione dell'itinerario. RF02.5.2 e' coperto per la parte di centrale, che avvisa tutti i convogli coinvolti; il punto da cui si riprende lo decide il treno (T3, T4 e la nota finale)

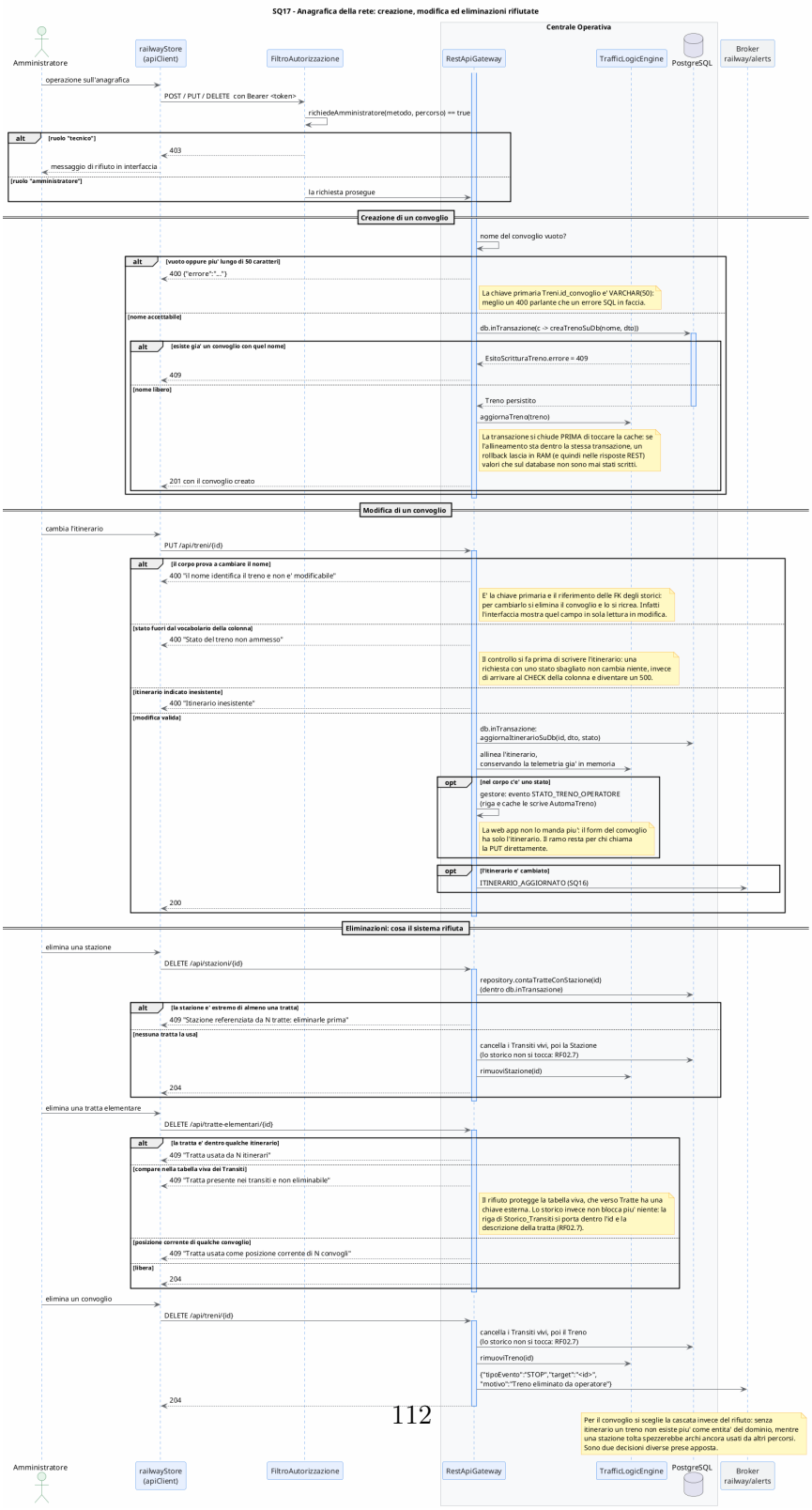


2.10.18 SQ17 amministrazione dell'anagrafica della rete

- il diagramma mostra i rifiuti, non i casi che vanno bene: sono quelli la parte osservabile del requisito sull'integrita' dei dati

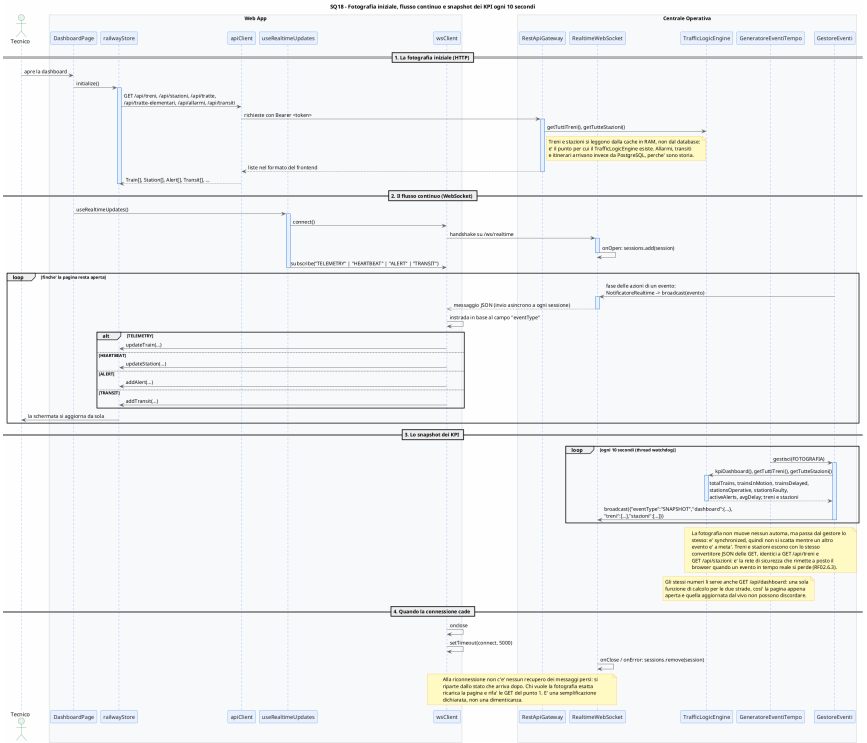
- una stazione ancora estremo di qualche tratta non si cancella, e nemmeno una tratta usata da un itinerario, presente nei transiti ancora vivi o posizione corrente di un convoglio

- per il convoglio invece si e' scelta la cancellazione a cascata degli storici: sono due decisioni diverse prese apposta, perche' una stazione tolta spezzerebbe archi ancora usati da altri percorsi
- il nome del convoglio non e' modificabile perche' e' la chiave primaria e il riferimento delle chiavi esterne di quattro storici
- dall'interfaccia si scrivono solo nome e coordinate della stazione, nome e itinerario del convoglio: lo stato (e i binari della stazione) la PUT lo accetta ancora, ma la web app non lo manda
- requisiti coperti: RF01.3.1, RF01.3.2 e RF01.3.3 completi; RF01.3.5 parzialmente, perche' l'integrita' la tengono anche i vincoli del database e non solo questi controlli


SQ17 - Anagrafica della rete: creazione, modifica ed eliminazioni rifiutate
Centrale Operativa
Amministratore
railwayStore (apiClient)
FiltroAutorizzazione
RestApiGateway
TrafficLogicEngine
PostgreSQL
Broker railway/alerts
operazione sull'anagrafica
POST / PUT / DELETE con Bearer <token>
richiedeAmministratore(metodo, percorso) == true
alt [ruolo "tecnico"]
403
messaggio di rifiuto in interfaccia
[ruolo "amministratore"]
la richiesta prosegue
Creazione di un convoglio
nome del convoglio vuoto?
alt [vuoto oppure piu' lungo di 50 caratteri]
400 {"errore":"..."}
La chiave primaria Treni.id_convoglio e' VARCHAR(50): meglio un 400 parlante che un errore SQL in faccia.
[nome accettabile]
db.inTransazione(c -> creaTrenoSuDb(nome, dto))
alt [esiste gia' un convoglio con quel nome]
EsitoScritturaTreno.errore = 409
409
[nome libero]
Treno persistito
aggiornaTreno(treno)
La transazione si chiude PRIMA di toccare la cache: se l'allineamento sta dentro la stessa transazione, un rollback lascia in RAM (e quindi nelle risposte REST) valori che sul database non sono mai stati scritti.
201 con il convoglio creato
Modifica di un convoglio
cambia l'itinerario
PUT /api/treni/{id}
alt [il corpo prova a cambiare il nome]
400 "il nome identifica il treno e non e' modificabile"
E' la chiave primaria e il riferimento delle FK degli storici: per cambiarlo si elimina il convoglio e lo si ricrea. Infatti l'interfaccia mostra quel campo in sola lettura in modifica.
[stato fuori dal vocabolario della colonna]
400 "Stato del treno non ammesso"
Il controllo si fa prima di scrivere l'itinerario: una richiesta con uno stato sbagliato non cambia niente, invece di arrivare al CHECK della colonna e diventare un 500.
[itinerario indicato inesistente]
400 "Itinerario inesistente"
[modifica valida]
db.inTransazione: aggiornaItinerarioSuDb(id, dto, stato)
allinea l'itinerario, conservando la telemetria gia' in memoria
opt [nel corpo c'e' uno stato]
gestore: evento STATO_TRENO_OPERATORE (riga e cache le scrive AutomaTreno)
La web app non lo manda piu': il form del convoglio ha solo l'itinerario. Il ramo resta per chi chiama la PUT direttamente.
opt [l'itinerario e' cambiato]
ITINERARIO_AGGIORNATO (SQ16)
200
Eliminazioni: cosa il sistema rifiuta
elimina una stazione
DELETE /api/stazioni/{id}
repository.contaTratteConStazione(id) (dentro db.inTransazione)
alt [la stazione e' estremo di almeno una tratta]
409 "Stazione referenziata da N tratte: eliminarle prima"
[nessuna tratta la usa]
cancella i Transiti vivi, poi la Stazione (lo storico non si tocca: RF02.7)
rimuoviStazione(id)
204
elimina una tratta elementare
DELETE /api/tratte-elementari/{id}
alt [la tratta e' dentro qualche itinerario]
409 "Tratta usata da N itinerari"
[compare nella tabella viva dei Transiti]
409 "Tratta presente nei transiti e non eliminabile"
Il rifiuto protegge la tabella viva, che verso Tratte ha una chiave esterna. Lo storico invece non blocca piu' niente: la riga di Storico_Transiti si porta dentro l'id e la descrizione della tratta (RF02.7).
[posizione corrente di qualche convoglio]
409 "Tratta usata come posizione corrente di N convogli"
[libera]
204
elimina un convoglio
DELETE /api/treni/{id}
cancella i Transiti vivi, poi il Treno (lo storico non si tocca: RF02.7)
rimuoviTreno(id)
{"tipoEvento":"STOP","target":"<id>", "motivo":"Treno eliminato da operatore"}
204
Per il convoglio si sceglie la cascata invece del rifiuto: senza itinerario un treno non esiste piu' come entita' del dominio, mentre una stazione tolta spezzerebbe archi ancora usati da altri percorsi. Sono due decisioni diverse prese apposta.

2.10.19 SQ18 dashboard in tempo reale e snapshot periodico

- tre meccanismi diversi tenuti insieme: la fotografia iniziale via HTTP, il flusso continuo via websocket e lo snapshot dei numeri di sintesi ogni 10 secondi
- i numeri della dashboard li calcola una funzione sola, usata sia dalla chiamata REST sia dallo snapshot: cosi' la pagina appena aperta e quella aggiornata dal vivo non possono discordare
- alla riconnessione non c'e' nessun recupero dei messaggi persi, si riparte dagli eventi nuovi: e' una semplificazione dichiarata
- requisiti coperti: RF01.1.4 e RF01.1.5 completi, RF02.6.3 completo per la fotografia periodica dello stato



2.10.20 riepilogo requisiti - diagrammi di sequenza

E' la lettura al contrario delle righe "requisiti coperti" scritte sopra ogni diagramma: qui si parte dal requisito e si trovano le conversazioni che lo realizzano.

Un requisito e' in **coperto da** quando i diagrammi elencati lo mostrano per intero, in **parzialmente** quando ne mostrano solo un pezzo. Le righe vuote in entrambe le colonne sono i requisiti che nessuna conversazione realizza: sono lasciate apposta, come nella tabella delle macchine a stati. E come quella, non e' lo **stato di implementazione** scritto accanto al requisito nel capitolo *Specifica*: qui si dice quale conversazione lo mostra, non se il sistema lo fa.

requisito	coperto da	parzialmente	nota
RF01.1.1	SQ04	SQ18	la telemetria porta la posizione, SQ18 e' il canale che la tiene aggiornata
RF01.1.2	SQ04	SQ18	
RF01.1.3	SQ05	SQ06, SQ18	lo stato della stazione dal battito, i presenti sui binari dal transito
RF01.1.4	SQ18		l'evento arriva quando accade, non a interrogazione
RF01.1.5	SQ18		i numeri di sintesi viaggiano nello snapshot da 10 secondi
RF01.2.1	SQ09	SQ10, SQ11	l'elenco si riempie da tre sorgenti diverse
RF01.2.2	SQ10, SQ09		SQ09 e' la porta del segnalato e SQ10 quella del dedotto; l'origine arriva all'elenco nel campo `origine`
RF01.3.1	SQ17		
RF01.3.2	SQ17		
RF01.3.3	SQ17		
RF01.3.4	SQ16	SQ17	comporre l'itinerario e assegnarci i convogli e' la stessa chiamata
RF01.3.5		SQ17	i rifiuti si vedono qui, ma l'integrita' la tengono anche i vincoli del db
RF01.4.1	SQ14		il comando parte, il campo viene avvisato e MANUTENZIONE resta finche' il guasto non viene chiuso
RF01.4.2	SQ14, SQ08		chiuso il guasto il ripristino sblocca chi era trattenuto; chi lo ha chiuso non viene registrato, ma non e' richiesto
RF01.4.3	SQ15		la precondizione "fermo in stazione" e' la guardia del comando
RF01.5	SQ06	SQ18	SQ06 scrive lo storico dei transiti, SQ18 lo porta a schermo
RF01.6	SQ13		
RF02.1.1.1.1	SQ12		
RF02.1.1.1.2	SQ12		la testa si toglie solo a invio riuscito: e' cio' che conserva l'ordine

Continued on next page

Continued from previous page

requisito	coperto da	parzialmente	nota
RF02.1.1.2.1	SQ08	SQ07, SQ10	la stazione diventa impercorribile per due strade diverse
RF02.1.1.2.2	SQ08	SQ14, SQ10	riparte al ripristino, non per scadenza di un tempo
RF02.1.1.2.3		SQ08	chi era gia' fermo li' viene trattenuto solo al controllo di ripartenza
RF02.1.2.1	SQ09		
RF02.1.2.2.1	SQ09	SQ08	la centrale chiude la stazione in cui il convoglio si e' rotto; chi ci e' diretto si ferma come in SQ08
RF02.1.2.2.2		SQ11	si apre solo un allarme sul convoglio, la tratta resta percorribile
RF02.1.3		SQ09, SQ14	apertura con deduplica e chiusura sono i due estremi del ciclo, ma in mezzo non c'e' ne' la presa in carico ne' l'assegnatario
RF02.1.4	SQ10		il prefisso nel messaggio e' cio' che permette la chiusura automatica
RF02.2.1.1	SQ08		il ritardo cresce mentre il convoglio e' trattenuto
RF02.2.2.1	SQ04	SQ06	la posizione e' interpolata dal motore e corretta dai passaggi
RF02.2.2.2	SQ06	SQ08	il ritardo lo misura la centrale al passaggio e lo comunica al convoglio con l'alert RITARDO; fra un passaggio e l'altro lo stima dalla telemetria
RF02.2.2.3	SQ04		
RF02.2.2.4	SQ04		velocita' e passeggeri sono gli unici dati ancora simulati
RF02.2.3		SQ06	l'inversione all'estremo e' nel motore, nessuna conversazione la mostra sola
RF02.2.4		SQ06	i passaggi spaiati nascono agli estremi e finiscono nel transito puntuale
RF02.3.1	SQ05	SQ07	
RF02.3.2	SQ05		
RF02.3.3	SQ07		
RF02.3.4	SQ05	SQ07	la stazione dichiara il proprio stato dentro il battito
RF02.4.1	SQ01, SQ02		
RF02.4.2	SQ01, SQ02		l'esito negativo e' l'uscita del processo, non un avviso

Continued on next page

Continued from previous page

requisito	coperto da	parzialmente	nota
RF02.4.3	SQ01, SQ02	SQ03	senza centrale non si entra in servizio e non si carica l'itinerario
RF02.5.1	SQ16		
RF02.5.2	SQ16, SQ03		il convoglio ancora assegnato ricarica passando da SQ03 e si riaggancia dove si trova; a meta' tratta il cambio scatta all'arrivo
RF02.6.1	SQ10		
RF02.6.2	SQ11		
RF02.6.3	SQ18		
RF02.7		SQ04, SQ06	lo storico si scrive qui, ma nessuna conversazione lo conserva da sola; itinerari percorsi e assegnazioni degli operatori non li scrive nessuno
RF03.1		SQ01, SQ12	l'autonomia si vede nel nodo che vive anche senza centrale
RF03.2		SQ04, SQ08	il disaccoppiamento e' il broker disegnato come partecipante a se'
RF03.3	SQ06	SQ04	SQ06 e' la via della rilevazione a terra, SQ04 quella della dichiarazione del treno
RF03.4	SQ07	SQ09	i sensori di terra e quelli di bordo usano solo le API pubbliche del nodo
RF03.5	SQ03		
RF04.1	SQ13		
RF04.2		SQ13, SQ16, SQ17	il 403 sull'anagrafica e' la parte visibile del ruolo; sui comandi operativi di SQ14 e SQ15 il ruolo non viene distinto
RF04.3			non e' una conversazione: e' il profilo `%tls` sui canali, vedi la sezione *Topic*

2.11 Diagrammi delle classi

Sette diagrammi e una mappa: `C1` il nodo Treno, `C2` il nodo Stazione, la mappa della Centrale che tiene insieme `C3`, `C4a`, `C4b` e `C5`, e `C6` la Web App. Le entita' non hanno un diagramma a se': stanno nel capitolo *Modello*.

La Centrale ha quattro figure e non due perche' e' l'unica componente che non sta in un diagramma solo: `C3` e' la faccia verso l'operatore, `C4a` la porta MQTT verso il campo, `C4b` il dominio che decide (il gestore degli eventi e gli automi), `C5` il piano della persistenza sotto a tutti.

In nessuno dei tre moduli Java c'e' un framework che crea gli oggetti:

ogni processo ha un `main` che li crea con `new`, una copia sola per classe, e ne assegna a mano i campi con cui si parlano, gli stessi che assegnano i test. Le frecce dei diagrammi sono quei campi.

Legenda degli archi. Sono quelli di UML e in tutto il capitolo ne compaiono sette:

`A --> B` **associazione diretta**: A conosce B e lo usa. Nei moduli Java e' un campo di A che punta a B, assegnato dal `main`, oppure un campo che conserva un valore di B (in `C1` l'`Itinerario` tenuto in attesa); in `C6` un import fra moduli.

`A ..> B` **dipendenza**: A usa B senza tenerne un campo, tipicamente una chiamata REST fra due processi. Tratteggiata anche quando il campo c'e' ma B sta fuori dal diagramma.

`A ..|> B` **realizzazione**: A implementa l'interfaccia B.

`A +-- B` **classe annidata**: B e' dichiarata dentro A, come i DTO di `RestApiGateway`.

`A o-- B` **aggregazione**: A raccoglie dei B che pero' esistono anche senza A. Solo in `C6`.

`A *-- B` **composizione**: i B nascono dentro A e vivono solo con lui. Solo in `C2`: i sensori di binario simulati li crea e li tiene il `SimulatoreSensori`.

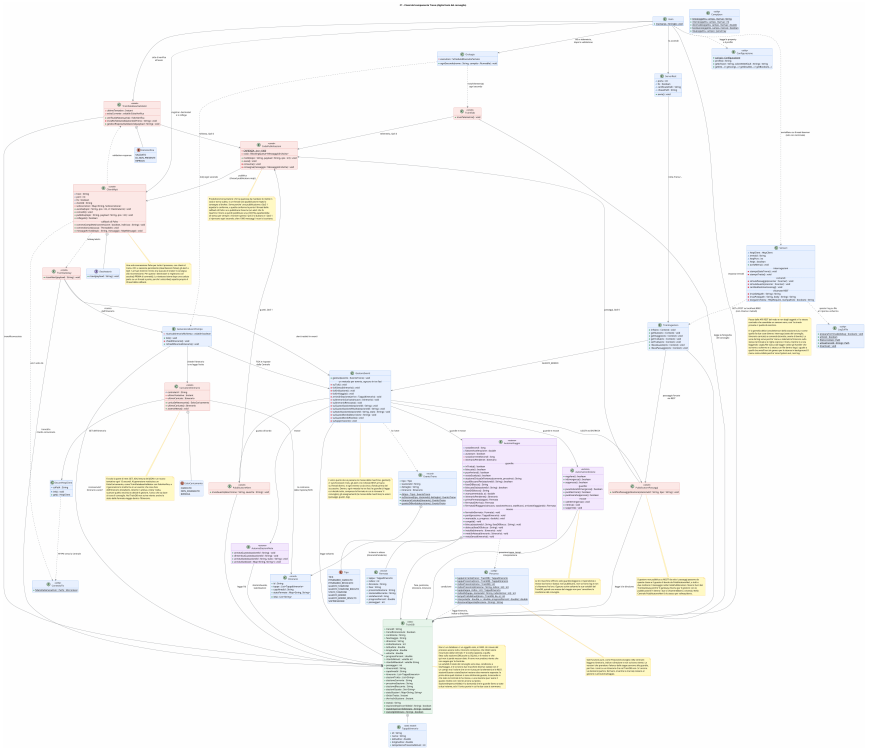
`A <..  B` **dipendenza entrante**: la stessa di `..>`, disegnata al contrario.

Non compare l'**ereditarieta'** (`--|>`): nel codice le sole sottoclassi estendono una classe di libreria (`VerificaToken` il `JwtAuthenticator` di pac4j, il formato dei log il `Formatter` di `java.util.logging`), e non sono gerarchie del sistema. Il filo tratteggiato senza punta lega le note al loro riquadro, gli archi `-[hidden]-` della mappa servono solo a impaginare, e i colori sono stereotipi locali a ogni figura.

2.11.1 Il componente Treno (C1)

- requisiti coperti: RF03.1 parzialmente, il convoglio e' un processo a se' con il proprio stato; RF02.2.2 per i campi che descrivono il convoglio
- il viaggio si legge in tre ruoli: i generatori di eventi (GeneratoreEventiTempo, TrainGateway, TrainIngestion) traducono quello che succede in un EventoTreno; GestoreEventi e' l'unico che fa muovere le macchine, un evento alla volta e sempre nell'ordine guardie, assegnamenti,

azioni; le macchine (AutomaViaggio, AutomaCondizione, AutomaStazioniNote) offrono solo guardie e mosse e non si conoscono fra loro. Quello che esce verso la rete lo pubblicano PubblicatorePassaggi e PubblicatoreAlert, che il gestore chiama dopo le mosse

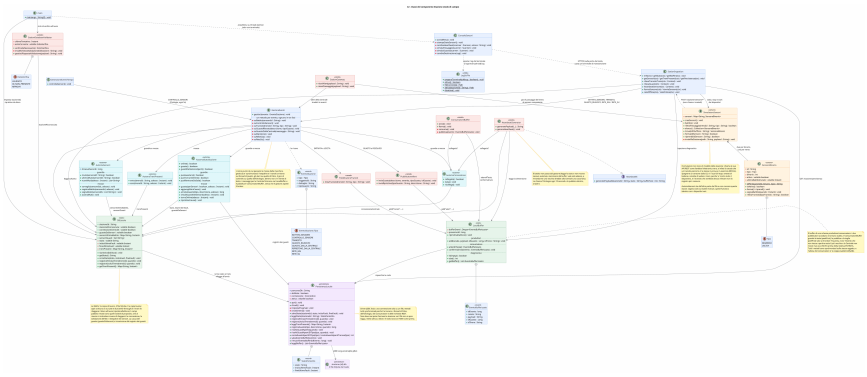


2.11.2 Il componente Stazione (C2)

- requisiti coperti: RF02.1.1.1.1 e RF02.1.1.1.2 per la struttura che regge il caching, RF03.1 per l'autonomia del nodo (con PersistenzaLocale il nodo si ricostruisce da solo anche dopo un riavvio, senza chiedere niente alla Centrale)

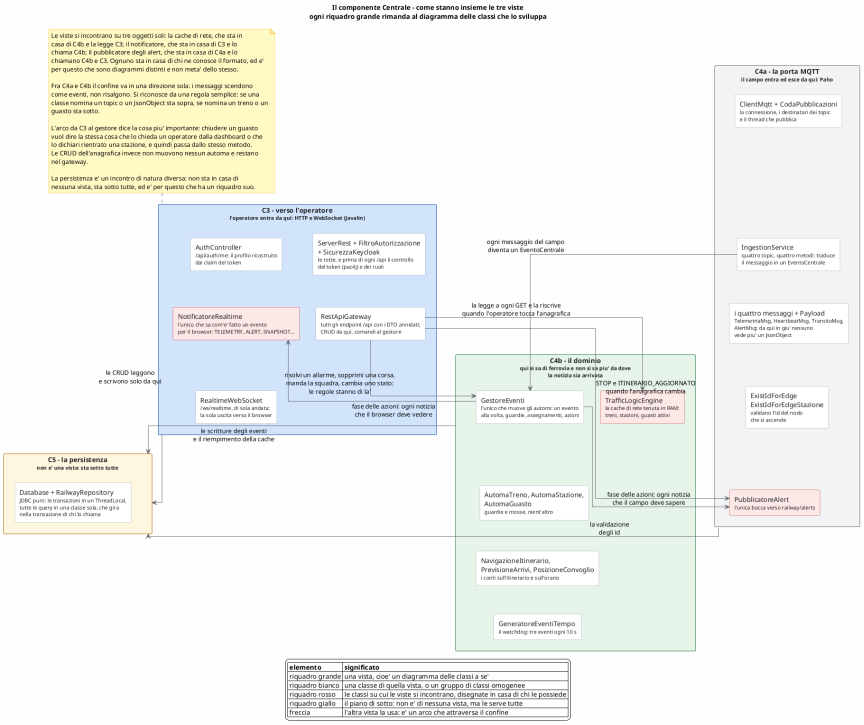
- si legge negli stessi tre ruoli del treno: i generatori di eventi (GeneratoreEventiTempo, StationIngestion, StationGateway) traducono quello che succede in un EventoStazione; GestoreEventi e' l'unico che fa muovere le macchine, un evento alla volta e sempre nell'ordine guardie, assegnamenti, azioni; le macchine (AutomaStatoStazione, AutomaSensori, AutomaTreniPresenti, AutomaConnessione) offrono solo guardie

e mosse e non si conoscono fra loro. Quello che esce verso la Centrale lo preparano PubblicatoreTransiti e PubblicatoreAlert, lo accoda il LocalBuffer e lo pubblica il ConsumatoreBuffer



2.11.3 Il componente Centrale: mappa delle tre viste

- requisiti coperti: nessuno in proprio, e' l'indice dei tre diagrammi che seguono
- la mappa serve soprattutto a far vedere il confine fra chi sa di MQTT (C4a) e chi sa di ferrovia (C4b): tutto quello che sta sopra traduce e basta, tutto quello che cambia lo stato passa da un metodo solo, GestoreEventi.gestisci()

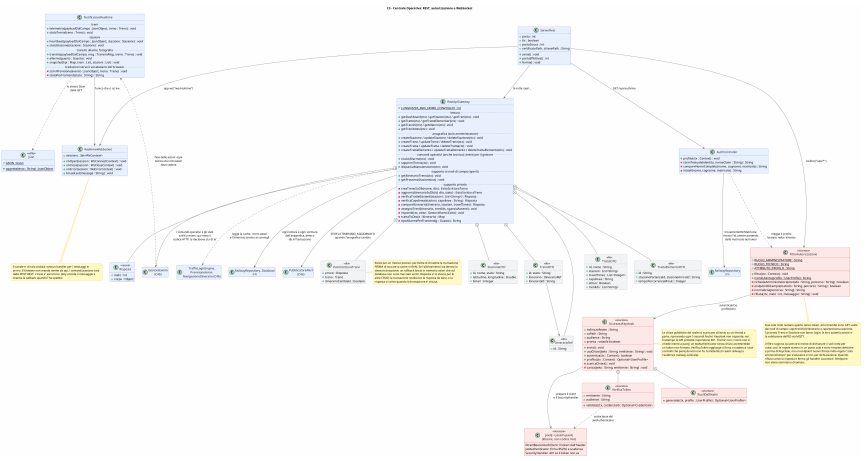


2.11.4 Il componente Centrale: verso l'operatore (C3)

- e' la faccia HTTP e websocket della centrale, fatta con Javalin: ServerRest registra le rotte, e prima di ogni richiesta sotto /api passa il before-handler di FiltroAutorizzazione, non un controllo sparso negli endpoint

- la validita' del token la stabilisce pac4j dentro SicurezzaKeycloak, i ruoli li decide il filtro; gli endpoint traducono la richiesta: le CRUD scrivono l'anagrafica in una transazione, i comandi operativi e gli stati scelti a mano diventano eventi per il gestore (C4b), e l'esito diventa il codice HTTP

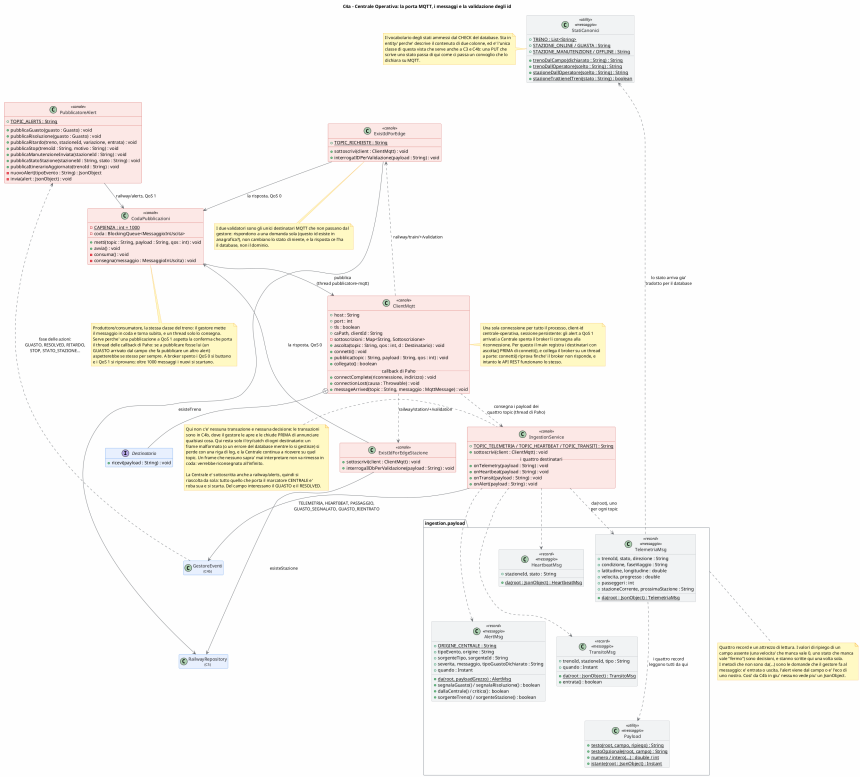
- requisiti coperti: RF01.3 e RF01.4 per la superficie dei comandi, RF01.6, RF04.1 e RF04.2 per la parte di accesso



2.11.5 Il componente Centrale: verso il campo, la porta MQTT (C4a)

- e' tutto cio' che la Centrale sa di MQTT, e finisce qui: il client Paho consegna i payload, l'ingestione li trasforma in un messaggio con un nome e poi in un evento per il gestore, e da C4b in giu' nessuno vede piu' un topic. In uscita e' lo stesso al contrario: il PubblicatoreAlert compone il messaggio e la CodaPubblicazioni lo consegna al broker su un thread suo

- requisiti coperti: RF03.3 per le due vie con cui il passaggio si conosce (il transito certificato e la telemetria, che entrano da due destinatari diversi), RF02.4 per la validazione degli id dei nodi che si accendono



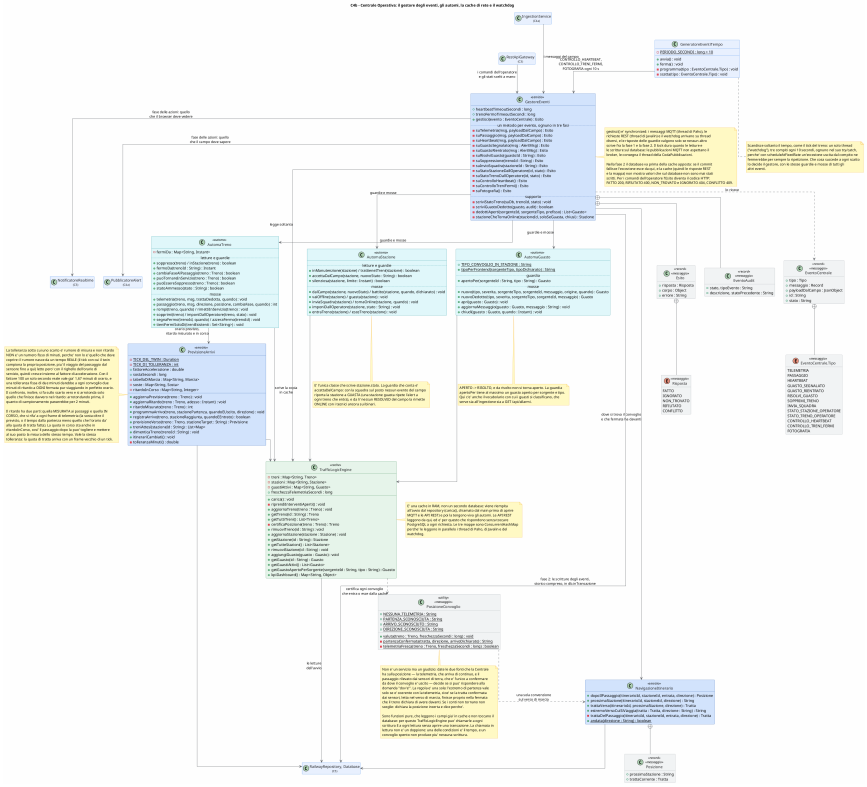
2.11.6 Il componente Centrale: il dominio, gestore degli eventi e automi (C4b)

- e' la vista in cui la Centrale sa di ferrovia e non sa piu' da dove la notizia sia arrivata: il messaggio MQTT di C4a, il comando REST di C3 e il tick del watchdog diventano tutti un EventoCentrale, e da li' in poi il trattamento e' lo stesso

- lo schema e' quello dei nodi di campo: GestoreEventi e' l'unico che fa muovere gli automi, un evento alla volta, sempre nell'ordine guardie, assegnamenti, azioni; gli automi (AutomaTreno, AutomaStazione, AutomaGuasto) offrono solo guardie e mosse, scrivono la sola copia in cache e non si chiamano fra loro; la riga del database la scrive il gestore, in una transazione, prima delle mosse

- requisiti coperti: RF02.6.1, RF02.6.2 e RF02.6.3 per le anomalie dedotte, RF02.1.3 per il ciclo di vita del guasto, RF02.1.2.2.1 per la stazione chiusa da un convoglio guasto, RF02.2.2.1 per la posizione

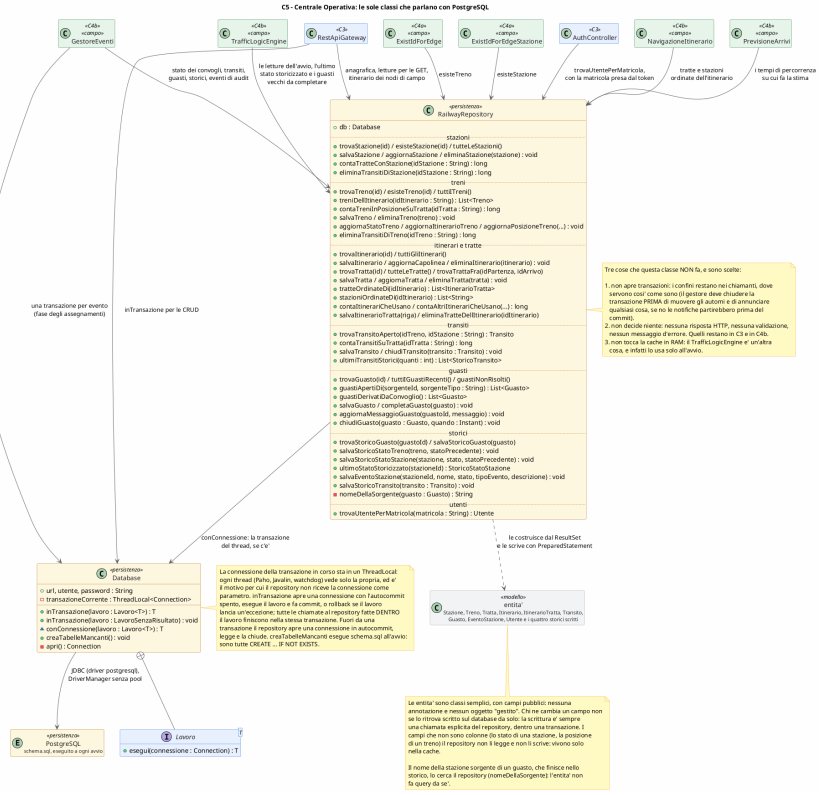
che la centrale e' disposta a garantire (`PosizioneConvoglio`), RF02.7 parzialmente per la scrittura della memoria storica



2.11.7 Il componente Centrale: la persistenza (C5)

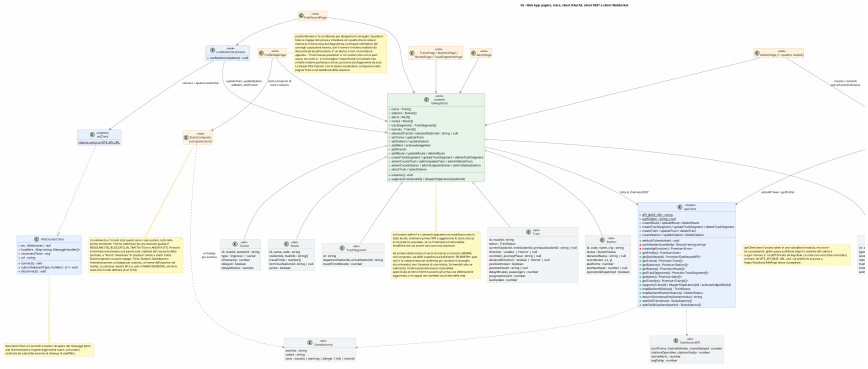
- non e' una quarta vista ma il piano sotto le altre tre: Database apre le connessioni e le transazioni JDBC, RailwayRepository e' l'unica classe della Centrale che scrive SQL, e li usano sia il lato operatore (C3) sia il lato campo (C4a e C4b)

- requisiti coperti: nessuno in proprio; da qui passano le scritture della memoria storica (RF02.7) e le letture dell'anagrafica che servono a RF01.3



2.11.8 Il componente Web App (C6)

- requisiti coperti: RF01.1, RF01.2 e RF01.3 per le schermate, RF01.6 per l'accesso; RF04.2 solo in apparenza, perche' il ruolo lo decide il server e non queste classi



2.11.9 riepilogo requisiti - diagrammi delle classi

requisito	coperto da	parzialmente	nota
RF01.1 (schermate)		C6, C3	le pagine stanno in C6, i dati che leggono arrivano dagli endpoint di C3
RF01.2 (guasti)		C6, C3, modello	l'entita' Guasto e il suo storico stanno nel capitolo Modello; il campo operatore resta sempre nullo
RF01.3 (anagrafica)		C3, modello	i comandi in C3, i vincoli e le chiavi esterne nel modello
RF01.4 (comandi)		C3, C1, C2	partono da C3 e arrivano ai gateway dei nodi; nessuna classe registra chi li ha impartiti
RF01.5 (storico)		modello, C3	le otto tabelle di storico e gli endpoint che le servono; solo quattro vengono davvero scritte, e nell'interfaccia non c'e' una schermata che le mostri
RF01.6 (accesso)	C3	C6	filtro lato server, client OAuth2 e custodia del token lato browser
RF02.1.1.1		C2	la coda locale, i pubblicatori che la riempiono e il consumatore che la svuota
RF02.1.1.2		C1, C2	le stazioni note al treno e l'automa del viaggio che si blocca
RF02.1.2		C1, C4a, C4b	il gateway di bordo, l'ingestione degli allarmi, il gestore e l'automa che li aprono
RF02.1.3		modello, C4b	il ciclo di vita e' un campo dell'entita', ma le sue transizioni stanno in un posto solo, AutomaGuasto: restano due stati, senza assegnatario
RF02.2.1		C1	il convoglio trattenuto resta fermo; il ritardo lo calcola la Centrale
RF02.2.2		C1, modello	i campi volatili del convoglio, negli stessi nomi ai due capi
RF02.2.3		C1	direzione e indice della stazione nel modello locale del treno
RF02.3		C2, C4b	lo stato del nodo e la sua copia nella cache della centrale

Continued on next page

Continued from previous page

requisito	coperto da	parzialmente	nota
RF02.4		C1, C2, C4a	i due validatori di campo e le due classi che rispondono in centrale
RF02.5		C3, C1	il metodo che ricompone l'itinerario e il gestore che lo riscarica e riaggancia il convoglio dove si trova
RF02.6		C4b	il watchdog con i suoi tre lavori periodici
RF02.7		modello, C5	le otto tabelle di storico stanno nel Modello e non in un diagramma; a scriverle e' tutte il repository, che e' l'unica classe che parla col database; quattro di esse restano sempre vuote
RF03.1		C1, C2	ogni nodo ha il proprio stato e il proprio punto di ingresso
RF03.2		C1, C2, C4a	nessuna classe condivisa fra i processi: e' la mappa C0 a dirlo
RF03.3		C4a, C2	due destinatari distinti in centrale (telemetria e transito) e, sul nodo, i sensori che certificano il passaggio
RF03.4		C1, C2	gli endpoint REST dei nodi sono l'unica porta dei sensori
RF03.5		C3, C1	l'endpoint dell'itinerario e il client che lo chiama
RF04.1, RF04.2	C3	C6	verifica del token (pac4j in SicurezzaKeycloak), filtro sui ruoli, flusso PKCE lato SPA
RF04.3			non e' una classe: e' la configurazione del profilo `%tls`

2.11.10 riepilogo classi - macchine a stati

E' la terza lettura dello stesso codice: nella sezione *Diagrammi degli stati* si parte dalla macchina e si trovano i metodi, qui si parte dalla classe e si trova la macchina. Le sigle sono quelle di quel capitolo, T = treno (T1. . . T11), S = stazione (S1. . . S11), M = centrale (M1. . . M13), trentacinque macchine in tutto.

Che cosa vuol dire "cuore". Ogni macchina compare una volta sola, sulla riga della classe che ne **scrive le transizioni**, cioe' che assegna il valore nuovo alla variabile di stato. Le classi che servono la macchina senza farla transitare hanno un trattino, e la nota dice di quale macchina sono il

contorno: i contenitori di stato (`TrainDB`, `DBLocale`, `TrafficLogicEngine`), i client e il `RailwayRepository`.

classe	macchine	perche' e' lei il cuore (oppure di che cosa e' il contorno)
C1 - il nodo Treno		
main	T1	legge l'id, aspetta l'esito di T2 e apre il servizio operativo: tutte le altre macchine girano dentro quello stato
TrainDatabaseValidator	T2	i tre valori di `EsitoVerifica` sono gli stati della macchina; l'interlocutore e' M2
GestoreEventi	T4	e' la macchina degli eventi: ognuno passa per le sue tre fasi (guardie, assegnamenti, azioni), ed e' l'unico che chiama le mosse delle altre
AutomaViaggio	T3, T5, T7	scrive l'itinerario ricevuto o lo butta via (T3), `faseViaggio` (T5), `direzione` e `indiceStazione` (T7), sempre su richiesta del gestore
AutomaCondizione	T6	scrive `condizione`: REGOLARE, EMERGENZA, SOPPRESSO
AutomaStazioniNote	T11	scrive `stazioniGuaste` e `statoStazioni`
TrainGateway	T9	e' la porta da cui la centrale entra nel convoglio: `riceviAlert` traduce gli alert in eventi per il gestore e annota il ritardo comunicato
TrainElab	T8	`inviaTelemetria`, chiamato dall'`Orologio` ogni secondo, e' l'unica sorgente della telemetria
TrainIngestion	T10	gli endpoint `/treno` sono l'unica porta dei sensori di bordo (RF03.4)
TrainDB	–	tiene le variabili di T5, T6, T7 e T11 ma non le fa cambiare: e' un oggetto solo, in RAM, e l'unico calcolo e' `stato()`, lo stato dichiarato fuori
GeneratoreEventiTempo	–	e' il battito di T4 e fa la richiesta di T3, ma manda solo eventi: nessuna variabile di stato la scrive lui

Continued on next page

Continued from previous page

classe	macchine	perche' e' lei il cuore (oppure di che cosa e' il contorno)
EventoTreno	–	l'alfabeto delle macchine: il tipo di evento e i dati che servono per reagire
Sensori	–	e' un client di T10, non la macchina: chiama gli endpoint dall'esterno come farebbe un sensore vero
CaricatoreItinerario	–	e' il contorno di T3: fa la GET verso M10 e restituisce l'`EsitoCaricamento`, ma l'itinerario nel `TrainDB` lo scrive AutomaViaggio; con le tappe legge anche lo stato delle fermate, la fotografia da cui parte T11
PubblicatorePassaggi	–	pubblica i passaggi che il gestore decide in T5 ed e' il primo anello di SQ06, ma non fa transitare nessuna macchina
PubblicatoreAlert	–	pubblica il guasto di bordo, l'azione della mossa di T6 verso EMERGENZA
Percorso, Itinerario	–	conti e dati del motore: prossima tappa, tempi e interpolazione per T5 e T7, e l'itinerario tenuto intero insieme al suo capolinea
SecureHttpClient	–	fornisce a T3 il client HTTP, con il contesto TLS che si fida della CA del progetto
Orologio	–	fa girare T4 e T8 ogni secondo, ma decide niente: chiama `tick()` e `inviaTelemetria()` dentro un try/catch
ClientMqtt, CodaPubblicazioni	–	il trasporto MQTT: il primo consegna i messaggi in arrivo a T2 e T9, la seconda pubblica quelli in uscita su un thread suo
ServerRest	–	accende Javalin e collega le rotte `/treno/...` ai metodi di TrainIngestion (T10)

Continued on next page

Continued from previous page

classe	macchine	perche' e' lei il cuore (oppure di che cosa e' il contorno)
Configurazione, ContestoTls, CampiJson	–	contorno tecnico: le property con il profilo `tls`, il contesto TLS sulla CA del progetto, la lettura dei campi JSON
C2 - il nodo Stazione		
main	S1	la stessa di T1 sull'altro nodo: id, ripristino da disco, attesa dell'esito di S2, poi il servizio operativo
StationDatabaseValidator	S2	la stessa conversazione di T2 con un altro topic; l'interlocutore e' sempre M2
HeartbeatGenerator	S3	il battito verso la centrale, che l'`Orologio` chiama ogni 10 secondi; non passa ne' dal gestore ne' dal buffer
AutomaConnessione	S4	scrive `connessioneCentrale`: COLLEGATA e ISOLATA, e ci si passa solo con le due POST /rete
ConsumatoreBuffer	S5	e' lui a far transitare il singolo evento: guarda la testa, la pubblica e la toglie solo a invio riuscito, e da staccato non tenta nemmeno
StationIngestion	S6	gli endpoint `/stazione`: traducono le chiamate dei sensori e delle leve di prova in eventi per il gestore
AutomaSensori	S7	scrive `sensoriUltimoBattito` e `sensoriGuasti`: chi batte, chi e' muto, chi e' appena scaduto
StationGateway	S8, S9	`riceviPassaggio` e' S8 (il passaggio del treno girato al sensore competente), `riceviAlert` e' S9 (gli alert firmati dalla Centrale tradotti in eventi)
AutomaStatoStazione	S10	scrive `stato` fra ONLINE e GUASTA, la causa del guasto e il registro dei guasti aperti
AutomaTreniPresenti	S11	scrive `treniPresenti`: un'ENTRATA mette il treno sui binari, un'USCITA lo toglie

Continued on next page

Continued from previous page

classe	macchine	perche' e' lei il cuore (oppure di che cosa e' il contorno)
GestoreEventi	–	non ha una macchina sua: e' il punto da cui passano le mosse di S4, S7, S10 e S11, sempre nelle tre fasi
EventoStazione, GeneratoreEventiTempo	–	l'alfabeto delle macchine e il giro di controllo dei sensori, che manda solo eventi
PubblicatoreAlert, PubblicatoreTransiti	–	preparano i messaggi per la centrale e li accodano nel buffer: sono le azioni delle mosse, non transizioni
LocalBuffer	–	e' la coda dove gli eventi di S5 aspettano, con la copia su disco; togliere la testa solo a invio riuscito e' cio' che conserva l'ordine (RF02.1.1.1.2)
SimulatoreSensori, SensoreBinario	–	sono i dispositivi, non la macchina: rilevano il passaggio di S8 e battono verso S7, ma passano dalle API REST del nodo come farebbe l'elettronica vera
ConsoleSensori	–	e' un client di S6, la gemella di `Sensori` sul treno: spegne e riaccende i sensori per provare il fail-stop
DBLocale, PersistenzaLocale	–	tengono le variabili di S4, S7, S10 e S11, in RAM e su disco: la stessa parte di `TrainDB` sull'altro nodo
SecureHttpClient	–	fornisce il client HTTP ai sensori simulati e alla console, che chiamano le API del nodo (in HTTPS sotto profilo `tls`)
C3 - la Centrale verso l'operatore		
FiltroAutorizzazione	M9 (meta' server)	e' il before-handler che ogni richiesta sotto /api attraversa: la meta' di M9 che decide. L'altra meta' e' in C6
SicurezzaKeycloak	–	e' il contorno di M9: dice se il token e' valido (pac4j), non chi puo' fare che cosa

Continued on next page

Continued from previous page

classe	macchine	perche' e' lei il cuore (oppure di che cosa e' il contorno)
RestApiGateway	M10	la vita della richiesta dopo il filtro, con i DTO annidati; i comandi operativi li apre lui ma la transizione la scrivono gli automi, attraverso il gestore
RealtimeWebSocket	M11	l'uscita comune di tutte le altre macchine verso la dashboard, di sola andata
NotificatoreRealtime	–	e' il contorno di M11: decide che forma ha l'evento, non quando parte. Nessuno stato del dominio nasce qui
AuthController, ServerRest	–	il primo sta dentro M9 ma ricostruisce solo il profilo dai claim; il secondo accende Javalin e collega le rotte
C4a - la porta MQTT		
IngestionService	–	e' il generatore degli eventi di M3-M6: riceve i messaggi e li traduce, ma non scrive nessuna variabile di stato
ClientMqtt, CodaPubblicazioni	–	il trasporto: il primo consegna i payload ai destinatari, la seconda pubblica su un thread suo
i quattro messaggi e Payload	–	sono dati, non comportamento: dicono che cosa il campo ha detto, e nessuno di loro decide niente
PubblicatoreAlert	–	scrive i messaggi per il campo: e' l'azione di molte mosse, non una transizione
ExistIdForEdge, ExistIdForEdgeStazione	M2	una macchina sola in due classi gemelle: cambiano il topic e la tabella interrogata, non il comportamento
C4b - il dominio		

Continued on next page

Continued from previous page

classe	macchine	perche' e' lei il cuore (oppure di che cosa e' il contorno)
GestoreEventi	M3, M4, M5, M6, M8	le macchine di ingestione e il watchdog sono i suoi metodi (`suTelemetria`, `suHeartbeat`, `suPassaggio`, `suGuastoSegnalato` e `suGuastoRientrato`, `suControllo...`): ognuno passa per guardie, assegnamenti e azioni, e le variabili le scrivono gli automi su sua richiesta
AutomaGuasto	M7	l'unico posto in cui un guasto si apre e si chiude, qualunque sia la strada da cui la notizia e' arrivata
AutomaStazione	M12	l'unica classe che scrive `Stazione.stato`; la guardia `accettaDalCampo` difende la manutenzione (RF01.4.1)
AutomaTreno	M13	scrive `Treno.stato`, la posizione e il ritardo nella cache; la riga del database la scrive il gestore
GeneratoreEventiTempo	–	e' il battito di M8, ma manda solo eventi: nessuna variabile di stato la scrive lui
EventoCentrale	–	l'alfabeto delle macchine: che cosa e' successo e i dati per reagire
NavigazioneItinerario, PrevisioneArrivi	–	nessuna transizione: rispondono a domande (dove sono, quando arrivo) e misurano e stimano il ritardo per AutomaTreno
PosizioneConvoglio	–	nemmeno lei fa transitare niente, ma non e' un contorno: e' un giudizio. Date le due fonti sulla posizione decide se la centrale e' in grado di rispondere, e sono funzioni pure, senza stato e senza database
TrafficLogicEngine	–	e' la cache dove vivono le variabili di M7, M12 e M13, come `TrainDB` a bordo: le transizioni gliele scrivono gli automi
C5 - la persistenza		

Continued on next page

Continued from previous page

classe	macchine	perche' e' lei il cuore (oppure di che cosa e' il contorno)
Database, RailwayRepository	–	non fanno transitare niente: aprono le transazioni ed eseguono le query per conto delle altre. Sono il contorno di tutte le macchine della centrale che lasciano una traccia su database, cioe' M3-M8 e M10
C6 - la Web App		
keycloak, authStore	M9 (meta' browser)	PKCE, scambio del codice e rinnovo del token: M9 e' l'unica macchina che attraversa due componenti
apiClient, WebSocketClient, railwayStore, useRealtimeUpdates, le pagine	–	sono i consumatori di M10 e M11: ridisegnano quello che ricevono, nessuno stato del dominio nasce qui
StatoComposto, CampoNumerico	–	componenti di presentazione: il primo stampa un badge per automa (T5, T6, T7, M12, S10), il secondo e' il campo numerico dei form di amministrazione

La macchina che non ha una riga. Manca solo M1, il ciclo di vita del processo della centrale: il suo cuore e' il `main` di `ServeCentraleOperativa`, che non compare nelle viste perche' queste partono da oggetti gia' creati e collegati. Sui nodi di campo T1 e S1 invece ci sono, perche' li' il `main` fa anche la validazione dell'id.

Le tre macchine di dominio della centrale (M7 guasto, M12 stazione, M13 treno) hanno ciascuna un automa che e' l'unico scrittore della sua variabile, e il gestore degli eventi e' l'unico a chiamarlo.

2.12 Interfaccia utente

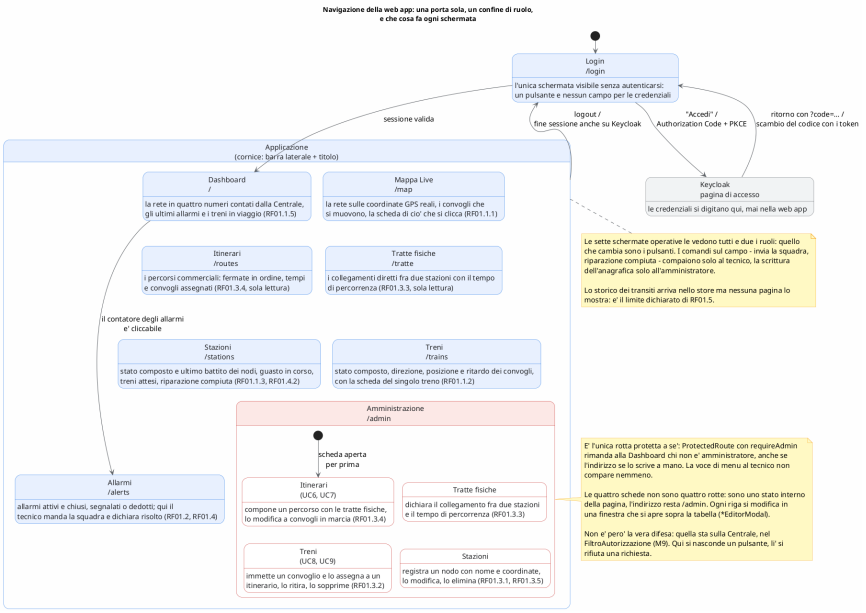
2.12.1 Mappa della navigazione

- requisiti coperti: RF01.6 per la porta d'ingresso e RF04.2 per la parte di autorizzazione che si vede lato browser: la voce *Amministrazione* non compare al tecnico e la rotta è protetta anche se qualcuno se la scrive a mano nella barra degli indirizzi

- ogni schermata porta scritto dentro che cosa fa e a quale requisito

risponde: la mappa dice in un colpo solo dove si va e perché quella pagina esiste

- la sezione *Amministrazione* è disegnata aperta, con le sue quattro schede. È l'unico punto dell'interfaccia da cui si scrive l'anagrafica della rete (RF01.3), e le schede non sono quattro rotte ma uno stato interno della pagina: cambiandole l'indirizzo resta `/admin`



3 Validazione

3.1 Come è stato verificato

La verifica procede su tre livelli, perché i requisiti non sono tutti della stessa natura. I **test automatici** controllano le singole funzioni e gli endpoint, e sono l'unica parte che si rilancia a costo zero a ogni modifica. Gli **scenari SV01-SV06** controllano il comportamento d'insieme, quello che nessun test unitario vede perché nasce dall'interazione fra nodi separati, e si eseguono a mano sul sistema acceso. I **requisiti non funzionali** non si asseriscono ma si misurano, e per ciascuno serve uno strumento diverso.

3.2 Ambiente di prova

I test automatici non hanno bisogno di nessun servizio esterno. La Centrale si accende per intero dentro il test (`CentraleDiProva`, vedi *I test hanno un database loro*): H2 in memoria al posto di PostgreSQL, una coppia di chiavi RSA generata sul momento al posto di Keycloak, e niente broker, con i messaggi del campo consegnati direttamente all'ingestione. I test dei nodi di campo costruiscono a mano gli oggetti che servono, come fa il loro `main`: quando serve il REST lo accendono su una porta libera assegnata dal sistema, il db locale sta in una cartella temporanea (o e' spento), e al posto del broker c'e' un client o una coda finta che si segna i messaggi invece di pubblicarli. Cosi' la suite puo' girare mentre il sistema vero e' acceso senza contendergli ne' le porte, ne' il database, ne' i client-id sul broker. Si lanciano con `./mvnw test` dentro ciascuno dei tre moduli Java e `npx vitest run` dentro `ClientWebAppIntefacciaUtente`.

Le prove manuali richiedono invece il sistema completo: `docker compose up` per broker, PostgreSQL e Keycloak, poi `./popola_db.sh`, che crea le tabelle con lo `schema.sql` della Centrale e carica in una sola transazione i dati demo di `populate_db.sql` (10 stazioni, 20 tratte, 15 itinerari, 15 convogli; si ferma se il database ha gia' delle stazioni), quindi i tre moduli Java (un `jar` per modulo, lanciato con `java -jar`, e `-Dprofilo=tls` per la versione cifrata) e la web app. Gli elenchi `stazioni.conf` e `treni.conf` dei nodi di campo contengono gli id dei dati demo: un id che non e' nel database viene rifiutato dalla validazione.

3.3 Test automatici

Esito dell'ultima esecuzione (1 ottobre 2026): **184 test, tutti superati** — 106 nella Centrale, 28 nei Treni, 20 nelle Stazioni e 30 nella web app. Nelle classi della Centrale che provano una sequenza (`AdminApiTest`, `ChiusuraGuastiTest`, `ItinerariTratteTest`, `MemoriaStoricaTest`, `RegressioniBugTest`) il conteggio comprende anche il primo passo, che prepara la rete, e l'ultimo, che la ripulisce.

modulo	classe	test	che cosa verifica
Centrale	`AdminApiTest`	14	gli endpoint dell'anagrafica e il filtro sui ruoli: senza token si viene respinti, il tecnico legge ma non crea, i nomi duplicati sono rifiutati (RF01.3, RF04.2)

Continued on next page

Continued from previous page

modulo	classe	test	che cosa verifica
Centrale	`SicurezzaTest`	13	la verifica dei token fatta con pac4j: passa il token buono e prendono 401 quello scaduto, quello per un'altra applicazione, di un altro realm, firmato da un'altra chiave, non firmato o manomesso, e qualunque token finche' le chiavi del realm non sono arrivate; il tecnico non tocca l'anagrafica, le due letture dei nodi di campo e la preflight del browser restano aperte (RF04.1, RF04.2)
Centrale	`GestoreEventiTest`	16	il gestore degli eventi, un evento per test: stato degli automi dopo la mossa, righe scritte sul database, messaggi partiti verso il campo e la web app. Il battito e la manutenzione (con lo stato che il nodo dichiara, tenuto anche quando la Centrale non lo accetta), la telemetria che porta condizione e fase del convoglio fino alla web app, anche a null nell'evento che parte senza telemetria, l'alert ripetuto, l'eco della Centrale, il convoglio rotto in stazione che la chiude, il rientro dal campo, il fail-stop e il treno fermo con le loro chiusure automatiche, i passaggi che misurano il ritardo, la soppressione, lo stato deciso dall'operatore, la validazione degli id (RF02.1, RF02.6, RF01.4)

Continued on next page

Continued from previous page

modulo	classe	test	che cosa verifica
Centrale	`PrevisioneArriviTest`	15	l'orario di servizio: la previsione somma tratte e soste, il ritardo nasce dal confronto con l'orario previsto e cresce anche per una sosta troppo lunga; cresce gia' mentre il treno e' fermo, in banchina, a meta' tratta e alla stazione di partenza, senza contare due volte lo stesso tempo quando arriva la misura del passaggio, e a corsa finita resta quello raggiunto (RF02.2.1, RF02.2.2.2)
Centrale	`ItinerariTratteTest`	9	un itinerario si appoggia solo su tratte esistenti; il rifiuto lascia l'itinerario com'era e non a metà; i treni assegnati alla creazione vengono avvisati (RF01.3.3, RF01.3.4)
Centrale	`PosizioneConvoglioTest`	8	quando la centrale sa dire dove sta un convoglio e quando ammette di non saperlo: fermo in stazione la posizione e' sempre nota, in linea serve la tratta confermata dai sensori letta nel verso giusto, e con una tratta ormai vecchia la partenza non si inventa (RF02.2.2.1, RF01.1.1)
Centrale	`RegressioniBugTest`	7	casi limite dell'anagrafica e dello snapshot: una PUT che non parla di assegnazioni non sgancia i convogli, un itinerario che ripassa sulla stessa tratta prende un 400, una tratta usata come posizione di un convoglio non si elimina, lo snapshot porta treni e stazioni

Continued on next page

Continued from previous page

modulo	classe	test	che cosa verifica
Centrale	`ChiusuraGuastiTest`	7	i due comandi del tecnico: la squadra non si manda su una stazione sana, l'invio la mette in manutenzione, ne' un alert ripetuto ne' il rientro dichiarato dal campo chiudono l'intervento, la chiusura dell'allarme la rimette in servizio (RF01.4.1, RF01.4.2)
Centrale	`MemoriaStoricaTest`	4	eliminare una stazione o un convoglio non cancella la loro storia (RF01.5, RF02.7)
Centrale	`AlertRitardoTest`	4	il ritardo misurato viaggia verso il convoglio con la causa giusta, e il rientro in orario non è un allarme (RF02.2.2.2)
Centrale	`DatabaseTest`	4	le transazioni JDBC: il lavoro riuscito va in commit, un'eccezione annulla tutto, una transazione dentro un'altra e' indipendente, lo schema si puo' rieseguire senza toccare i dati
Centrale	`GuastoTest`	2	i campi del guasto e la distinzione fra allarme segnalato e dedotto (RF01.2.2)
Centrale	`AvvioTest`	1	la Centrale di prova si accende, crea le tabelle e risponde
Centrale	`TrafficLogicEngineTest`	1	un intervento aperto allo spegnimento riprende all'avvio: la stazione torna in MANUTENZIONE e non OFFLINE (RF01.4.1)
Centrale	`RealtimeWebSocketTest`	1	la WebSocket vera di Javalin: un client si collega come la web app e riceve i broadcast (RF01.1.4)

Continued on next page

Continued from previous page

modulo	classe	test	che cosa verifica
Treni	`RiaggancioItinerarioTest`	9	il riaggancio dopo un cambio di itinerario: il treno resta dov'è, si inverte all'estremo nuovo, aspetta se è in mezzo a una tratta, chi ha chiuso la corsa riparte solo se il capolinea è stato tolto, e chi era trattenuto resta fermo o riparte secondo lo stato delle fermate arrivato insieme al percorso (RF02.5, RF01.3.4, SV05)
Treni	`GestoreEventiTest`	7	le due variabili di stato del convoglio non si cancellano a vicenda: il blocco di una stazione non toglie l'EMERGENZA, la soppressione resiste a un itinerario nuovo e a un guasto di bordo, il guasto della stazione di capolinea non riapre la corsa; e la ENTRATA parte dopo le mosse, quindi con la direzione gia' invertita; all'arrivo con un percorso nuovo che passa due volte dalla stessa stazione il riaggancio sceglie l'occorrenza vicina alla stazione di arrivo (RF02.1.1.2, RF02.5.1, RF02.2.3)
Treni	`StazioneInManutenzioneTest`	6	la stazione in manutenzione ferma il convoglio in arrivo e tiene fermo quello che è già dentro; il guasto risolto non basta se la squadra è ancora sul posto, mentre la stazione dichiarata agibile fa ripartire anche prima della chiusura dell'allarme; la manutenzione di un'altra stazione non lo riguarda (RF01.4.1, RF02.1.1.2)

Continued on next page

Continued from previous page

modulo	classe	test	che cosa verifica
Treni	`TelemetriaTest`	1	il frame di telemetria porta, oltre allo stato riassunto, la condizione e la fase del viaggio: il treno trattenuto a meta' tratta e' FERMO, REGOLARE e BLOCCATO$_{\text{INTRATTA}}$ (RF02.2.2.3)
Treni	`RitardoComunicatoTest`	3	il ritardo che la Centrale comunica finisce a bordo, il rientro lo azzera, quello di un altro convoglio viene ignorato
Treni	`ExampleTest`	2	il guasto di bordo esce come alert nel formato giusto, con una coda finta al posto del broker, e resta un JSON valido anche con le virgolette nella descrizione
Stazioni	`GestoreEventiTest`	11	gli automi della stazione: il sensore muto la guasta una volta sola e il suo ritorno la rimette online, ma non cancella un guasto che non e' suo (segnalato o della Centrale); al riavvio la causa si ricostruisce dal registro; il treno che entra in una stazione guasta viene trattenuto; il ripristino della Centrale ricomincia a sorvegliare i sensori; la stazione non reagisce all'eco dei propri allarmi (RF02.3.3, RF02.3.4, RF02.1.1.2.3)
Stazioni	`SensoriFailStopTest`	5	i sensori di binario, con le API REST vere su una porta libera: si chiamano con l'id della stazione, battono via REST, il sensore muto la guasta e il suo ritorno la rimette operativa, il sensore fermo non rileva il passaggio, le richieste sbagliate sono rifiutate (RF02.3.3, RF03.4)

Continued on next page

Continued from previous page

modulo	classe	test	che cosa verifica
Stazioni	`ConsumatoreBufferTest`	4	lo store and forward: gli eventi partono in ordine sul topic del loro canale, a rete staccata restano in coda e partono al ritorno, col broker giu' la testa non si perde e non viene sorpassata, e gli eventi non spediti sopravvivono a un riavvio (RF02.1.1.1.1, RF02.1.1.1.2)
Web app	`TrafficMapPage`, `apiClient`, `Card`	27	il rendering della mappa, le chiamate con il token e i componenti di base; i convogli a posizione incerta, elencati e non disegnati finche' partenza e arrivo non tornano noti; lo stato composto: il treno trattenuto a meta' tratta e' REGOLARE, $\text{BLOCCATO}_{\text{INTRATTA}}$, ANDATA e non "in stazione", senza itinerario e al capolinea non c'e' verso, senza telemetria resta lo stato della Centrale, la stazione in manutenzione mostra anche lo stato del nodo (RF01.1.2, RF02.2.2.3)
Web app	`CampoNumerico`	3	il campo numerico dei form di amministrazione: lo 0 di partenza non resta davanti alle cifre scritte, il campo svuotato resta vuoto, il valore segue le modifiche che arrivano da fuori (RF01.3)

3.3.1 Le prove dei casi limite

`RegressioniBugTest` merita una riga a parte perché non nasce da un requisito ma da un caso limite: i nomi dei metodi sono l'enunciato del comportamento che garantiscono — `modificareLeStazioniNonSganciaITreniAssegnati`, `itinerarioCheRipassaSullaStessaTrattaVieneRifiutato`, `trattaUsataComePosizioneDiUnTrenoN`

3.4 Esecuzione degli scenari

Gli scenari sono enunciati nel capitolo *Scenari di verifica* con il comportamento atteso passo per passo; qui sta il modo in cui vengono provocati sul sistema acceso. Le leve sono gli endpoint REST dei nodi di campo, cioè la stessa porta da cui entrerebbero i sensori veri (RF03.4), più la console interattiva `Sensori` dei due nodi.

scenario	come si provoca	dove si osserva l'esito
SV01	`POST /stazione/sensore/guasto` sulla stazione B, poi i due comandi del tecnico dalla dashboard	schermata delle stazioni e degli allarmi; i due convogli si fermano e ripartono da soli
SV02	`POST /stazione/rete/offline` su A, si fanno transitare dei convogli, poi `POST /stazione/rete/online`	`GET /stazione/buffer` durante l'interruzione, lo storico dei transiti dopo
SV03	`POST /treno/sensore/guasto` su due convogli, uno in sosta e uno in tratta	l'elenco degli allarmi: la sorgente è la stazione nel primo caso, la tratta nel secondo
SV04	si ferma un convoglio fuori stazione e si aspetta `T_fer`	l'allarme compare da solo ed è marcato come dedotto
SV05	si modifica l'itinerario dalla dashboard a convogli in marcia	posizione e ritardo dei due convogli prima e dopo
SV06	si lascia percorrere una corsa intera, andata e ritorno; poi si marca il capolinea su un estremo e si lascia arrivare il convoglio	lo storico dei transiti (entrate e uscite nell'ordine reale) e, dopo il capolinea, il treno fermo con l'entrata rimasta senza uscita

3.5 Verifica dei requisiti non funzionali

#	requisito	come si misura
RNF01	prontezza	si cronometra fra la chiamata al sensore e la comparsa sulla schermata
RNF02	nessuna perdita	si contano i transiti provocati durante SV02 e li si ritrova tutti nello storico
RNF03	degradazione	si spengono i nodi uno alla volta e si annota che cosa resta funzionante
RNF04	riservatezza	cattura del traffico sotto profilo `%tls`: il payload non è leggibile
RNF05	numerosità	si avviano più istanze di stazione e di treno con id diversi

Continued on next page

Continued from previous page

#	requisito	come si misura
RNF06	riavviabilità	si arresta e si riavvia un nodo a sistema acceso, senza toccare nient'altro

3.6 Limiti della verifica

Due cose vanno dette, perché non sono coperte.

I test automatici si fermano al confine del processo verificano gli endpoint e la logica interna di ogni nodo, con il broker sostituito da una coda finta, ma non la conversazione via broker fra processi separati. Che la telemetria pubblicata dal treno arrivi davvero in Centrale e' verificato dagli scenari a mano, non da un test.

Scenari e requisiti non funzionali sono verificati a mano si rieseguono a ogni modifica solo se qualcuno si ricorda di farlo.